

Vorlesung

Mensch-Maschine-Interaktion

WS 2024/2025



Kapitel E.

Computergraphik und OpenGL

Priv.-Doz. Dr. Frank Weichert
Lehrstuhl für Computergraphik (Informatik VII)
Technische Universität Dortmund
<https://graphics.cs.tu-dortmund.de>

Hinweis:

„Der Nutzerin/dem Nutzer ist bekannt, dass die nachfolgenden Inhalte und Materialien urheberrechtlich geschützt sind. Die Nutzung ist ausschließlich für den persönlichen Gebrauch im Rahmen von universitären Zwecken zulässig. Insbesondere ist die Vervielfältigung, Bearbeitung, Verbreitung und jede Art der Verwertung sowie die Weitergabe an Dritte nicht gestattet. Zuwiderhandlungen werden zivil- und strafrechtlich verfolgt.“

E. Computergraphik und OpenGL

Übersicht

E.1. Graphik-Pipeline-basierte Bilderzeugung

- E.1.1. Einleitung
- E.1.2. Graphik-Pipeline
- E.1.3. Grundkonzepte von OpenGL
- E.1.4. Pufferspeicherobjekte
- E.1.5. Shader
- E.1.6. Vereinfachte OpenGL-Shader Pipeline
- E.1.7. Einbindung von Shader-Programmen

E.2. Szenengraphen

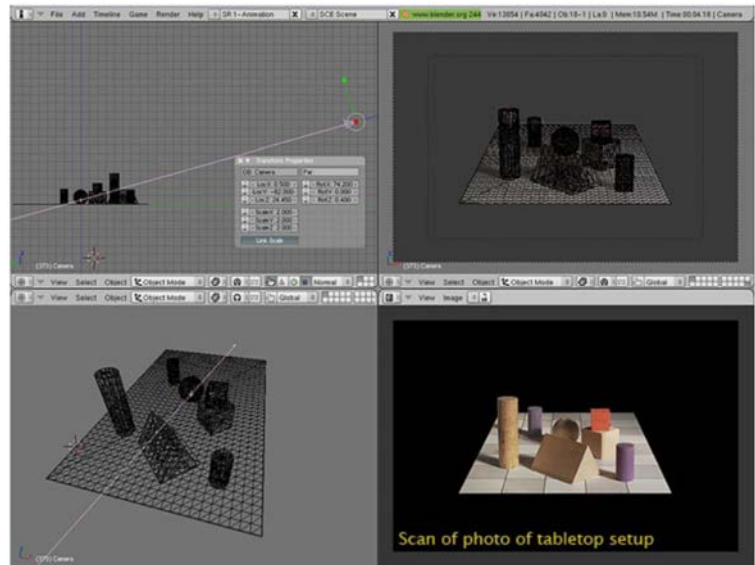
- E.2.1. Einleitung
- E.2.2. Aufbau von Szenengraphen
- E.2.3. Java 3D Szenengraphen
- E.2.4. Beispiel: HelloJava3Da
- E.2.5. Virtuelle und reale Szene

E.3. Strahlverfolgungsbasierte Bilderzeugung

- E.3.1. Einführung
- E.3.2. Strahlverfolgungsverfahren
- E.3.3. Anti-Alias-Behandlung
- E.3.4. Strahl-Dreieck-Schnitt
- E.3.5. Spezifikation von Eingabeszenen
- E.3.6. Effiziente Strahlverfolgung
- E.3.7. Fotorealistisches Rendering

Graphik-Pipeline

- Geometrie + Transformationen
- Kameras und Sichtbarkeit
- Beleuchtung und Schattierung
- Rasterisierung
- Texturierung



<https://www.blender.org>

Graphik-Pipeline

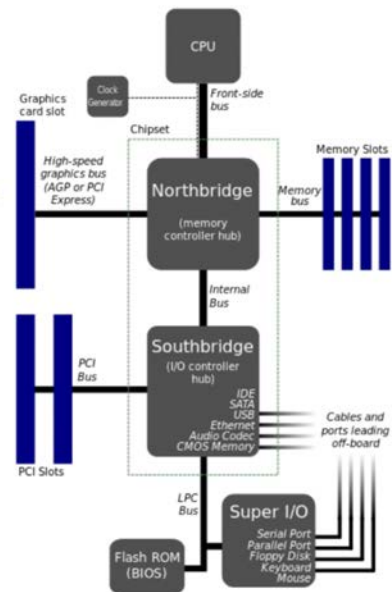
- monolithischen Grafik-Workstations der 80er Jahre wurden durch modulare GPUs (Graphics Processing Units) ersetzt: NVIDIA, AMD, Intel
- frühe Versionen dieser GPUs implementierten eine Rendering-Pipeline mit festen Funktionen (*fixed-function rendering*) in Hardware
- GPUs wurden ab den späten 90er Jahren programmierbar
z.B. 2001: Nvidia GeForce 3 (erste programmierbare Shader)



- heutige GPUs: programmierbare (z.B. OpenGL, CUDA, OpenCL) Prozessoren

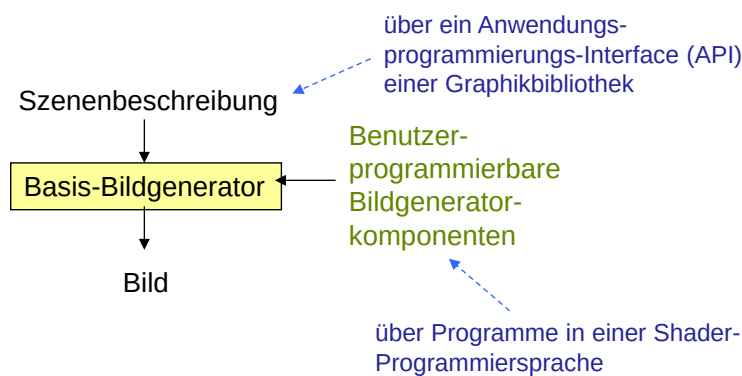
Graphik-Pipeline

- GPU: massiv paralleler Prozessor



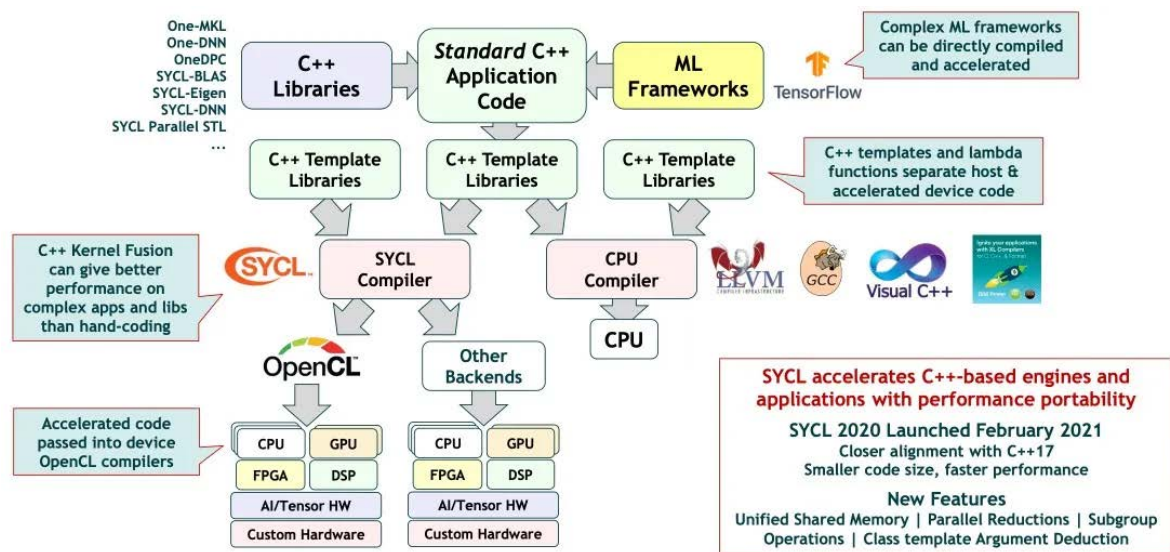
- OpenGL ist die/eine Schnittstelle zur GPU

Grobstruktur aus Software-Sicht



- Bsp. für Graphik-Bibliotheken: **OpenGL**, Vulkan, DirectX, OpenInventor, **Java3D**
- Bsp. für Shader-Programmiersprachen: **GLSL (OpenGL)**, HLSL (Microsoft), Cg (NVIDIA)

Standard im Zusammenhang mit Bildverarbeitung und KI ist SYCL



OpenGL

- OpenGL ist ein Software-Interface für Graphik-Hardware
→ Hardware-unabhängig, Hersteller-unabhängig
- OpenGL ist eine registrierte Trademark von SGI
- OpenGL wird vom „ARB“ (Architecture Review Board) überwacht
- „Open“?
 - Weiterentwicklung durch die Khronos-Gruppe (<https://www.khronos.org/opengl/>)
NVIDIA, AMD (ATI), IBM, Intel, SGI,
 - Von jedem Lizenznehmer erweiterbar (mittels Extensions)
- Keine Handhabung von Fenstern/Windows, Benutzereingabe
- Der Standard im embedded Bereich (OpenGL ES)

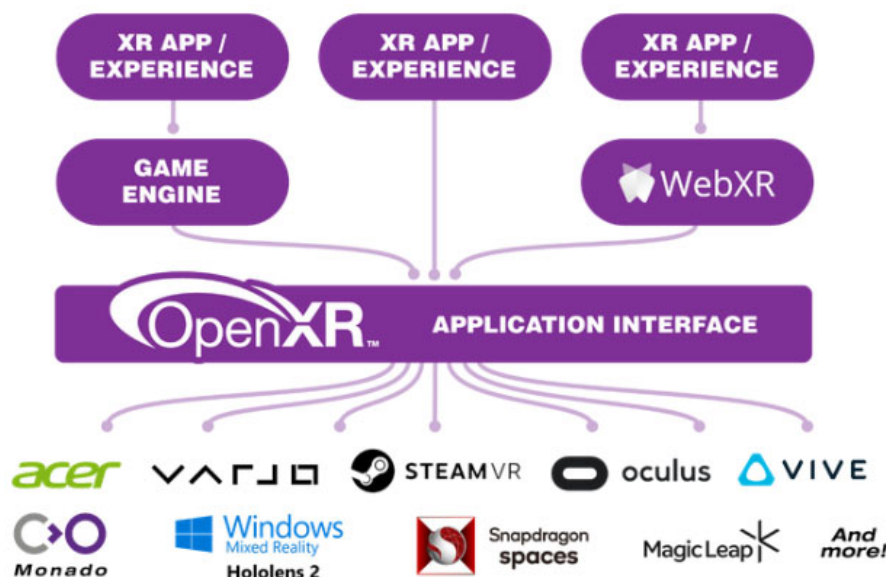
Aktuelle Version: OpenGL 4.6 (API-Update 2022)

“Khronos free open standards for 3D graphics, Virtual and Augmented Reality, Parallel Computing, Machine Learning, and Vision Processing”



<https://www.khronos.org>

OpenXR (s. Kapitel H. “Virtuelle/Augmentierte Realität”)



<https://www.khronos.org>



OpenGL-Varianten

OpenGL ES

- für eingebettete und handgehaltene Geräte wie Mobiltelefone entwickelt – optimiert auf geringen Energieverbrauch durch einen etwas reduzierten Funktionsumfang und speichereffizientere und Hardware-angepasste Implementierung
- Aktuelle Version: OpenGL ES 3.2, beruht auf OpenGL 3.x

INFORMATION:

<https://www.khronos.org/opengles/>



WebGL

- WebGL (Web Graphics Library) ist ein Bestandteil von Web-Browsern, mit dessen Hilfe hardwarebeschleunigte 3D-Grafiken direkt im Browser, ohne zusätzliche Erweiterungen, dargestellt werden können.
- realisiert als JavaScript Implementierung von OpenGL ES 2.0
- läuft auf den meisten aktuellen Browsern

https://www.khronos.org/webgl/wiki/Main_Page

OpenGL Extensions

Ziel: Erweiterungen in OpenGL-Implementierungen, die einem definierten Erweiterungsmechanismus folgen und registriert sind
(<http://www.opengl.org/registry>)

Zweck: OpenGL-Systemhersteller und Hardware-Herstellern die Möglichkeit bieten, neue Funktionalitäten verfügbar zu machen.



GLUT

- ist ein fenstersystemunabhängiges Werkzeug, das eine Schnittstelle zum umgebenden Fenstersystem, unter Anderem zur Eingabe, zur Verfügung stellt.
- Die GLUT-Funktionen sind mit dem Präfix **glut** gekennzeichnet.
- Wesentliche verfügbare Versionen:
 - *Freeglut*, von Pawel W. Olszta mit Beiträgen von Andreas Umbach and Steve Baker, ist die aktuellste Version
 - *Originalversion*, von Mark Kilgard, ist etwas veraltet, wird aber wohl von Apple in der aktuellsten Version 3.7 verwendet.

QUELLE: The Open GL Programming Guide, 8. Auflage, Anhang A



Weitere OpenGL-Extensions

GLU – OpenGL Utility Library

enthält zusätzliche Funktionen zum Zusammensetzen von Matrizen für spezielle Ansichtsorientierungen und Projektionen, zur Polygonzerlegung und zur Darstellung von Flächen, z.B. von B-Spline-Flächen (NURBS-Flächen). Die GLU-Funktionen sind mit dem Präfix **glu** gekennzeichnet.
→ In den aktuellen OpenGL Versionen wird GLU nicht mehr unterstützt
Ersatz durch externe Bibliotheken, glm (<https://github.com/g-truc/glm>)

GLEW

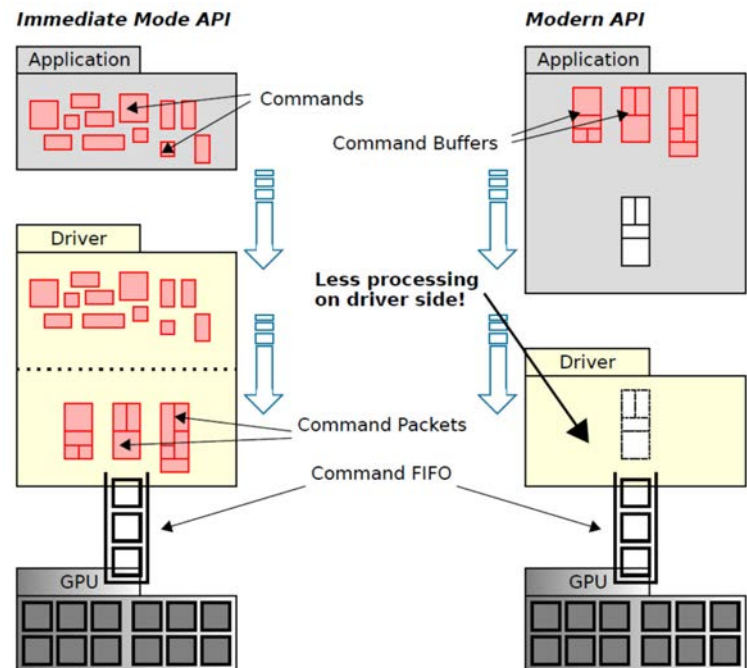
GLEW stellt einen effizienten Laufzeitmechanismus zur Verfügung, um herauszufinden, welche OpenGL-Extensions von der Zielplattform unterstützt werden.

GLEW Homepage: <http://glew.sourceforge.net>

Vulkan

Treiber Overhead

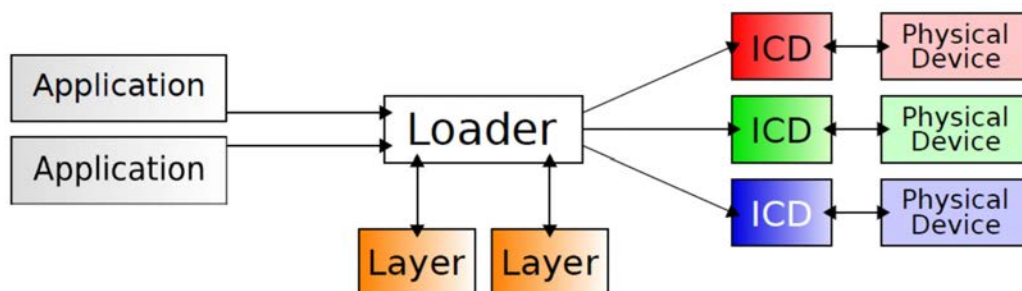
- Immediate Mode APIs: OpenGL
- Moderne APIs: Vulkan



Vulkan

Loader als Schnittstelle zwischen Applikation und Treiber

- andere APIs: Runtime, die API-Calls direkt an Treiber weiterleitet
- Vulkan: Applikation linked mit Loader
- Anwendung kommuniziert nicht direkt mit Treiber (Vulkan: installable client driver (ICD)), sondern mit Loader.
- Loader kann bestimmte Logik selbst abbilden (Windows Registry, JSON Manifest Files etc.), ohne mit Treiber zu kommunizieren oder in Kernel-Mode zu wechseln.



Literatur

- Mark Segal, Kurt Akeley, The OpenGL Graphics System: A Specification (Version 4.3 – August 6, 2012, Version 4.4 - October 18, 2013, Version 4.5, October 30, 2014), <https://www.khronos.org/opengl>
- J. Kessenich, D. Baldwin, R. Rost, The OpenGL Shading Language <https://www.khronos.org/opengl>
- The Open GL Programming Guide, Silicon Graphics International, Addison-Wesley, 1993 (historisch), 8. Auflage: OpenGL 4.3, 2013, 9. Auflage: OpenGL 4.5
- S. Kosarevsky, V. Latypov, 3D Graphics Rendering Cookbook: A comprehensive guide to exploring rendering algorithms in modern OpenGL and Vulkan, Packt Publishing, 2021
- Offizielle OpenGL-Web-Seite: <http://www.opengl.org>
- Inoffizielle, deutschsprachige Web-Site: <http://wiki.delphigl.com>
- OpenGL Shader-Tutorial: <http://www.lighthouse3d.com/tutorials/>

Graphik-Pipeline

- Allgemeine Aufgabe: Überführung von 3D-Inhalte auf einem 2D-Bildschirm
- Für eine echte 3D-Wahrnehmung werden zwei Bilder für die beiden Augen (Stereo) benötigt, → s. Kapitel H. „Virtuelle/Augmentierte Realität“
- (Problem:) Unterschiedliche Sichtweisen auf die zugehörige Pipeline



© Maarten Boot

Pixel-basierte Grafiken



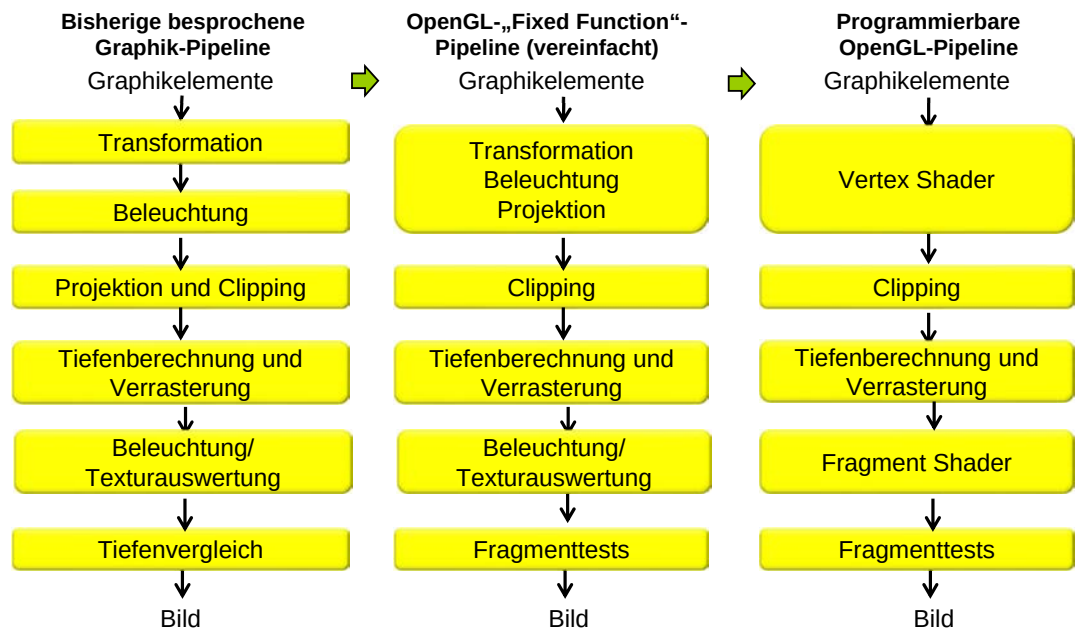
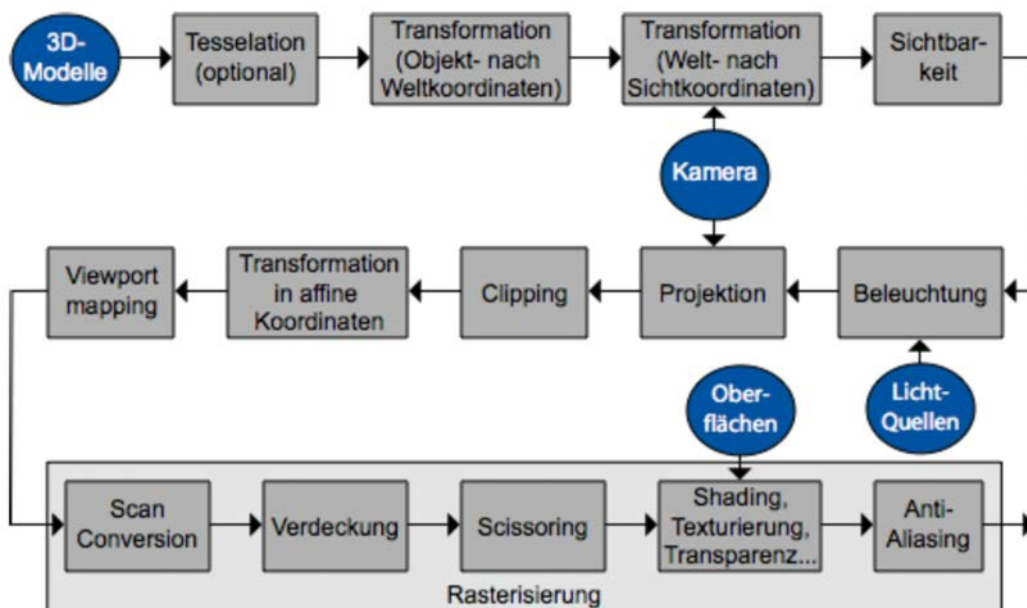
© KeithByrne (via DeviantArt)

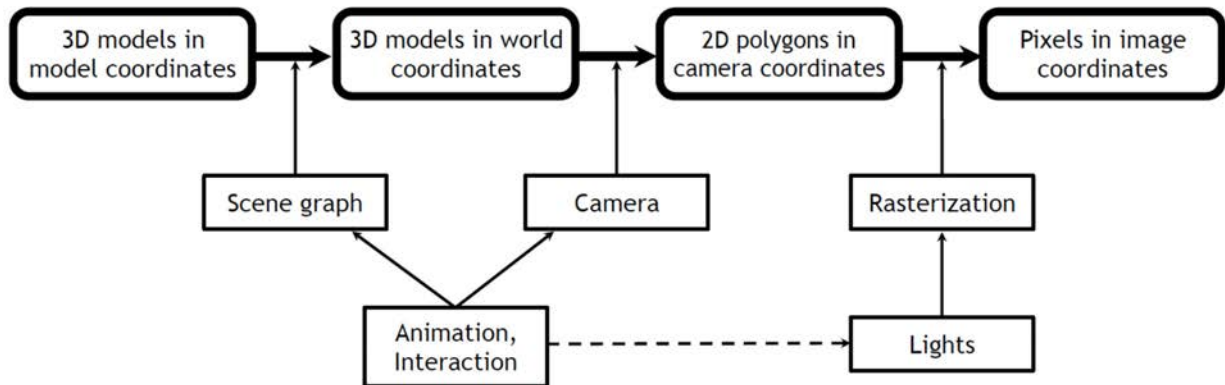
2D-Grafiken (Vektorgrafiken)

3D-Grafiken



© Crystal Dynamics / Square Enix

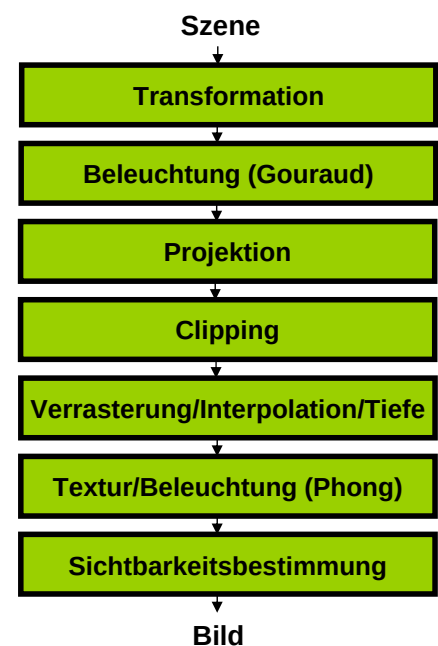
1. Sichtweise: **OpenGL-Pipeline Sichtweise**2. Sichtweise: **Detailliertere OpenGL-Pipeline Sichtweise**

3. Sichtweise: **Geometrie-zentrierte Sichtweise****Darstellung der Graphik-Pipeline**

Eingabe: Darzustellende Szene aus Polygonen

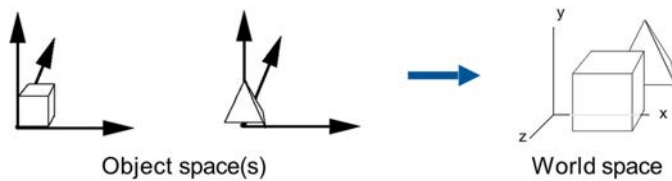
Ausgabe: Bild

1. **Transformation** der Szene in das Kamerakoordinatensystem
2. **Beleuchtungsberechnung** für die Polygoneckpunkte (Knoten) (falls Gouraud-Shading gewünscht)
3. **Projektion** auf die Bildebene der Kamera
4. **Clipping** an der Sichtpyramide der Kamera
5. **Verrasterung** (mit Interpolation der Eckpunktwerte und Tiefenberechnung)
6. **Beleuchtungsberechnung** für die Fragmente (**Texturierung**, sowie evtl. Phong-Shading, Fragmente = „Polygonpixel“)
7. **Sichtbarkeitsberechnung und Bildaufbau** mit dem Tiefenpufferalgorithmus.

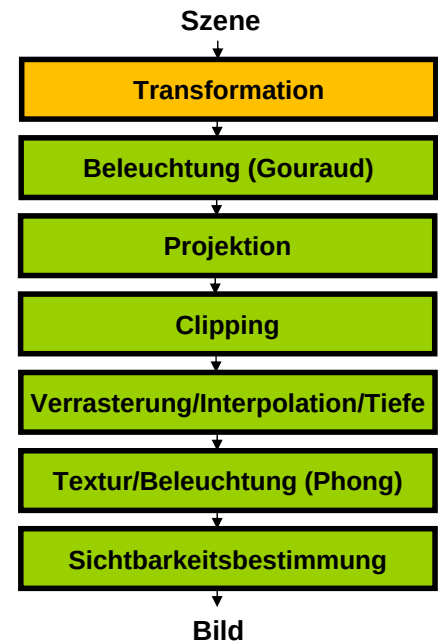


Darstellung der Graphik-Pipeline

- Jedes 3D Modell (Objekt) wird im eigenen Koordinatensystem definiert (**object space**)
- **Model Transformation** positioniert die Objekte in einem allg. Koordinatensystem (**world space**)

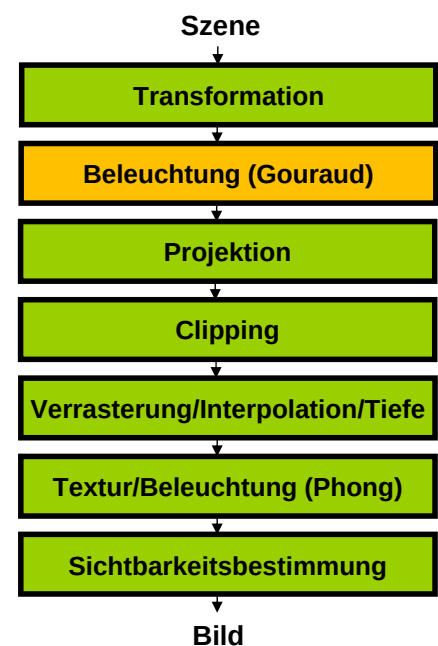
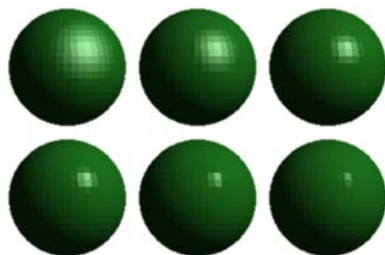


Quelle: Zachmann, Bremen



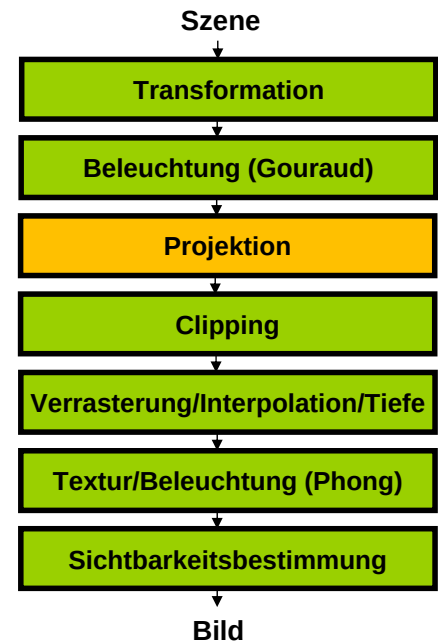
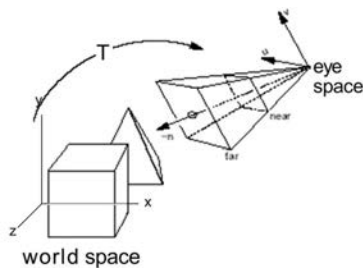
Darstellung der Graphik-Pipeline

- Beleuchten von Polygonen (für Schattierung und Highlights) gemäß der Material-Eigenschaften, Oberflächeneigenschaften und Lichtquellen
- Lokale Beleuchtungsmodelle (kein Lichttransport von einem Objekt zum anderen)



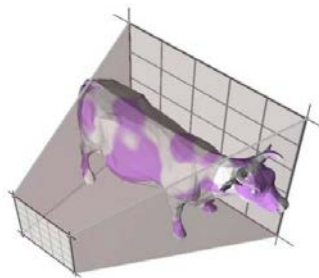
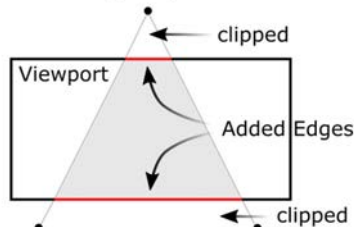
Darstellung der Graphik-Pipeline

- Umwandeln von **Welt-Koordinaten** nach **Kamera-Koordinaten**
- Bestimme eine Transformation für die komplette Szene, sodass die Betrachter-Position in den Ursprung verschoben wird und Blickrichtung entlang der (Z)-Achse liegt

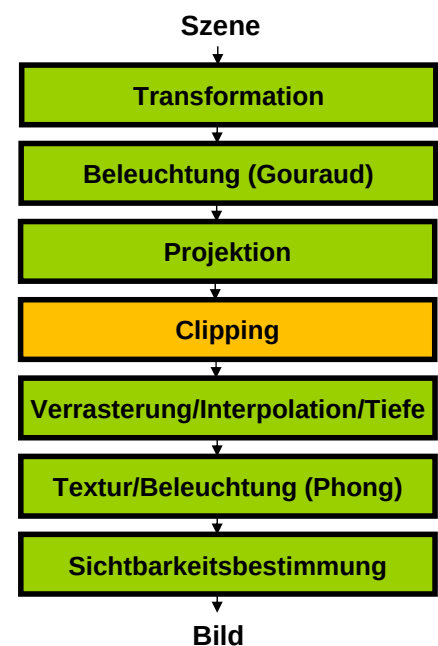
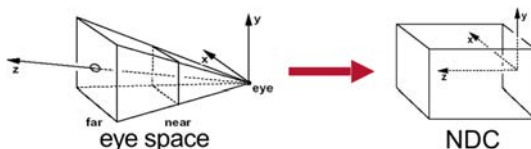


Darstellung der Graphik-Pipeline

- Abschneiden der Polygone, die außerhalb des sichtbaren Bereiches liegen (**view frustum**)

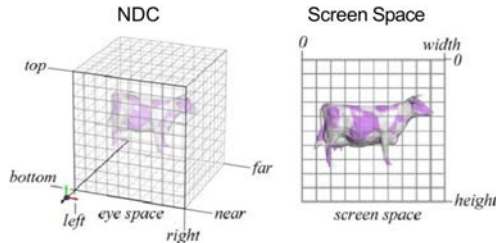


- Transformiere die Szene in **normalisierte Koordinaten** (Normalized Device Coordinates = NDC)

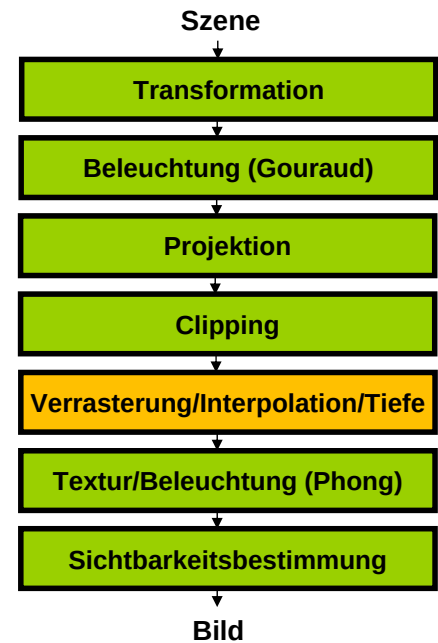
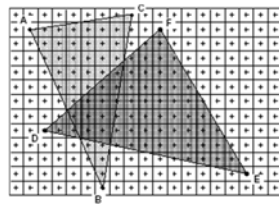


Darstellung der Graphik-Pipeline

- Objekte nach 2D projizieren und räumlich diskretisiert (screen space)

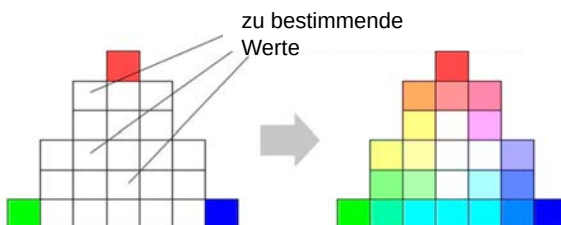


- Rasterisierung (= Diskretisierung) der Polygone in Pixel

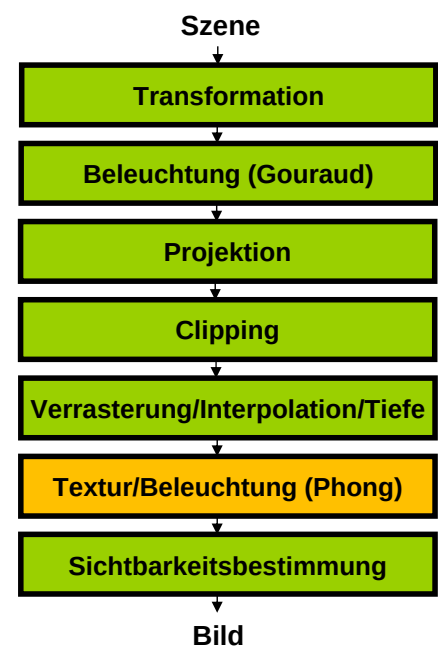
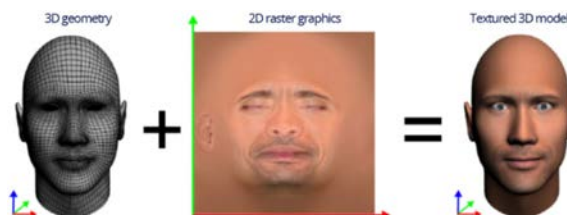


Darstellung der Graphik-Pipeline

- Bestimmung der Werte pro Pixel durch Interpolation der gegebenen Werte pro Vertex

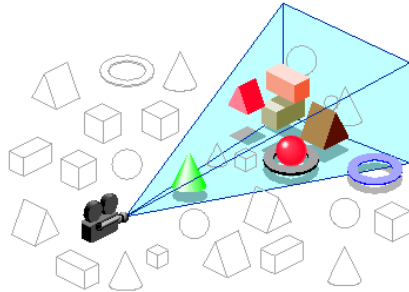


- 2D-Texturen sind 2D-Rastergrafiken, die auf 3D-Geometrien aufgebracht werden

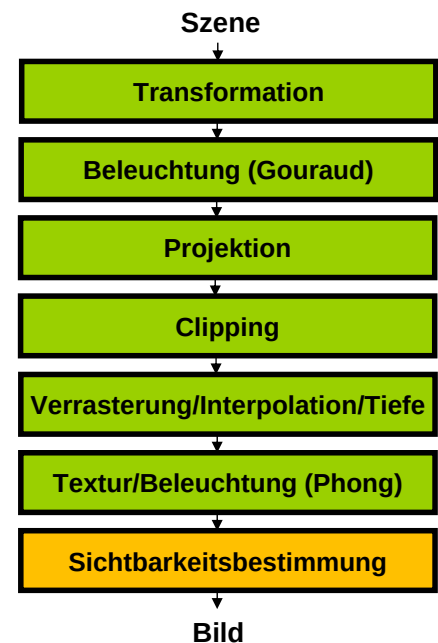


Darstellung der Graphik-Pipeline

- Verdeckungen bestimmen



- Evtl. weitere Pixel-Operationen:
 - Blending mit vorhandenem Frame-Buffer-Inhalt
 - Maskierung (z.B. wegen Verdeckung durch andere Fenster)
 - Farbtransfer

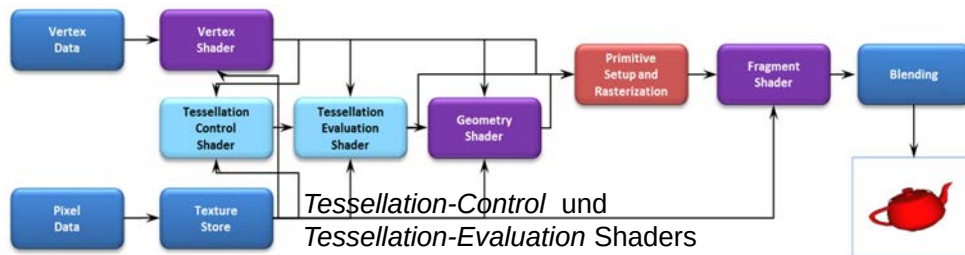


OpenGL

- Plattform-, hardware- und sprachunabhängige Programmierschnittstelle Schnittstelle für 3D-Grafikhardware (**3D Rendering API**)
 - ~200 Funktionen zur Definition von Geometrie und zur Ausführung von typischen 3D-Computergraphik-Aufgaben
 - Unabhängig vom Windowing-System (keine Ereignisse, Fenster, Menüs, ...)
- **OpenGL implementiert eine Grafik-Pipeline**
 - Geometrische Primitive: Punkte, Linien, Dreiecke, (bis OpenGL 3 auch Quads)
 - Puffer und Texturen
 - programmierbare Verarbeitungsschritte in der Pipeline (Shader)
 - frei programmierbare Verarbeitung von Geometrie, Schattierung, Tessellation, ...
- **Low-Level-API**
 - keine höheren Konzepte für Modellierung oder Animation
 - kein Szenengraph
- Erweiterungen ermöglichen den Zugriff auf bestimmte Fähigkeiten eines Grafikprozessors
- GL Shading Language (GLSL) für programmierbare GPUs

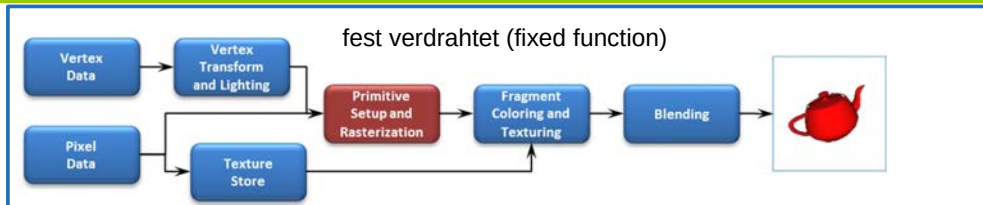
Grundkonzepte von OpenGL

- Programmierbare Graphik-Pipeline (ab OpenGL 2.0)
- Zustandsmaschine
- Objektkonzept
- Client-Server-Modell

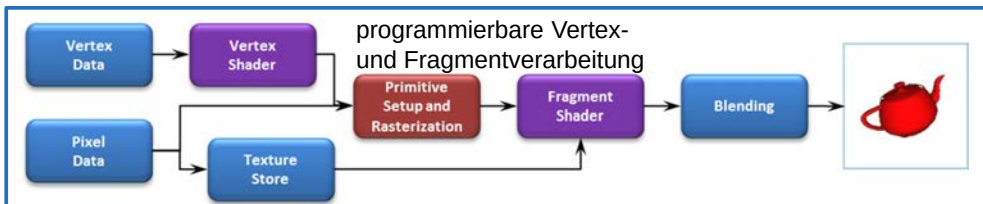


OpenGL 4.1(- 4.6)

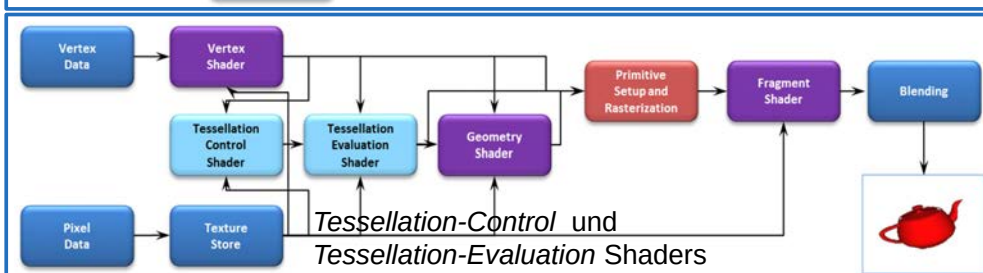
OpenGL 1.x



OpenGL 2.0



OpenGL 4.1(- 4.6)



Programmable Pipeline

Grundkonzepte von OpenGL

- Programmierbare Graphik-Pipeline (ab OpenGL 2.0)
- Zustandsmaschine
- Objektkonzept
- Client-Server-Modell

Client-Server-Modell:

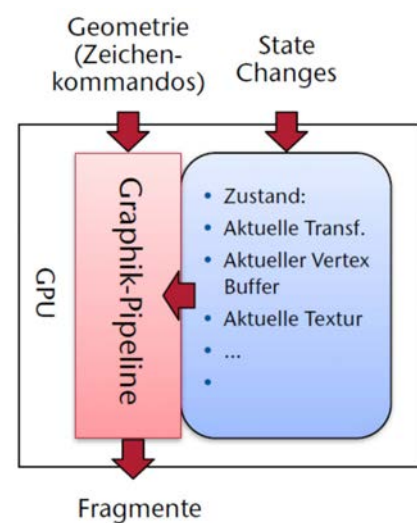
- Client: Anwendungsprogramm
- Server: OpenGL-Graphiksystem
- Speicherbereiche im Server: „Buffer“
 - OpenGL-Framebuffer-Objekt, Renderbuffer-Objekte
 - OpenGL-Buffer-Objekte: Speicherbereiche im Server

Weitere Ebenen zur Effektivierung, z.B.

- Vertex Buffer Objects, Vertex Array Objects
- OpenGL Shading Language (u.a. Vertex- und Fragment-Shader)

Zustandsmaschine - GPU als „State-Machine“

- „Maschine“ verbleibt so lange in einen Zustand, bis er wieder verändert wird
- Der Kontext von Operationen und damit deren Wirkung ist durch den aktuellen Zustand gegeben,
- OpenGL-Status setzen
`glEnable(...);`
`glDisable(...);`
`gl*(...); // verschiedene spezifische Zustände`
- Lesen von OpenGL-Zuständen
`glIsEnabled(...);`
`glGet*(...); // bestimmte Zustände`
 → um den aktuellen Zustand zu lesen, muss die Pipeline möglicherweise leer laufen



**Objektkonzept:**

- Objektklassen, die mit einer spezifischen Menge von Kommandos verbunden sind
 - Texturen, Geometrie, Shader usw. sind OpenGL-Objekte
 - Keine Objekte im Sinne von OOP
- Anlegen, Initialisieren und Löschen von Objekten
 - Erzeugen eines **Handle** (in OpenGL auch **name** genannt)
`GLuint someHandle;`
`glGen{Texturen|Puffer|...}(1, &someHandle);`
 - Objekt auswählen/aktivieren (Bindung) - Direct State Access
`glBind*(<Ziel>, someHandle);`
 - Übergabe der Daten an OpenGL (keine erneute Übergabe erforderlich, solange sich die Daten nicht ändern)
 - Deselektieren eines Objekts (Aufheben der Bindung), z.B. `glBind*(<Ziel>, 0);`
 - Sofern obsolet: Objekt löschen, z.B. `glDelete*(1, &someHandle);`

**Grundstruktur von OpenGL-Programmen**

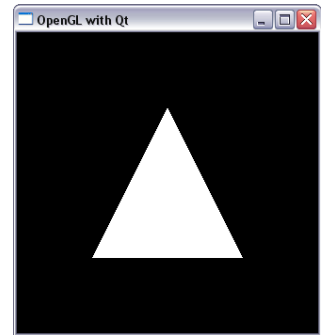
- **Drei Arten von Funktionen**
 - Zustand ändern
 - Daten (z.B. Vertex-Koordinaten) übergeben
 - Geometrie darstellen
- **Klarer Namensraum**
 - Befehle fangen mit `gl...` an
 - Konstanten mit `GL...`
- **Umfassende Erweiterungen**
 - Glm: Mathematische Funktionen (Vektoren, Matrizen, ...)
 - glew/glfw/...: Libraries zur Benutzerinteraktion, Fenstermanagement
 - Qt: umfassende GUI- und Interaktions-Elemente

Einfaches OpenGL-Programm

```

class Renderer {
public:
    void init() {}
    void resize(int w, int h) { glViewport(0, 0, w, h); }
    void display() {
        glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        glLoadIdentity();
        glOrtho(-1.0f, 1.0f, -1.0f, 1.0f, -1.0f, 1.0f);
        glColor3f(1.0f, 1.0f, 1.0f);
        glBegin(GL_TRIANGLES);
        glVertex3f(-0.5f, -0.5f, 0.0f); // vertex on the left
        glVertex3f( 0.5f, -0.5f, 0.0f); // vertex on the right
        glVertex3f( 0.0f,  0.5f, 0.0f); // vertex at the top of the triangle
        glEnd();
    }
    void dispose() {}
};

```

**Immediate Mode („direkter Modus“)**

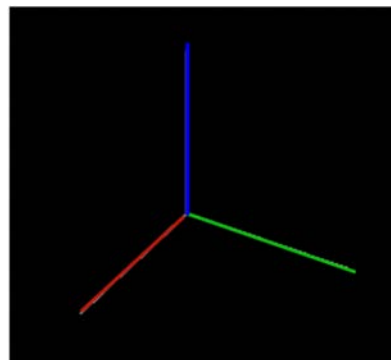
- Primitive werden sofort, wenn sie festgelegt sind, an das Display geschickt (Standard)
- GPU kennt immer nur die Primitive, die aktuell durch die Pipeline laufen

Beispiel

```

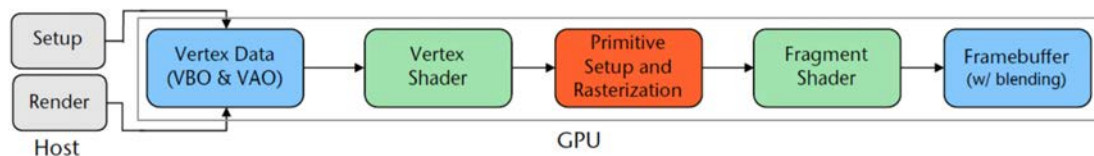
glBegin( GL_LINES );
glColor3f ( 1.0, 0.0, 0.0 );
glVertex3f( 0.0, 0.0, 0.0 );
glVertex3f( 1.0, 0.0, 0.0 );
glColor3f ( 0.0, 1.0, 0.0 );
glVertex3f( 0.0, 0.0, 0.0 );
glVertex3f( 0.0, 1.0, 0.0 );
glColor3f ( 0.0, 0.0, 1.0 );
glVertex3f( 0.0, 0.0, 0.0 );
glVertex3f( 0.0, 0.0, 1.0 );
glEnd();

```



Retained Mode („zurückhaltender Modus“)

- Primitive werden in sog. Buffer Objects gespeichert
Vertex Buffer Objects = VBO, Vertex Array Objects = VAO
- VBO's können auf der GPU (Graphikkarte) gehalten werden → Performance
- Pipeline:



- Mehrfaches Rendern mit unterschiedlichen Eigenschaften

Buffer Objects (Pufferspeicherobjekte)

- In der „Basisversion“ sind für das Zeichnen grafischer Primitive (wie Polygone, Linien oder Punkte) zahlreiche Funktionsaufrufe erforderlich
- Jeder Funktionsaufruf überträgt dabei nur eine kleine Menge an Daten an die OpenGL-Pipeline

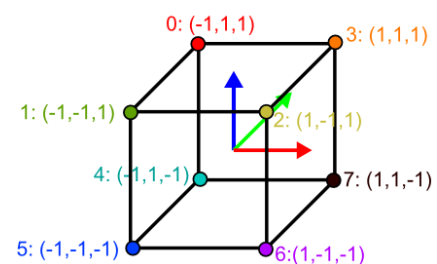
Beispiel (Würfel):

```

float vertexData[] =
{
  -1.0f,  1.0f,  1.0f,
  -1.0f, -1.0f,  1.0f,
   1.0f, -1.0f,  1.0f,
   1.0f,  1.0f,  1.0f,
  -1.0f,  1.0f, -1.0f,
  -1.0f, -1.0f, -1.0f,
   1.0f, -1.0f, -1.0f,
   1.0f,  1.0f, -1.0f
};
  
```

```

int indicesData[] =
{
  0, 1, 2, 3,
  0, 4, 5, 1,
  4, 7, 6, 5,
  2, 6, 7, 3,
  1, 5, 6, 2,
  3, 7, 4, 0
};
  
```



Buffer Objects (Pufferspeicherobjekte)

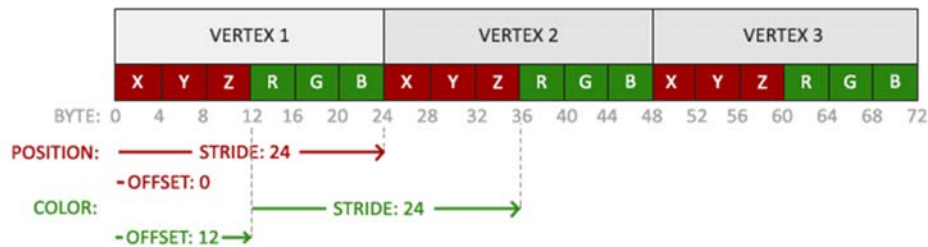
- „Basisversion“ führt zu einer langsamen Verarbeitung, da die Operationen der Grafikkarte nicht parallel auf eine große Anzahl von Daten ausgeführt werden können, sondern die Hardware ständig auf Daten wartet
- Daher werden seit OpenGL >= 3.1 die entsprechenden Funktionen (`glBegin`, `glVertex`, etc.) nur noch im „Compatibility Profile“ unterstützt
- Vertex-Daten nun in größeren Blöcken übertragen, die auf der Grafikkarte mit Hilfe von **Buffer Objects (Vertex Buffer Objects)** gespeichert und verwaltet werden
- Zweck: Speichern von Daten, ursprünglich speziell Vertex-Array-Daten, auf dem Server (Grafikkarte).

A. Installieren von Vertex Buffer Objects (VBOs)

- Erstellung erfolgt in der Regel vor dem eigentlichen Rendering, z.B. in der Funktion `init()`
- *Drei Schritte:*
 1. **Schritt** : Die Funktion `glGenBuffers(1, &bufID)` erzeugt einen eindeutigen Bezeichner
 2. **Schritt**: Mit `glBindBuffer(GL_ARRAY_BUFFER, bufID)` wird ein neuer VBO erstellt. Mögliche Ziele sind `GL_ARRAY_BUFFER` oder `GL_ELEMENT_ARRAY_BUFFER`
 3. **Schritt**: Die Funktion `glBufferData(GL_ARRAY_BUFFER, size, inData, GL_STATIC_DRAW)` überträgt die Vertex-Daten

B. Aktivieren von Vertex Buffer Objects (VBOs)

- *Schritt 1:* Binden des Puffers mit `glBindBuffer(GL_ARRAY_BUFFER, bufID)`
- *Schritt 2:* Definieren eines **Stride** und einer Startadresse für den Speicherzugriff



```
// position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// color attribute
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
```

B. Aktivieren von Vertex Buffer Objects (VBOs)

- *Schritt 1:* Binden des Puffers mit `glBindBuffer(GL_ARRAY_BUFFER, bufID)`
- *Schritt 2:* Definieren eines Stride und einer Startadresse für den Speicherzugriff
- *Schritt 3:* Aktivieren des Puffers für einen bestimmten Zweck `glEnableClientState(GL_VERTEX_ARRAY)`.
Mögliche Parameter sind: `GL_VERTEX_ARRAY`, `GL_COLOR_ARRAY`, `GL_SECONDARY_COLOR_ARRAY`, `GL_INDEX_ARRAY`, `GL_NORMAL_ARRAY`, `GL_TEXTURE_COORD_ARRAY` usw.

C. Zeichnen (Rendering) von Vertex Buffer Objects (VBOs)

- Nach dem Aktivieren eines VBOs wird das Rendering aller enthaltenen Daten mit einem einzigen Funktionsaufruf gestartet:

```
glDrawArrays(GL_POLYGON, first, count);
```

- Mögliche Argumente sind die gleichen wie bei `glBegin(...)`, wie `GL_POLYGON`, `GL_LINE_LOOP`, `GL_LINES`, `GL_POINTS`, etc.
- Wenn der VBO nicht mehr verwendet wird, kann der Speicher durch `glDeleteBuffers(1, &bufID)` freigegeben werden.

Vertex Array Objects (VAO)

- erlauben das Anlegen der Vertex Arrays mehrerer (Teil-)Szenen. Sie können nach Bedarf angebunden werden, wodurch ihre Daten zum Schreiben und Darstellen zur Verfügung stehen.
- Ein VAO enthält nur die Zustände, aber keine Vertex-Daten. Die Vertex-Daten sind weiterhin in den VBOs abgelegt.

Schritte zum Installieren von Vertex Array Objects:

1. Erzeugung von VAO-Namen
2. Anbinden eines VAO-Objects
3. Assoziieren von Buffer Objects mit dem VAO.

Aktivieren von bereits vorhandenen Vertex Array Objects:

durch Aufruf der Anbindefunktion mit deren Objekt Nummer.

**Zu 1. Erzeugung von VAO-Namen:**

```
void glGenVertexArrays(GLsizei n, GLuint *arrays);
GLboolean gllsVertexArray(GLuint array);
void glDeleteVertexArrays(GLsizei n, GLuint *arrays);
```

Verwendung wie bei Buffer Objects

Zu 2. Anbindung eines VAO:

```
void glBindVertexArray(GLuint array);
```

Verwendung wie bei Buffer Objects

Zu 3. Assoziieren von Buffer Objects mit dem VAO:

Buffer Objects, die im Zustand eines angebindenen VAO angebinden werden, werden mit diesem VAO assoziiert.

**Buffer Objects (Pufferspeicherobjekte)****VBO**

```
float points[] = { 0.0f, 0.5f, 0.0f, 0.5f, -0.5f, 0.0f, -0.5f, -0.5f, 0.0f };
float colours[] = { 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f };

GLuint points_vbo = 0; glGenBuffers(1, &points_vbo);
glBindBuffer(GL_ARRAY_BUFFER, points_vbo);
glBufferData(GL_ARRAY_BUFFER, 9 * sizeof(float), points, GL_STATIC_DRAW);

GLuint colours_vbo = 0; glGenBuffers(1, &colours_vbo);
glBindBuffer(GL_ARRAY_BUFFER, colours_vbo);
glBufferData(GL_ARRAY_BUFFER, 9 * sizeof(float), colours, GL_STATIC_DRAW);
```

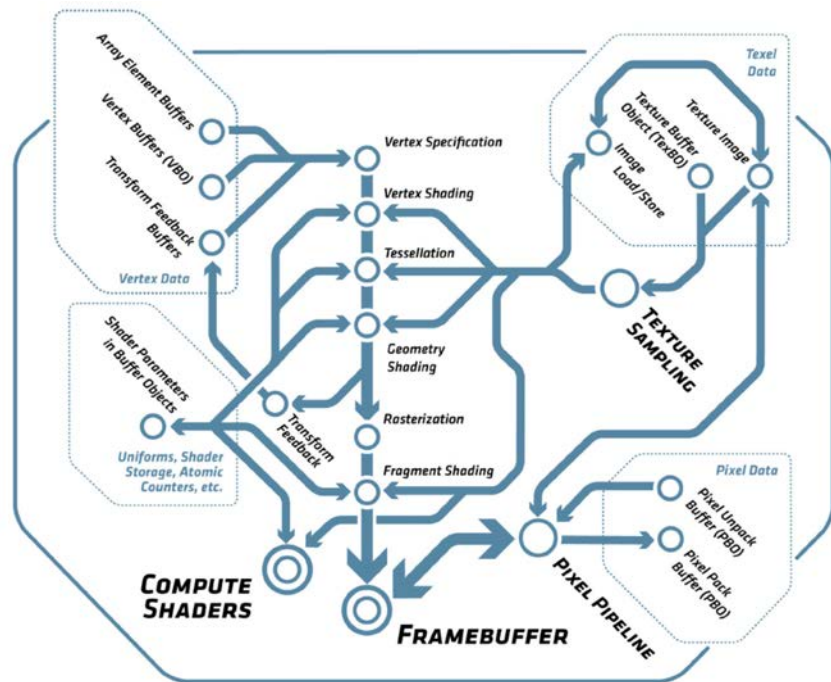
VAO

```
GLuint vao = 0; glGenVertexArrays(1, &vao);
glBindVertexArray(vao); glBindBuffer(GL_ARRAY_BUFFER, points_vbo);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, NULL);
glBindBuffer(GL_ARRAY_BUFFER, colours_vbo);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, NULL);

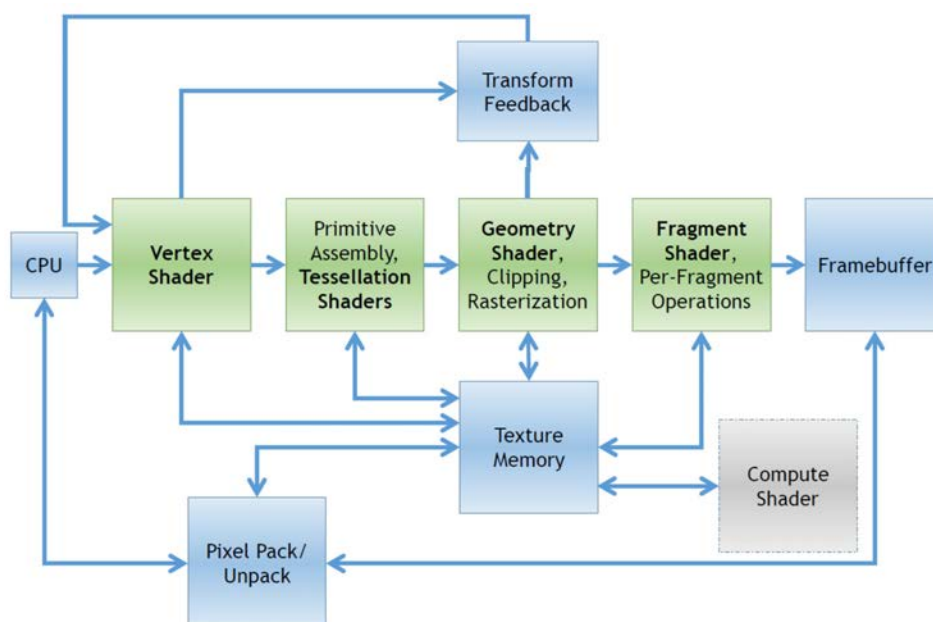
glEnableVertexAttribArray(0);
glEnableVertexAttribArray(1);
```

Kein lauffähiges vollständiges Programm

OpenGL-Pipeline (Version 4.6)



OpenGL-Pipeline (Vereinfachung)



OpenGL Shading Language (GLSL)

Allgemein

- SIMD-Prinzip (Single Instruction, Multiple Data)
→ mehrere Dateneinheiten (z. B. Vertex-Pos.) werden gleichzeitig auf die gleiche Weise behandelt; Vorstellung: ein Prozessor für jeden Vertex/Primitiv/Fragment vor
- Sprache ähnlich wie C
- Grunddatentypen: `float`, `double`, `int`, `bool`
- Vektoren mit Komponenten: `vec{2.0, 3.0, 4.0, 5.0}`, `bvec{true, false}`
- Matrizen von 2x2 bis 4x4, auch asymmetrisch: `[d]mat2`, `[d]mat3`, `[d]mat2x3`,
- Arrays (nur eindimensional, flexibel deklariert, feste Länge):
`float a[5] = float[](3.4, 4.2, 5.0, 5.2, 1.1);`

OpenGL Shading Language (GLSL)

Modifizierer (Qualifizierer)

- `const`: Konstante, definiert zur Kompilierzeit
- `in`: Eingabeattribut des aktuellen Shaders (optional, default)
- `out`: Ausgabe-Attribut des aktuellen Shaders
- `uniform`: schreibgeschützt
- globale Variablen: Konstante pro Draw Call
 - von Anwendungen gesetzt oder
 - aus Puffern
- `Buffer`: globaler Speicher, lesbar und beschreibbar
- `shared`: nur im Compute Shader, auch beschreibbar

OpenGL Shading Language (GLSL)

Funktionen

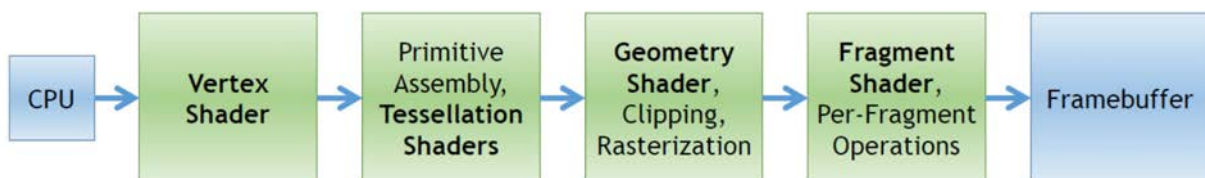
- Shader ist durch Funktionen strukturiert
obligatorisch: `void main()`
- Eigene Funktionen:
 - `in`: Eingabeparameter (optional)
 - `out`: Rückgabewert (oder Rückgabeanweisung)
 - `inout`: sowohl Ein- als auch Ausgabe

Beispiel: `void computeLighting(vec3 lightPos, vec3 normal,
out float diffuse, out float specular) {...}`

- Funktionen können überladen werden
- Rekursion ist (noch) nicht erlaubt

Shader-Konzept

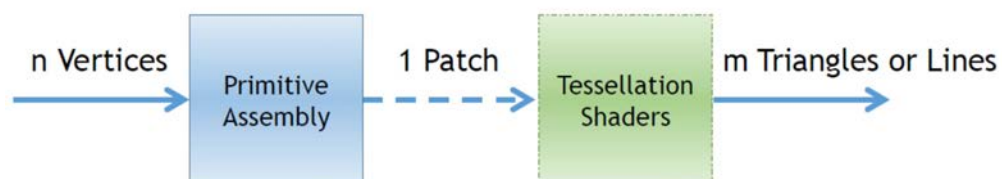
- Shader sind Quellcodes für einen der programmierbaren Schritte der Grafikpipeline
→ Vertex-, Tessellation-Control-, Tessellation-, Evaluation-, Geometrie- und Fragment-Shader (+ Compute-Shader)



- Grundstruktur und Fähigkeiten sind bei allen Typen gleich
 - Bedienung und Funktionen sind identisch
 - Semantik und Layout der Ein- und Ausgabedaten variiert
 - Probleme können in verschiedenen Stufen gelöst werden, aber meist ist nur ein bestimmter Shader sinnvoll
- Umsetzung hauptsächlich über Hochsprachen, z.B. GLSL (OpenGL), HLSL (Direct3D)

Geometrieverarbeitung: Vertex-Shader

- Transformation einzelner Vertices und ihrer Attribute (z.B. Normalen, ...)
 - kein Erzeugen neuer Eckpunkte
 - kein Löschen von Vertices (außer durch Clipping)
- Berechnen aller Attribute, die pro Vertex gültig sind
 - spart Berechnungen im Vergleich zum Fragment-Shader
 - z.B. Beleuchtung pro Vertex (altmodisch)
- Setzen von Attributen, die pro Fragment interpoliert werden
z.B. Normale für Beleuchtung pro Vertex

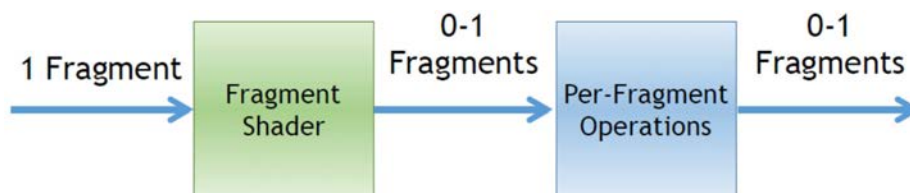
Geometrieverarbeitung (1): Assembly + Tessellierung

- Primitive Assembly setzt Primitive aus Vertices zusammen (nicht programmierbar):
 - GL_POINT, GL_LINE, GL_TRIANGLE, GL_LINE_STRIP usw.
 - GL_PATCH als Eingabeprimativ für Tessellationssshader
- Tessellierung (parallel pro Primitiv):
 - Tessellation Control Shader bestimmt die Unterteilung
 - Tessellator führt die Unterteilung aus (nicht programmierbar)
 - Tessellation Evaluation Shader bearbeitet die resultierende Geometrie



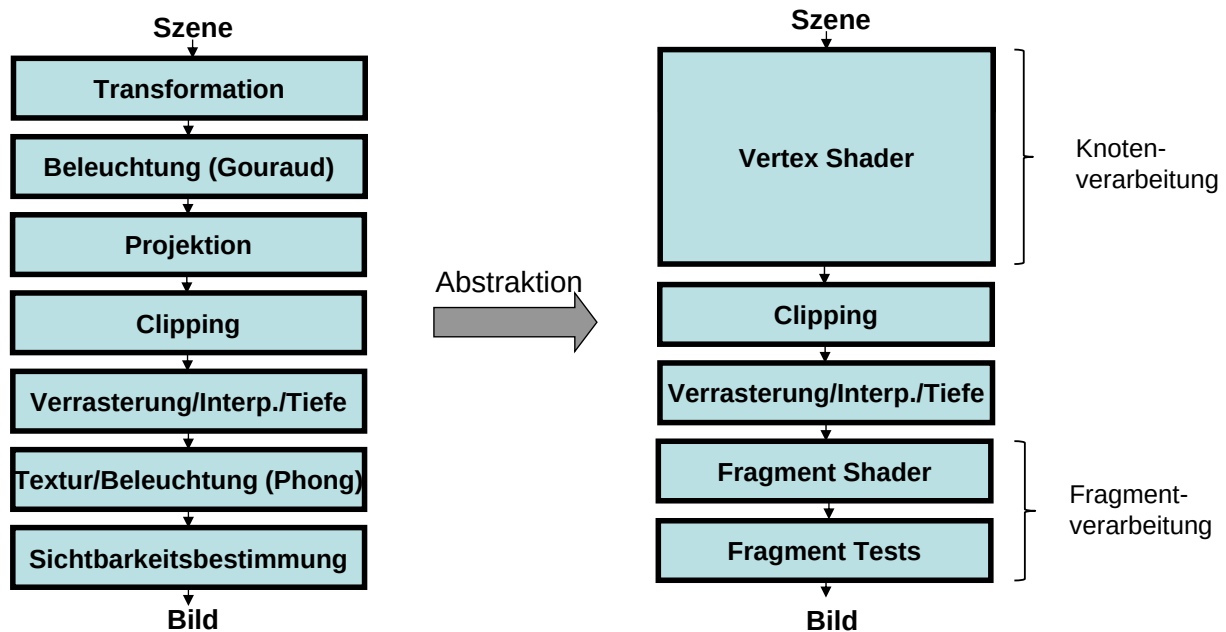
Geometrieverarbeitung (2)

- **Geometrie-Shader**
 - arbeitet mit beliebigen Primitiven, sequentiell pro Primitiv
 - Kann Primitive erweitern, löschen oder ändern (Punkte zu Dreiecken usw.)
 - Pro Primitiv können nur begrenzte Attribute ausgegeben werden
- **Clipping (auf Sichtvolumen)**
 - verwirft nicht sichtbare Punktprimitive
 - modifiziert Linien und Dreiecke durch Schnitte mit dem Betrachtungsvolumen
- Perspektivische Teilung
- Rasterisierung erzeugt Fragmente für den Ausgabepuffer

Verarbeitung von Fragmenten: Fragment Shader

- ermöglicht die Berechnung pro Pixel für den Ausgabepuffer
 - Ausgabefarbe
 - Beleuchtung/Schattierung pro Pixel
 - Berechnung von Daten innerhalb des Primitive, z. B. Volumenrendering, implizite Oberflächen, Glyphen
- Eingabeattribute werden innerhalb des Primitive interpoliert (optional)
- Fragmente können verworfen werden: discard
- Fragmentoperationen: Tests, Überblendung usw.

Programmierbare OpenGL-Pipeline (vereinfacht)



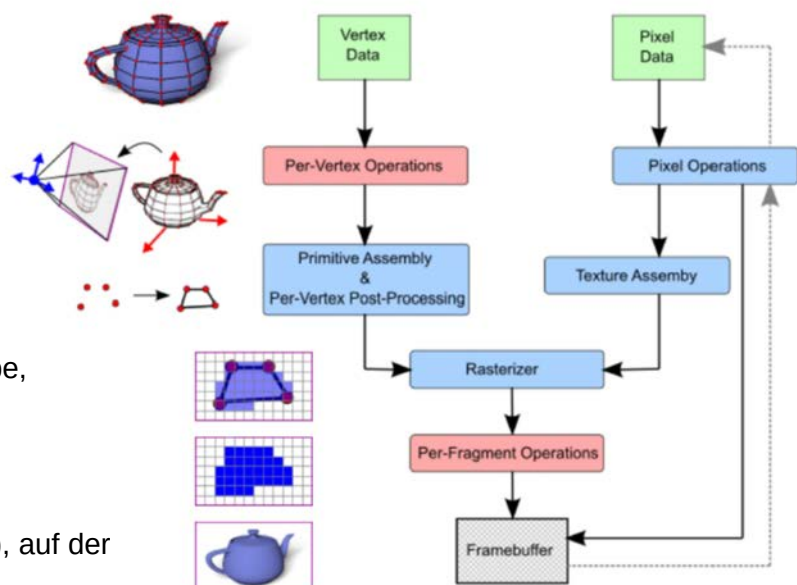
Programmierbare OpenGL-Pipeline (vereinfacht)

Vertex: alle Attribute zur vollständigen Spezifikation eines Eckpunktes eines Primitives 3D Koordinaten, Farbe, Normale, Textur-Koordinaten, usw.

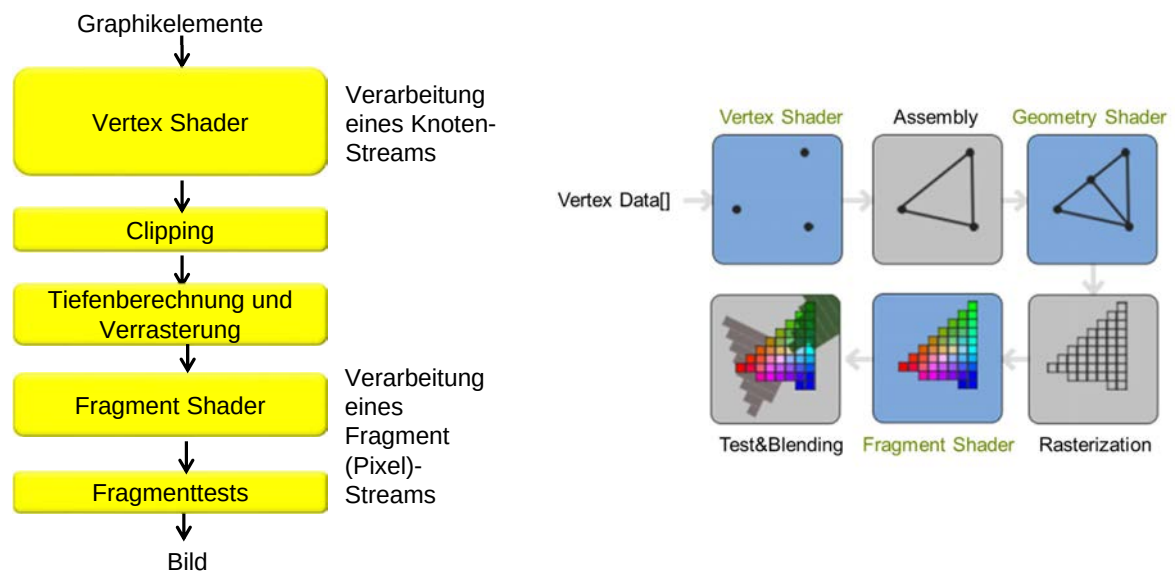
Pixel: Anzahl Bytes im Framebuffer bzw. Bildschirmpunkt

Fragment: Menge von Daten (Farbe, Koordinaten, Alpha, ...), die zum Einfärben eines Pixels benötigt werden

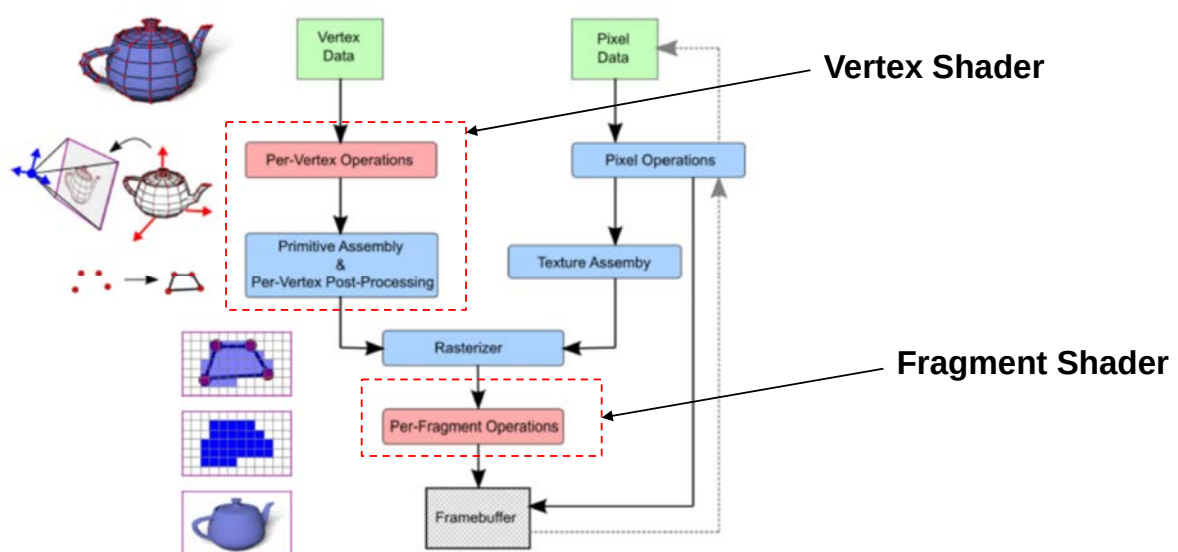
Framebuffer: Canvas („Leinwand“), auf der OpenGL zeichnet, Rastergrafik

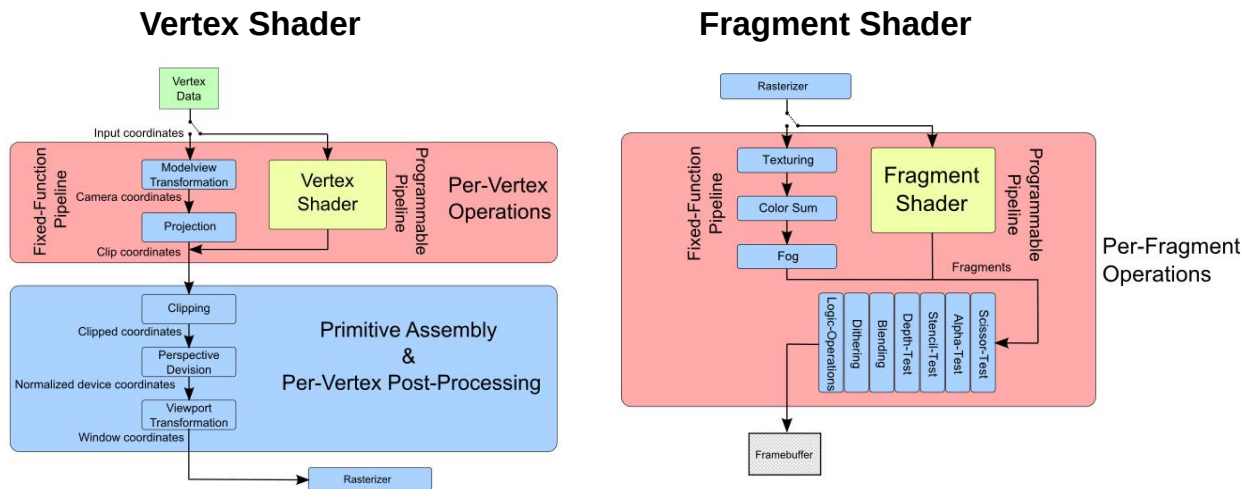


Verarbeitung durch Vertex- und Fragment-Shader

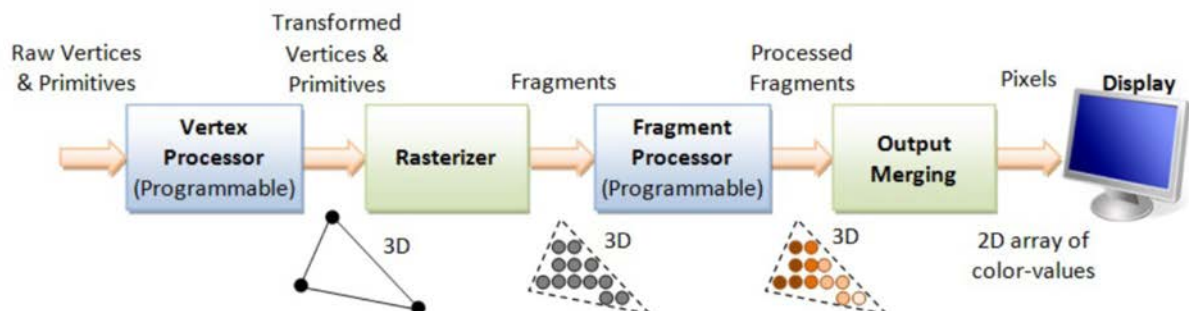


Verarbeitung durch Vertex- und Fragment-Shader



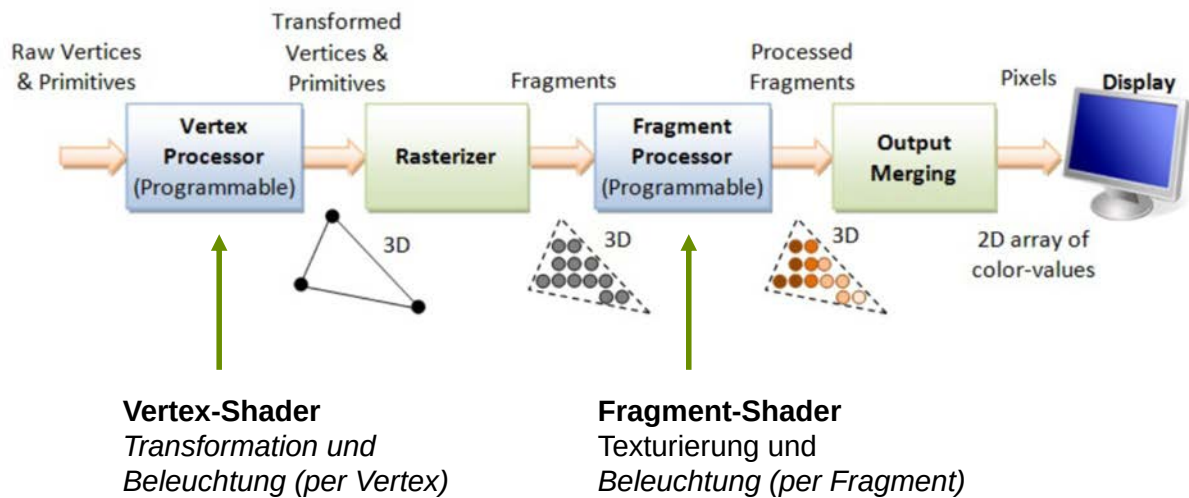


Grobstruktur aus Shader-Sicht

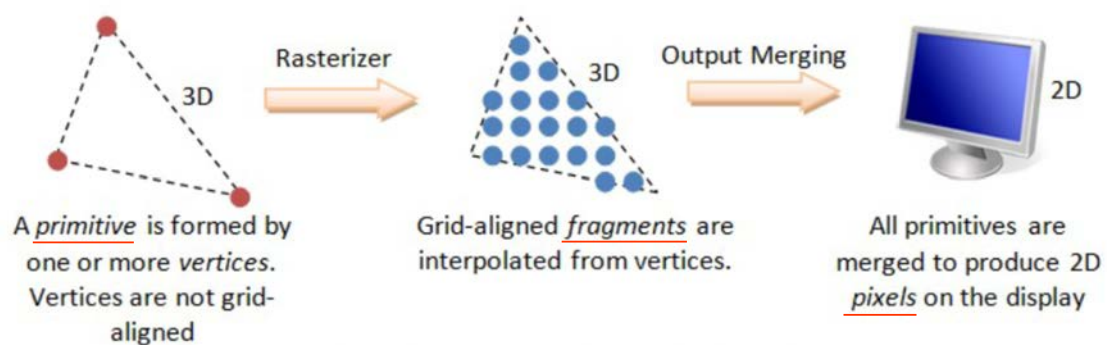


1. Vertex-Verarbeitung: Verarbeitung und Transformation der einzelnen Vertices und Normalen
2. Rasterisierung: Konvertierung jedes Primitives (verbundene Vertices) in eine Reihe von Fragmenten. Ein Fragment kann als ein Pixel mit Attributen wie Position, Farbe, Normale und Textur interpretiert werden.
3. Fragment-Verarbeitung: Individuelle Fragmente verarbeiten.
4. Zusammenführen der Ausgabe: Kombinieren der Fragmente aller Primitive (im 3D-Raum) zu 2D-Farbpixeln für die Anzeige.

Grobstruktur aus Shader-Sicht

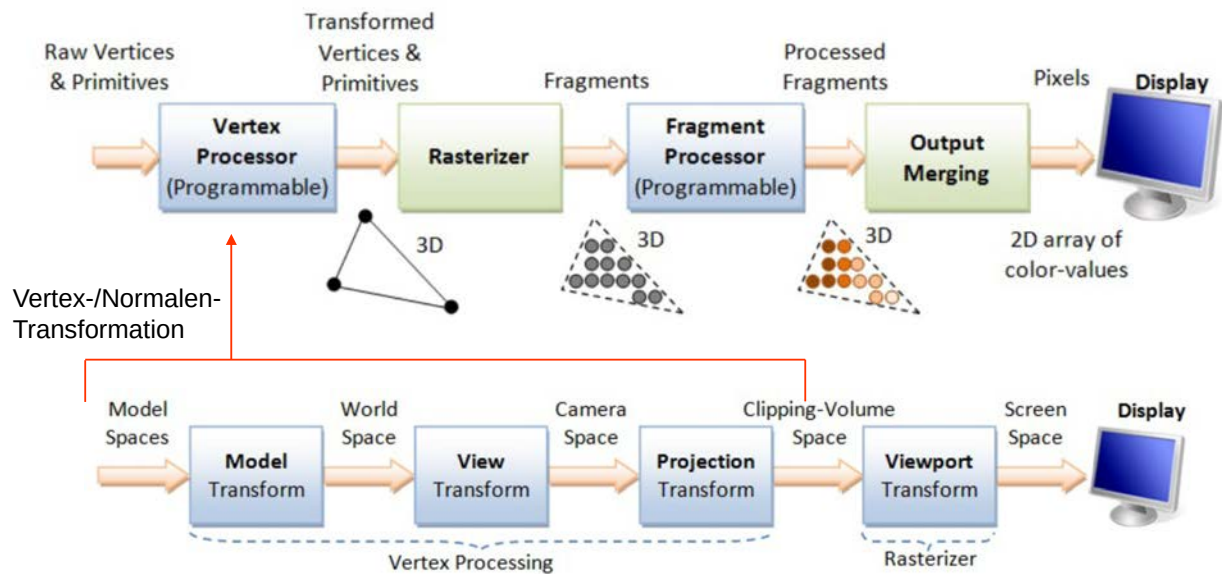


Primitive, Pixels und Fragments



- Fragmente werden durch gerasterte 2D-Koordinaten auf dem Bildschirm repräsentiert, inkl. zugehöriger Attribute (z.B. Texturkoordinaten, Tiefenwert, Alphawert)
- Pixel entsprechen der sichtbaren Information auf dem Bildschirm

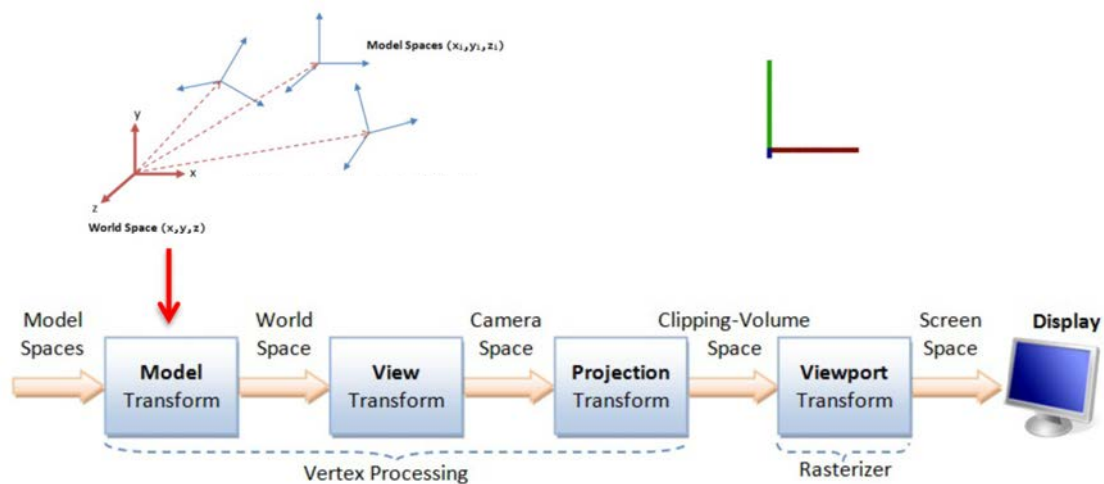
Vertex-Transformation



Koordinaten-Transformationspipeline

Vertex-Transformation

1. Anordnung der Objekte (Modelle) in der Welt (Model Transform)

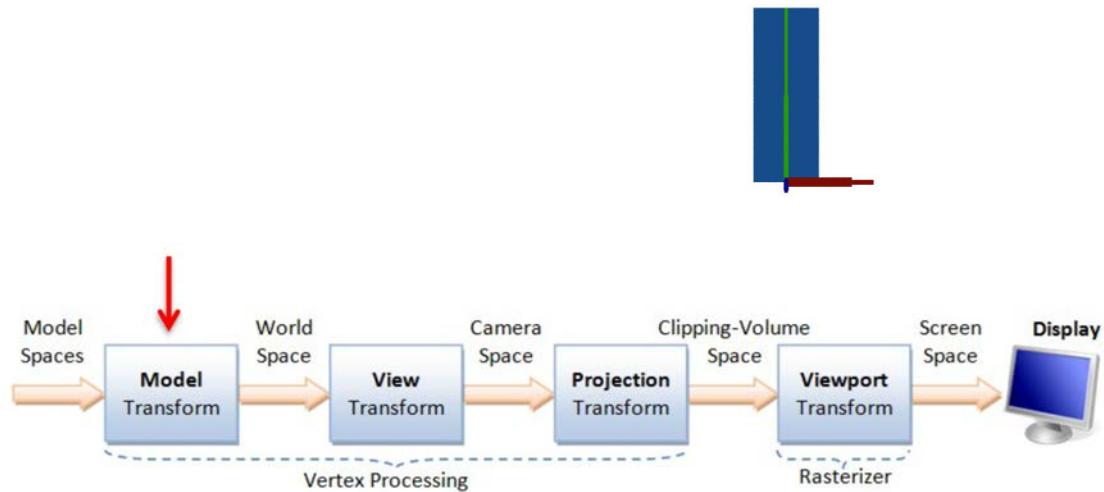


Koordinaten-Transformationspipeline



Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)

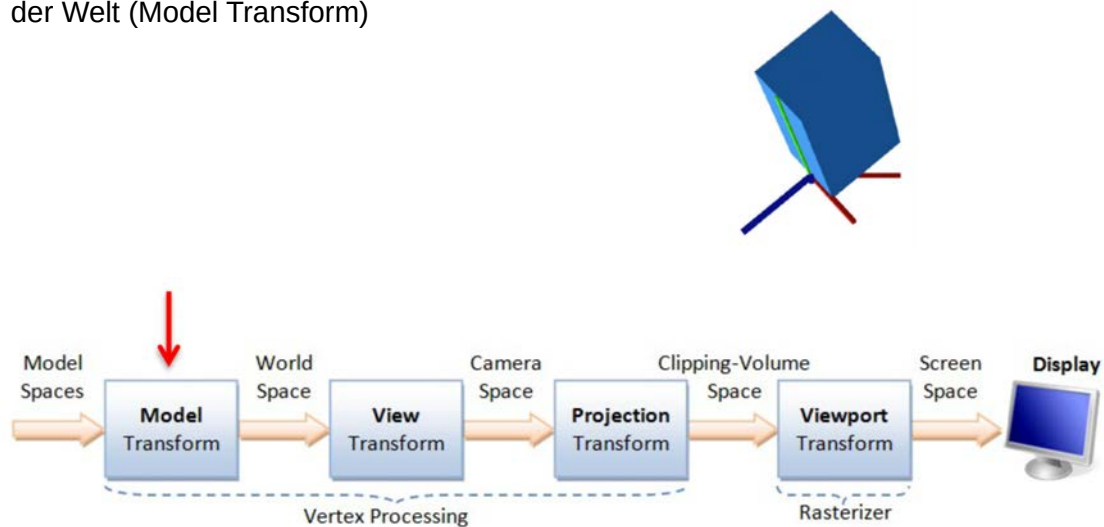


Koordinaten-Transformationspipeline



Vertex-Transformation

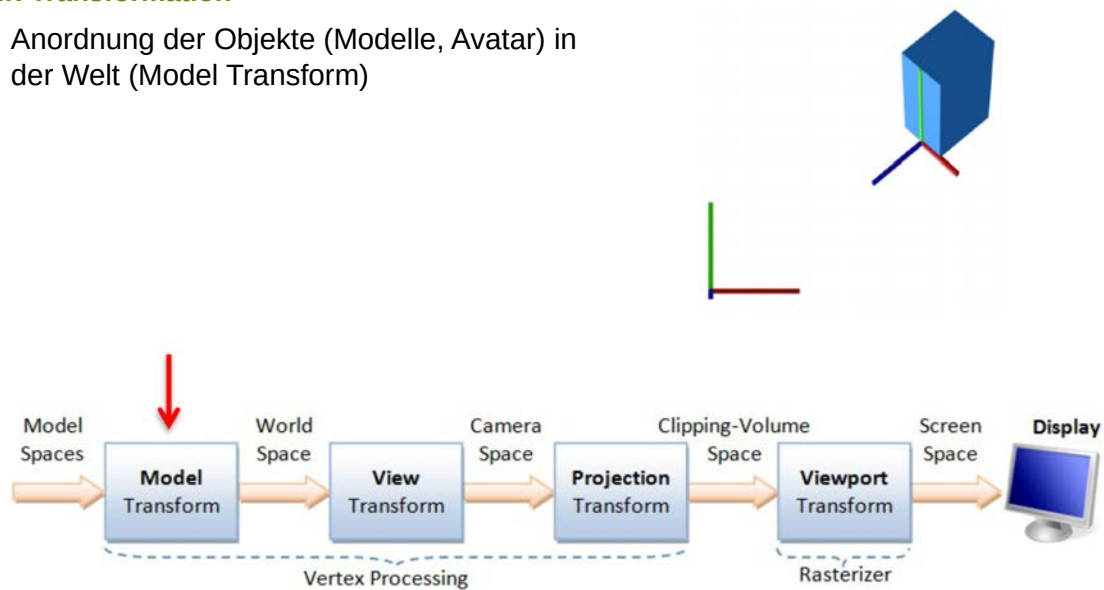
1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)



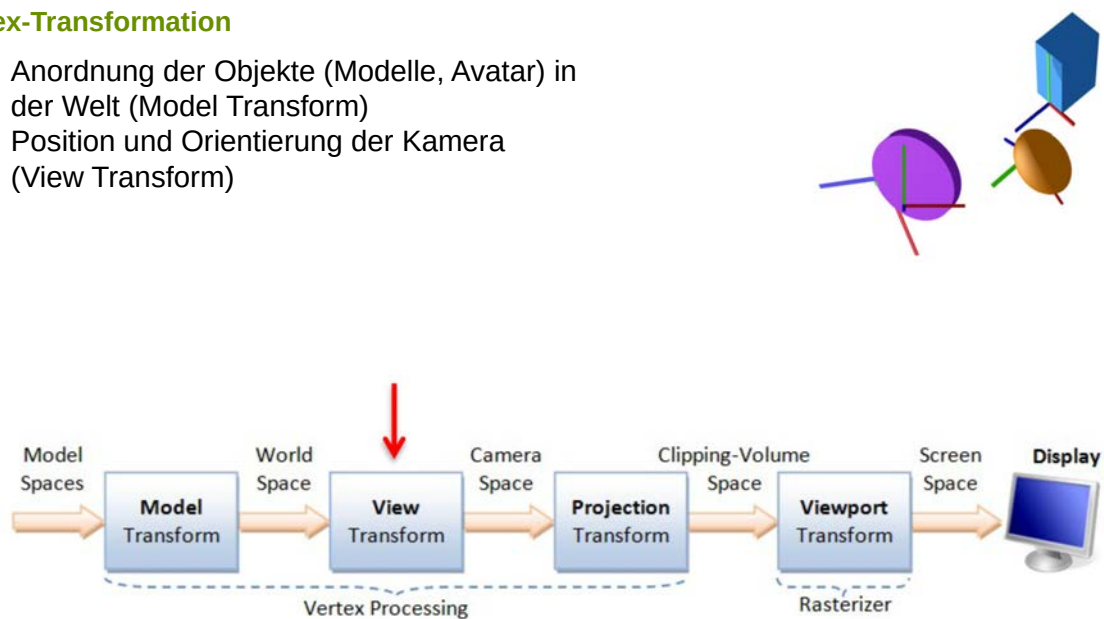
Koordinaten-Transformationspipeline

Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)

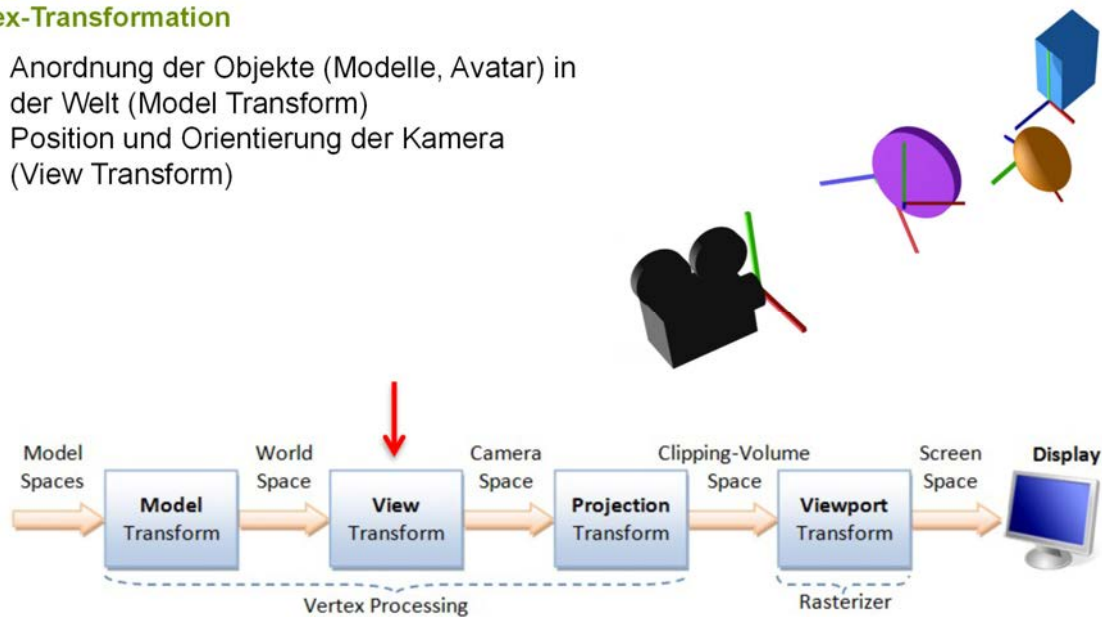
**Koordinaten-Transformationspipeline****Vertex-Transformation**

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)

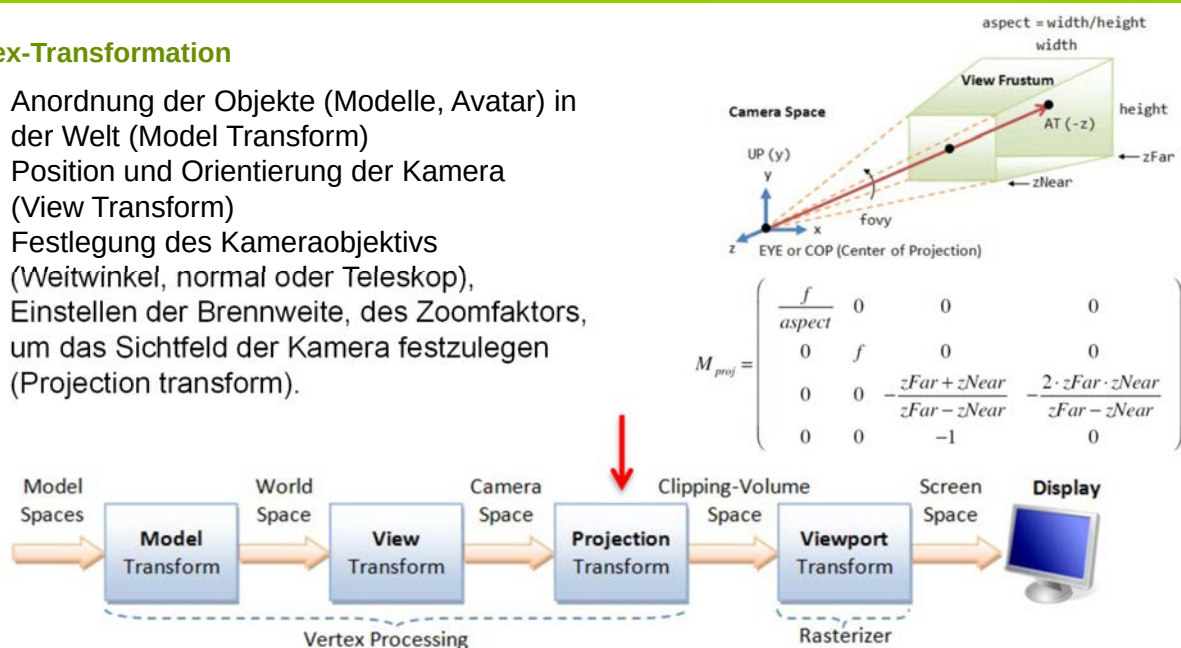
**Koordinaten-Transformationspipeline**

Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)

**Koordinaten-Transformationspipeline****Vertex-Transformation**

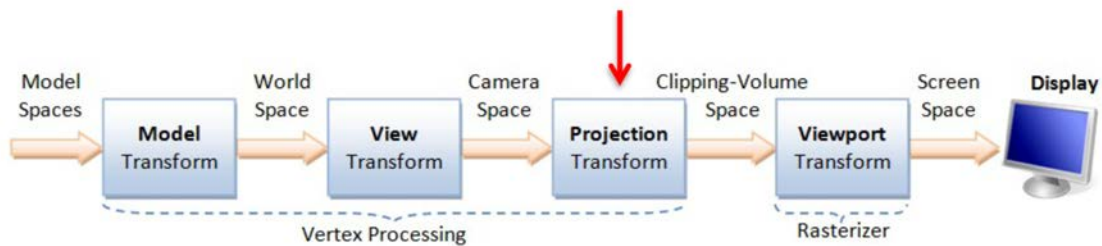
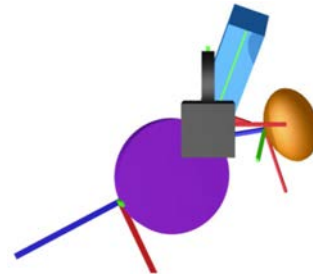
1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).

**Koordinaten-Transformationspipeline**



Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).

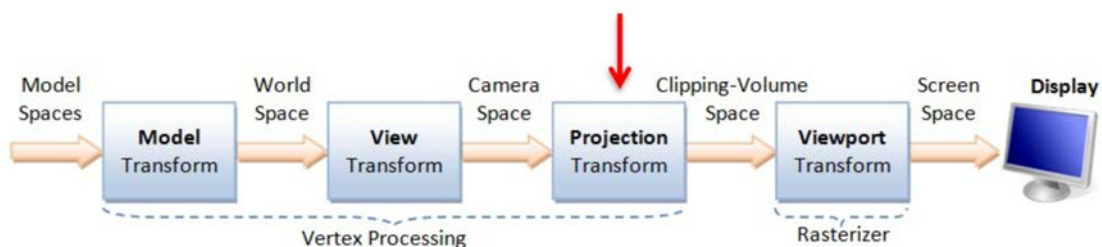
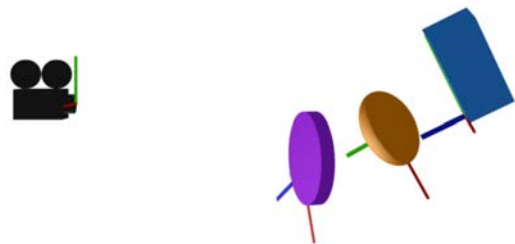


Koordinaten-Transformationspipeline



Vertex-Transformation

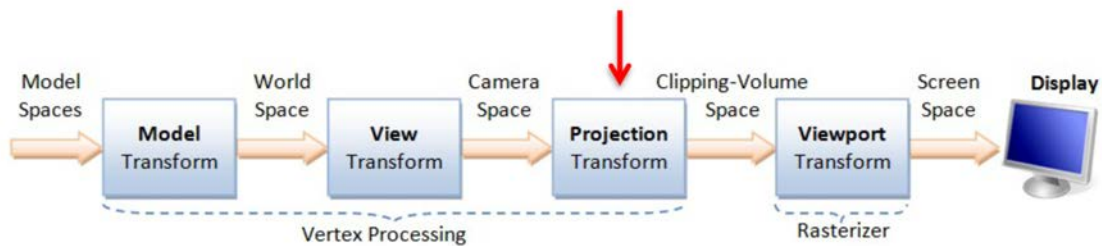
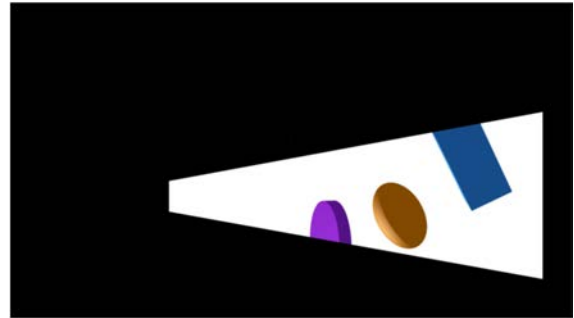
1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).



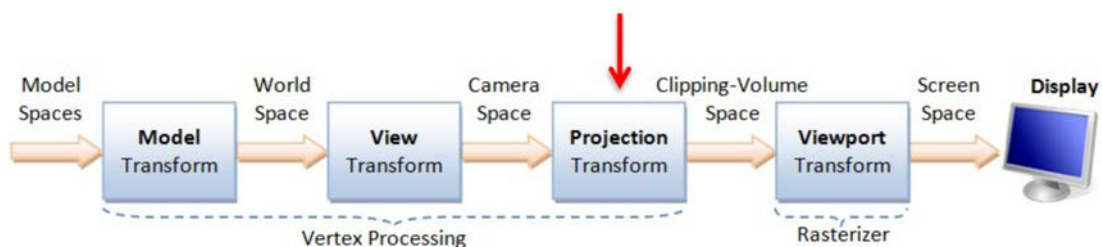
Koordinaten-Transformationspipeline

Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).

**Koordinaten-Transformationspipeline****Vertex-Transformation**

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).

**Koordinaten-Transformationspipeline**

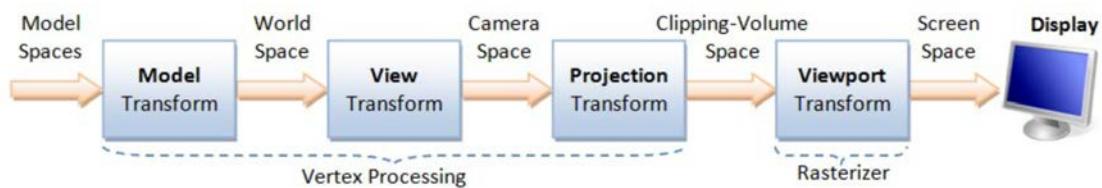
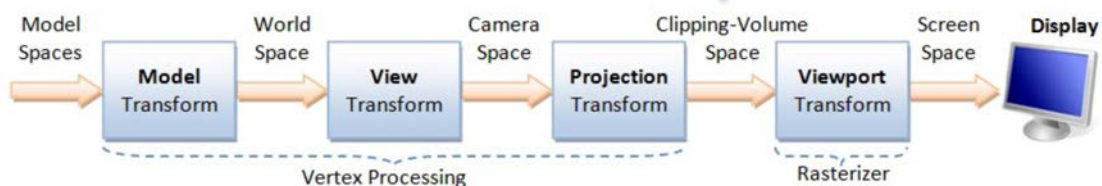
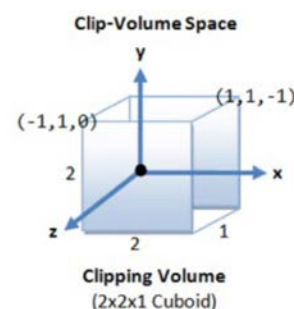
Vertex-Transformation

1. Anordnung der Objekte (Modelle, Avatar) in der Welt (Model Transform)
2. Position und Orientierung der Kamera (View Transform)
3. Festlegung des Kameraobjektivs (Weitwinkel, normal oder Teleskop), Einstellen der Brennweite, des Zoomfaktors, um das Sichtfeld der Kamera festzulegen (Projection transform).

Resultierende Modelview Projection Matrix

$$v_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot v$$

$$= M_{proj} \cdot M_{mv} \cdot v$$

**Koordinaten-Transformationspipeline****Clip Space****Koordinaten-Transformationspipeline**



Erstellung von Shader-Programmen in OpenGL

- Shader-Programme werden zur Laufzeit kompiliert und an die Grafikkarte übergeben
- Um ein Shader-Programm zu erstellen, das Vertex- und Fragment-Shader enthalten kann, wird der folgende Befehl verwendet

```
progID = glCreateProgram();
```

- Erstellen, Zuweisen und Kompilieren von Shader-Code:

```
vertID = glCreateShader(GL_VERTEX_SHADER);  
fragID = glCreateShader(GL_FRAGMENT_SHADER);  
glShaderSource(vertID, 1, &vertShaderSrcCodeString, &vStrLength);  
glShaderSource(fragID, 1, &fragShaderSrcCodeString, &fStrLength);  
glCompileShader(vertID);  
glCompileShader(fragID);
```

- Zuweisung der Shader dem Programm und Erzeugung einer ausführbaren Datei:

```
glAttachShader(progID, vertID);  
glAttachShader(progID, fragID);  
glLinkProgram(progID);
```



Verwendung von Shader-Programmen in OpenGL

- Um ein Shader-Programm zu aktivieren, wird der folgende Befehl verwendet

```
glUseProgram(progID);
```

- In diesem Fall verhält sich OpenGL wieder wie eine State-Machine, d.h. alle nachfolgenden Rendering-Funktionen verwenden das gerade aktive Shader-Programm
- Das bedeutet auch, dass ein Wechsel zwischen verschiedenen Shader-Programmen zur Laufzeit möglich ist (und in der Praxis oft vorkommt). Zum Beispiel werden verschiedene Shader verwendet, um verschiedene Materialien zu simulieren.

Löschen von Shader-Programmen in OpenGL

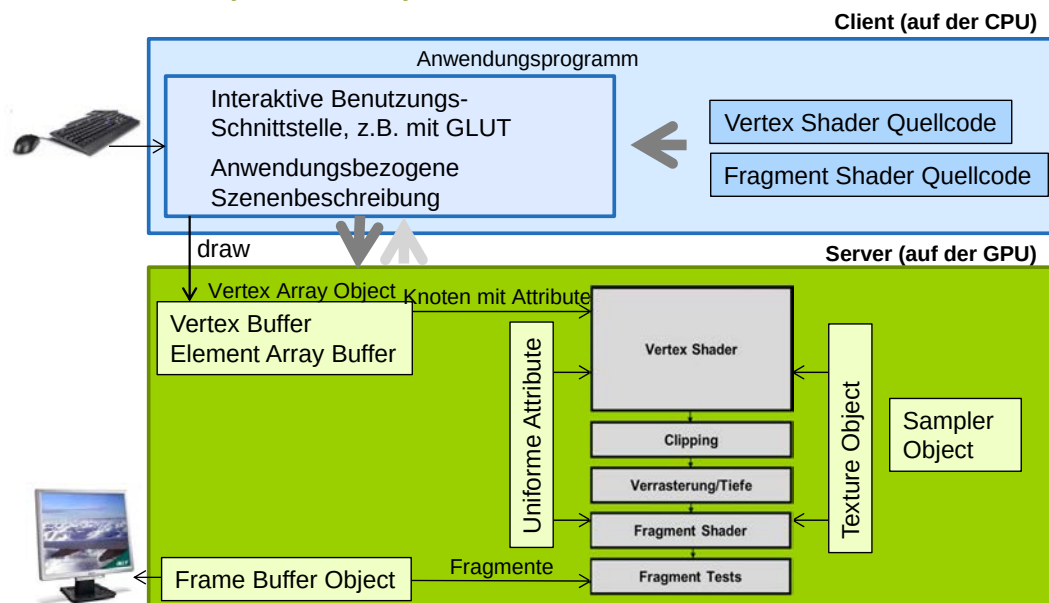
- Zum Löschen werden die folgenden Funktionen verwendet:

```
glDeleteProgram(progID);  
glDeleteShader(vertID);  
glDeleteShader(fragID);
```



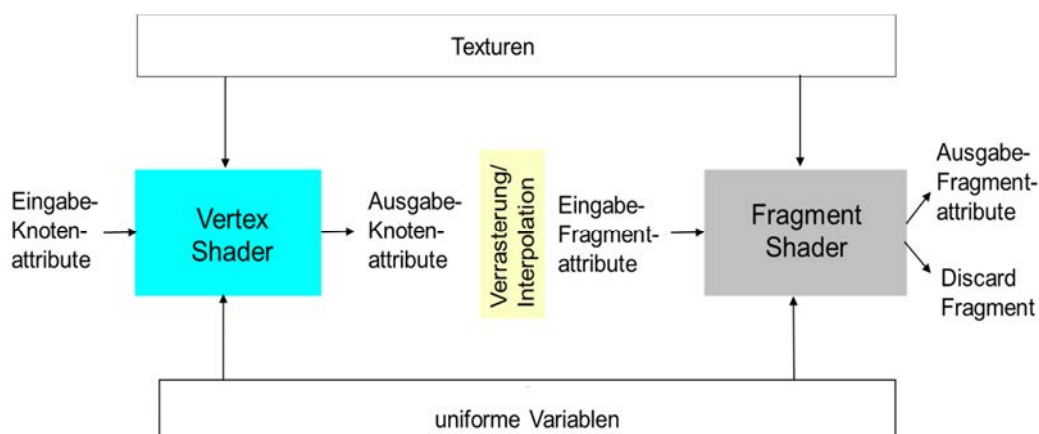
Vereinfachtes OpenGL-Prinzipbild

GPU = Graphics Processing Unit (Graphikkarte)



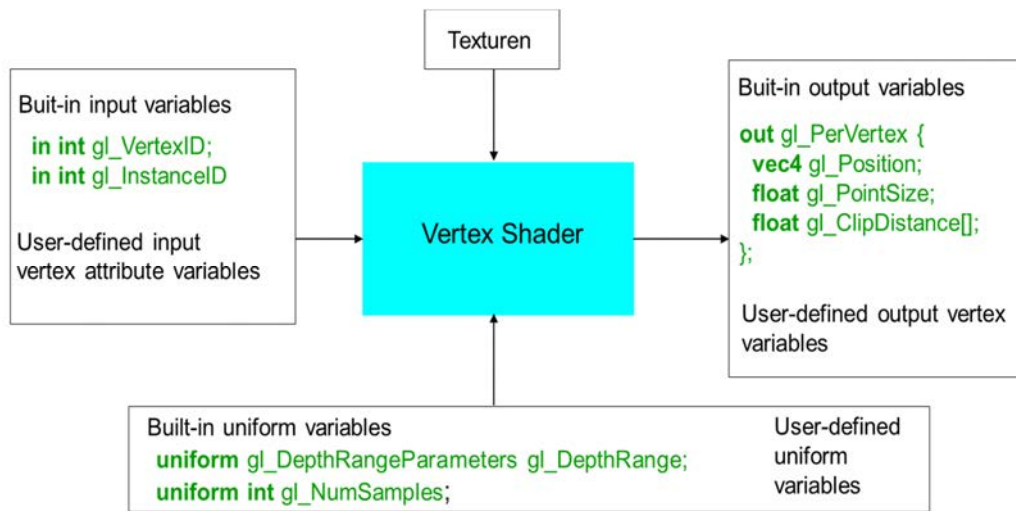
Kommunikation zwischen OpenGL-Anwendungsprogramm und Shadern

Die Kommunikation zwischen OpenGL und den Shadern sowie zwischen den Shadern entlang der OpenGL-Pipeline geschieht über verschiedene Typen von Variablen:

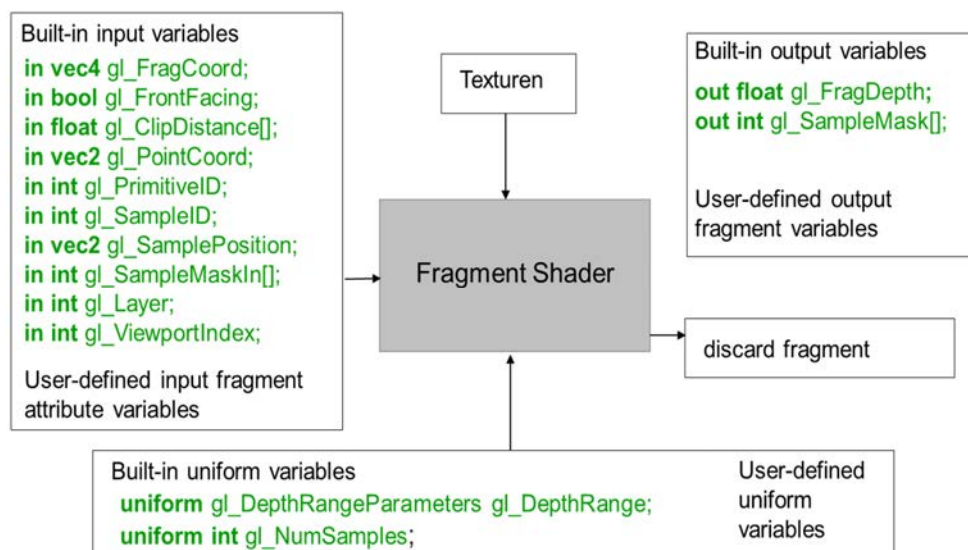




GLSL Vertex Shader Pipeline-Eingabe- und Ausgabevariablen



GLSL Fragment Shader-Pipeline-Variablen



**Beispiel:****Vertex Shader**

```
#version 330
uniform mat4 uMVPMatrix;

in vec3 vPosition;
in vec3 vColor;

out vec3 fColor;

void main() {
    gl_Position = uMVPMatrix * vec4(vPosition, 1.0);
    fColor = vColor;
}
```

eingebautes out-Attribut

Fragment Shader

```
#version 330
precision mediump float;

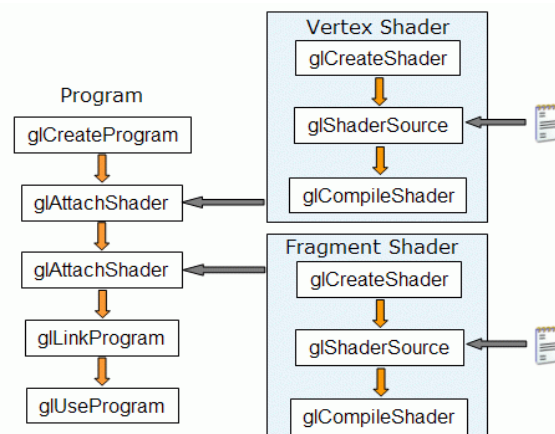
in vec3 fColor;

out vec4 outputColor;

void main() {
    outputColor = vec4(fColor, 1.0);
}
```

**OpenGL-Einbindung von Shader Programmen**

Vorgehensweise bei der Einbindung: Jeder Shader ähnelt einem C-Modul und wird analog getrennt **compiliert**. Die Menge der compilierten Shader wird dann in ein Programm zusammengebunden („**gelinkt**“).





OpenGL-Einbindung von Shader Programmen

```
...
glCompileShader(my_vertex_shader);
glCompileShader(my_fragment_shader);
glAttachShader(shader_programme, my_fragment_shader);
glAttachShader(shader_programme, my_vertex_shader);
```

```
glBindAttribLocation(shader_programme, 0, "vertex_position");
glBindAttribLocation(shader_programme, 1, "vertex_colour");
```

```
glLinkProgram(shader_programme);
```

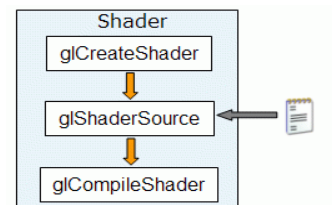
...

Beispiele: siehe auch Übungen



Beispiel:

```
private void makeShaders() {
    // Erstelle shader-code
    String vertexShaderCode =
        "#version 330" +
        "uniform mat4 uMVPMatrix;" +
        "in vec3 vPosition;" +
        "in vec3 vColor;" +
        "out vec3 fColor;" +
        "void main() {" +
        "    gl_Position = uMVPMatrix * vec4(vPosition, 1.0);" +
        "    fColor = vColor;" +
        "}";
```



```
// Erstelle shader-code
String fragmentShaderCode =
    "#version 330" +
    "precision mediump float;" +

    "in vec3 fColor;" +

    "out vec4 outputColor;" +

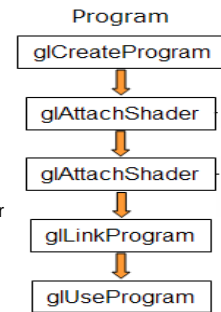
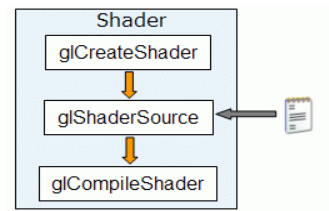
    "void main() {" +
    "    outputColor = vec4(fColor, 1.0);" +
    "}";
```

// Lade Shader

```
int vertexShader = createShader(GL_ES20.GL_VERTEX_SHADER,
    vertexShaderCode);
int fragmentShader = createShader(GL_ES20.GL_FRAGMENT_SHADER,
    fragmentShaderCode);
```

// Erstelle Programm

```
mProgram = GLES20.glCreateProgram(); // Erstelle Shader Programm
GLES20.glAttachShader(mProgram, vertexShader); // Setze Vertex Shader
GLES20.glAttachShader(mProgram, fragmentShader); // Setze Fragment Shader
GLES20.glLinkProgram(mProgram); // Linke Programm
}
```



```
private int createShader(int type, String shaderCode){
```

```
// Erzeuge einen Shader. Typ = Vertex oder Fragment
int shader = GLES20.glCreateShader(type);
```

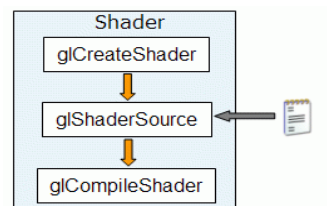
```
// Shader aus Code laden
GLES20.glShaderSource(shader, shaderCode);
checkGLError("glShaderSource");
GLES20.glCompileShader(shader);
checkGLError("glCompileShader");
```

```
// Logging abfragen
String log = GLES20.glGetShaderInfoLog(shader);
Log.e("Debug", log);
```

```
return shader;
}
```

// Fehlerbehandlung

```
private void checkGLError(String glOperation) {
    int error;
    while ((error = GLES20.glGetError()) != GLES20.GL_NO_ERROR) {
        Log.e("Error", glOperation + ": glError " + error);
        throw new RuntimeException(glOperation + ": glError " + error);
    }
}
```



E.1. Graphik-Pipeline-basierte Bilderzeugung

- E.1.1. Einleitung
- E.1.2. Graphik-Pipeline
- E.1.3. Grundkonzepte von OpenGL
- E.1.4. Pufferspeicherobjekte
- E.1.5. Shader
- E.1.6. Vereinfachte OpenGL-Shader Pipeline
- E.1.7. Einbindung von Shader-Programmen

E.2. Szenengraphen

- E.2.1. Einleitung
- E.2.2. Aufbau von Szenengraphen
- E.2.3. Java 3D Szenengraphen
- E.2.4. Beispiel: HelloJava3Da
- E.2.5. Virtuelle und reale Szene

E.3. Strahlverfolgungsbasierte Bilderzeugung

- E.3.1. Einführung
- E.3.2. Strahlverfolgungsverfahren
- E.3.3. Anti-Alias-Behandlung
- E.3.4. Strahl-Dreieck-Schnitt
- E.3.5. Spezifikation von Eingabeszenen
- E.3.6. Effiziente Strahlverfolgung
- E.3.7. Fotorealistisches Rendering

Szenengraphenkonzept

- geht auf Inventor von Silicon Graphics zurück
- wird in verschiedenen anderen Graphikbibliotheken und Szenenbeschreibungssprachen verwendet: Performer, OpenInventor, VRML
- Gegenstand von Standardisierungsaktivitäten: OpenSceneGraph Initiative

Allgemeine Eigenschaften

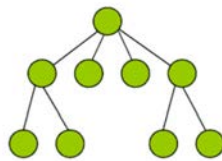
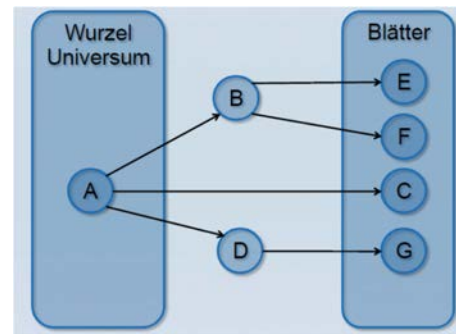
- Verwalten von 3D-Szenen
- Elemente einer Szene werden in einer Graphstruktur gespeichert
- Jede reale Implementierung ist optimiert für einen besonderen Einsatzzweck
- Können verschiedene Operation mit Szenen beschleunigen
- „Modell-orientiert“, reine 3D-APIs sind Punkt-orientiert

Szenengraphen sind Objekt-orientierte Datenstrukturen

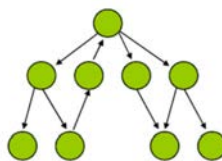
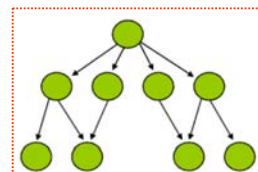
- Directed Acyclic Graph (DAG)
- Baum (Wurzel enthält gesamte Szene)
- Besteht aus Objekten und Transformationen
- Jedes Objekt kann selbst ein Baum sein

Terminologie für Szenengraphen

- Allgemein ist die Anzahl der Kinder nicht festgelegt.
- A ist der Vater von B, E das Kind von B
- Unterschied Baum und Graph



Baum

gerichteter
nicht-zyklenfreier Graphgerichteter
zyklenfreier Graph

→ Szenengraph

Szenengraphen bestehen aus mindestens drei Knotentypen

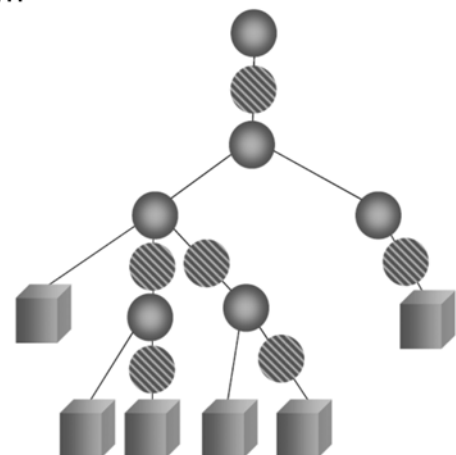
- Gruppen (Knoten mit einem grauen Kreis)
- Graphikelement (inkl. Materialeigenschaften) (Knoten mit einem grauen Quadrat)
- Transformationen (Knoten mit einem gestrichelten Kreis)

Szenengraph dient zur Verwaltung einer komplexen Szene

- Gruppierung von Geometrien zu Gruppen
- Gruppierung von Gruppen zu Gruppen
- Gruppierung von Gruppen zu einer Szene

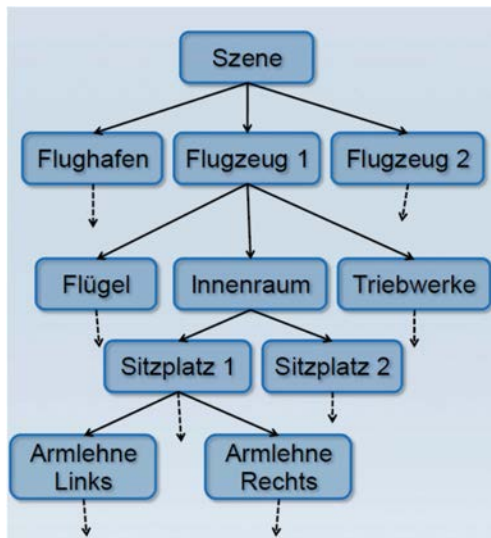
Struktur

- gerichtet
- azyklisch
- heterogen
- Geometrie / darstellbare Primitive in den Blättern



Quelle: Hergenröther, h_da

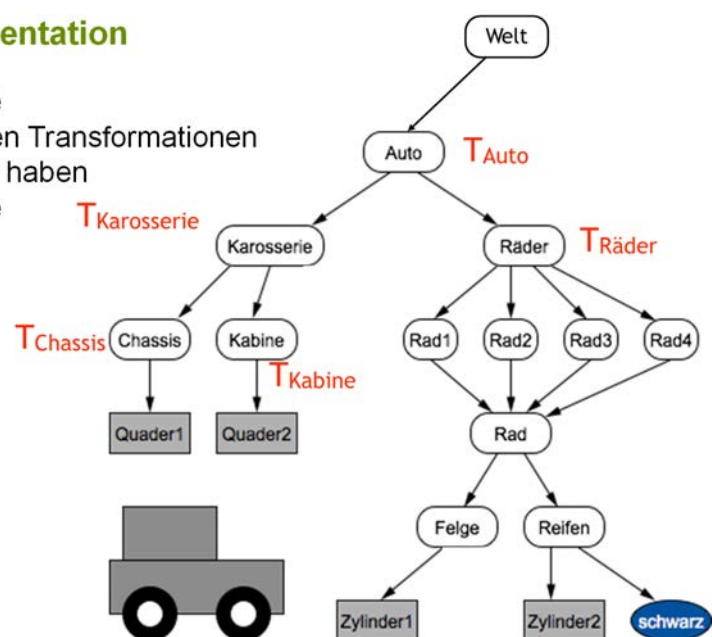
Beispiele für Szenengraphen



Quelle: Sebastian Schäfer, SR Heidelberg

Szenengraphen: Geometrirepräsentation

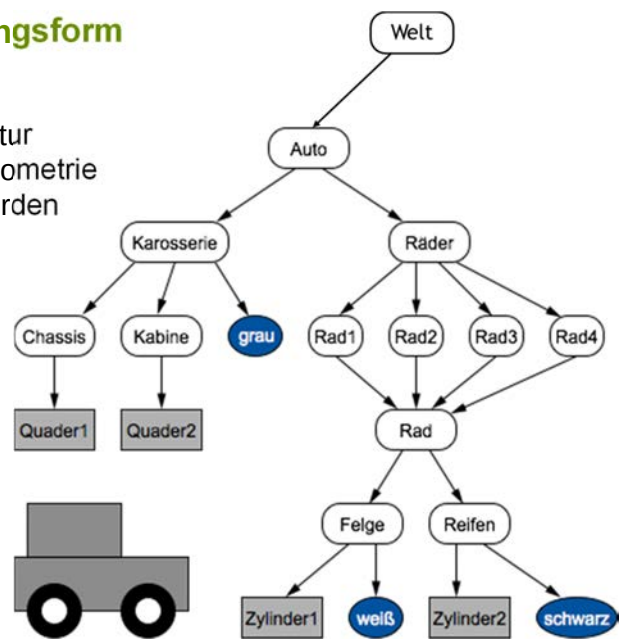
- Blätter sind grundlegende 3D-Objekte
- Nicht-Blatt-Knoten (Gruppen) enthalten Transformationen
 - können ein oder mehrere Kinder haben
 - Transformation durch homogene Matrix gegeben
- Wurzel ist die gesamte Welt



Quelle: Andreas Butz, LMU München

Szenengraphen: Darstellung/Erscheinungsform

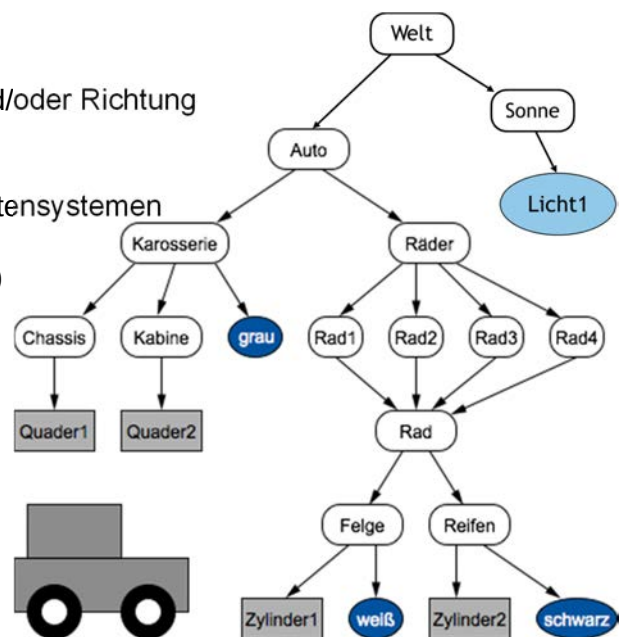
- Szenegraph enthält zudem die Darstellung
 - z.B. Farbe, Reflexion, Transparenz, Textur
 - Wiederverwendung vergleichbar zur Geometrie
- Darstellung kann nur teilweise spezifiziert werden
→ nicht spezifizierte Werte werden vererbt



Quelle: Andreas Butz, LMU München

Szenengraphen: Beleuchtung

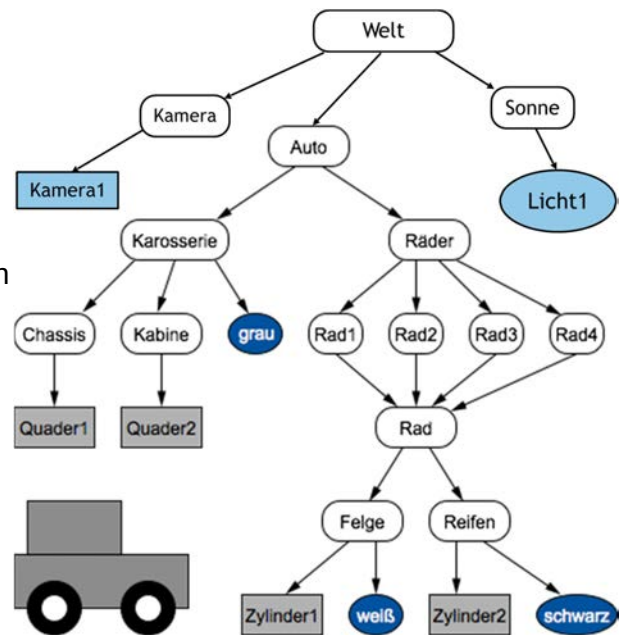
- Lichtquellen benötigen ebenfalls Position und/oder Richtung
 - direkt in den Szenengraphen einfügen
 - Transformation vglb. zu Geometrie
- Lichtquellen können sich in lokalen Koordinatensystemen von Geometriegruppen befinden
→ sich mit ihnen bewegen (z.B. Autolampen)



Quelle: Andreas Butz, LMU München

Szenengraphen: Kamera

- Kameras benötigen ebenfalls Position und/oder Richtung
 - direkt in den Szenengraphen einfügen
 - Transformation vglb. zu Geometrie
- Kameras können sich in lokalen Koordinatensystemen von Geometriegruppen befinden
 - sich mit ihnen bewegen (z.B. Autofahrer)



Quelle: Andreas Butz, LMU München

Java 3D:

- Hierarchie von Java-Klassen, die als Schnittstelle zur dreidimensionalen Bilderzeugung und Klangerzeugung (zumindest als Interface) dienen
- basiert auf dem Konzept des Szenengraphen
- wird erweitert durch Utility Classes
- wird in Kombination mit anderen Klassenbibliotheken verwendet, insbesondere Java AWT beziehungsweise Java Swing zur Einbettung in ein Fenstersystem
- aktuelle Information:
<https://www.oracle.com/java/technologies/javase/java3d.html>

QUELLE:

Sun Microsystems, Getting Started with the Java 3D API: A Tutorial for Beginners



Zugrundeliegende Philosophie

Szenendarstellung als hierarchische Gruppierung:

Szene := Transformation Gruppe

Gruppe := {Bestandteil, . . . , Bestandteil}

Bestandteil := Transformation Gruppe|Graphikelement

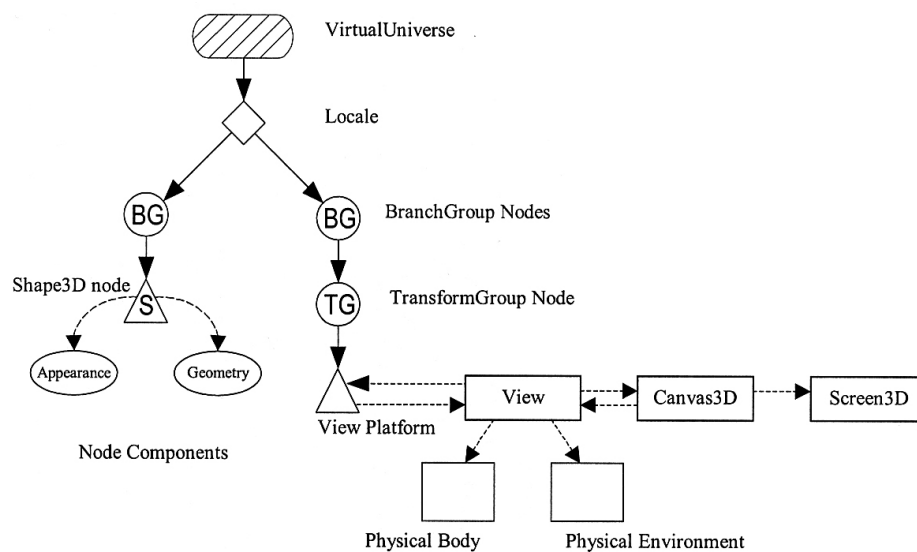
Transformation := eine affine Transformation

Graphikelement := Strecke|Polygonzug|Markierung| . . .

Bemerkung: Durch mehrfache Verwendung derselben Gruppe können visuell komplexe Szenen kurz beschrieben werden.



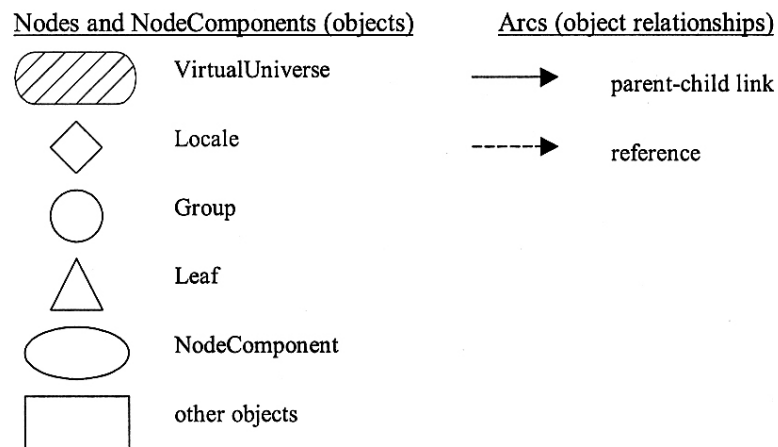
Beispiel eines Java 3D-Szenengraphen





Aufbau eines Java 3D-Szenengraphen

besteht aus Knoten und gerichteten Kanten



Knotentypen:

Die Knoten sind Objekte und Instanziierungen von Java 3D-Klassen:

VirtualUniverse:

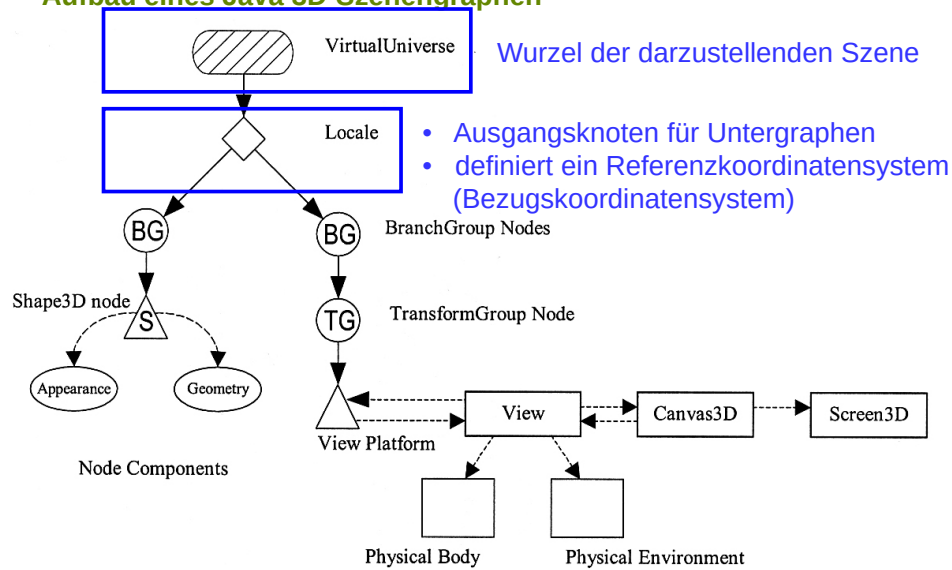
- definiert die Wurzel der darzustellenden Szene.
- In einem Szenengraphen gibt es im Allgemeinen nur einen VirtualUniverse-Knoten.
- Ein VirtualUniverse-Knoten verweist auf eine Liste von Locale-Objekten.

Locale:

- definiert ein Referenzkoordinatensystem im virtuellen Universum.
- Ein Locale-Objekt kann auf mehrere Untergraphen des Szenengraphen verweisen.



Aufbau eines Java 3D-Szenengraphen



Kanten:

Die Kanten repräsentieren zwei Typen von Beziehungen zwischen Java 3D-Instanzen:

Eltern-Kind-Beziehung:

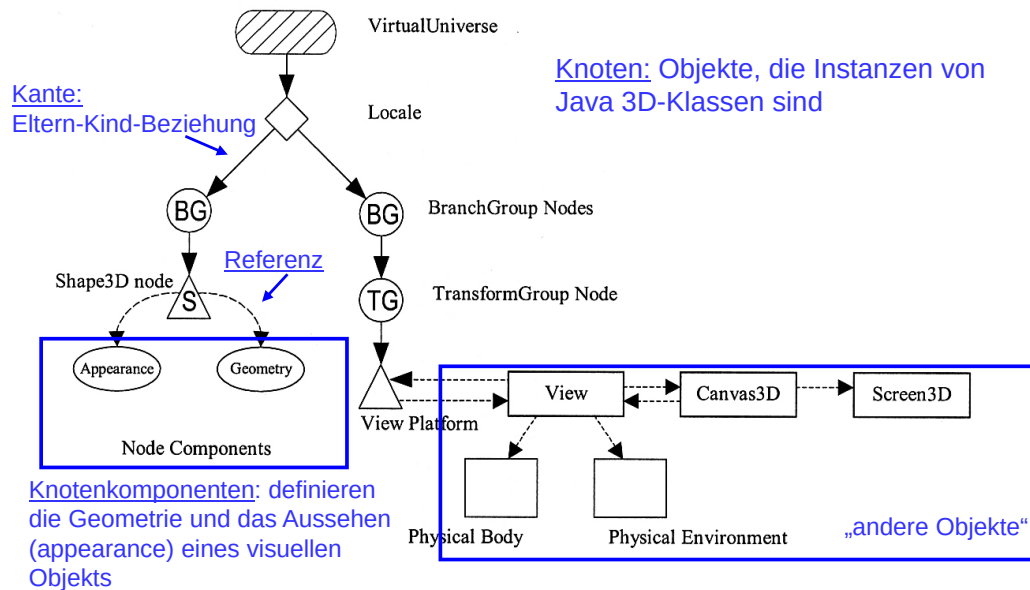
- ein Gruppenknoten kann eine beliebige Anzahl von Kindern haben, aber nur einen Elternknoten
- ein Blattknoten kann einen Elternknoten haben, aber keine Kindknoten

Referenz-Beziehung:

- eine Referenz assoziiert eine Knotenkomponente mit einem Szenengraphenknoten.



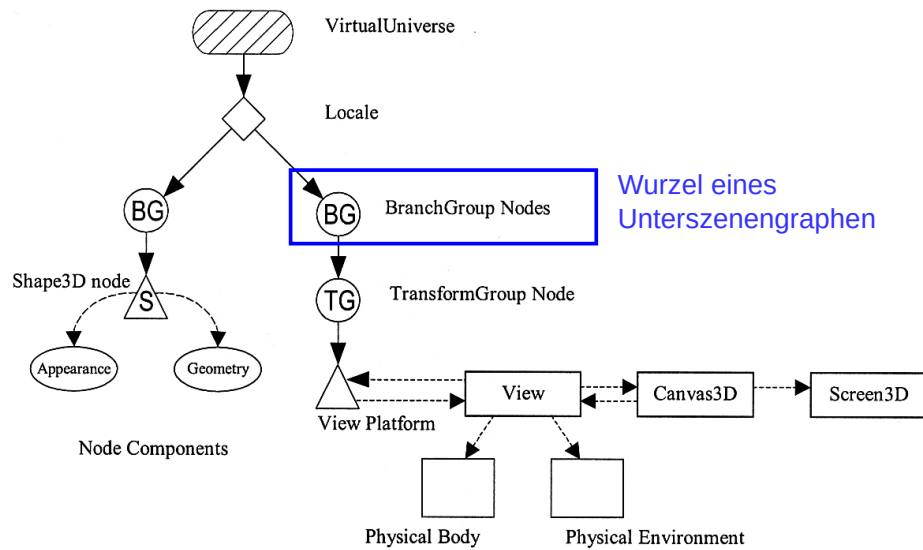
Aufbau eines Java 3D-Szenengraphen



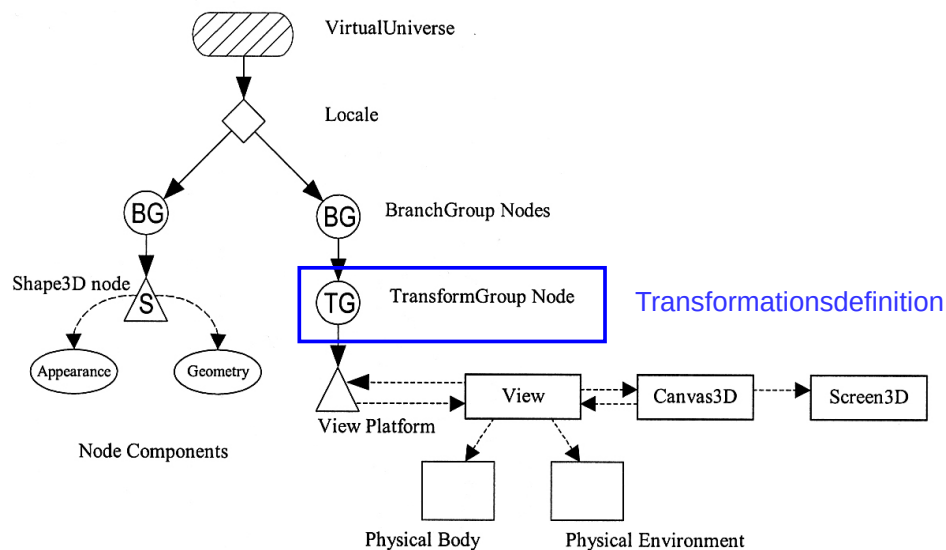
Group (Gruppenknoten):

- spezifiziert den Ort und die Orientierung von visuellen Objekten des virtuellen Universums.
- Es gibt zwei Typen von Gruppenknoten,
 - Verzweigungsgruppenknoten (BranchGroup (BG)) und
 - Transformationsgruppenknoten (TransformGroup (TG)).
- Ein Verzweigungsgruppenknoten ist die Wurzel eines Unterszenengraphen.

Aufbau eines Java 3D-Szenengraphen

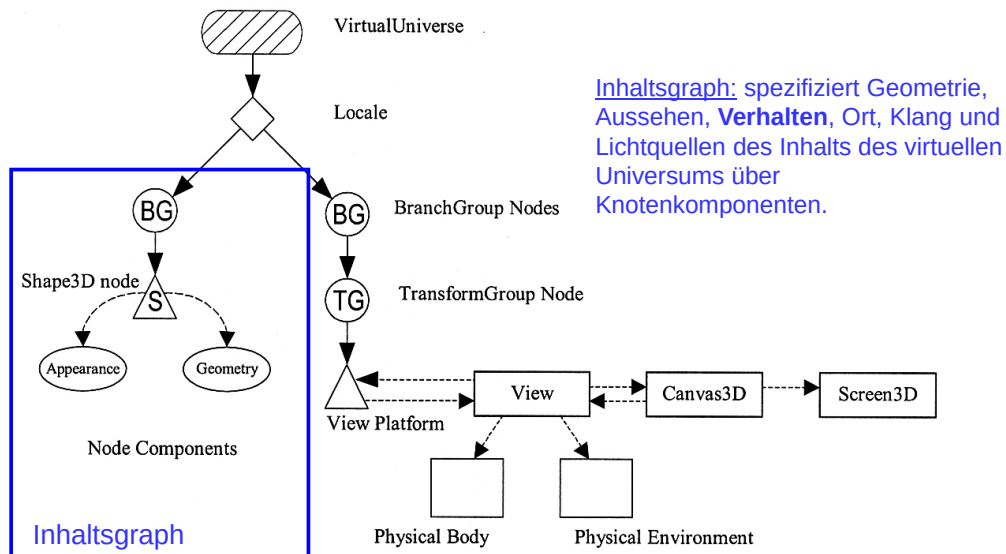


Aufbau eines Java 3D-Szenengraphen





Aufbau eines Java 3D-Szenengraphen



Leaf (Blattknoten):

spezifiziert die Gestalt, den Klang und das Verhalten von visuellen Objekten des virtuellen Universums.

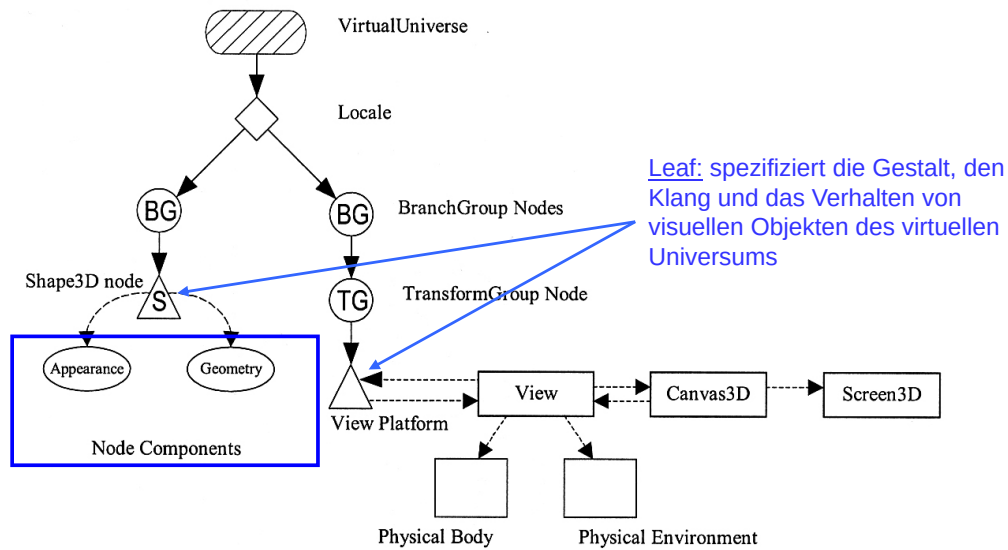
NodeComponent (Knotenkomponente):

definiert die Geometrie und das Aussehen (appearance) eines visuellen Objekts

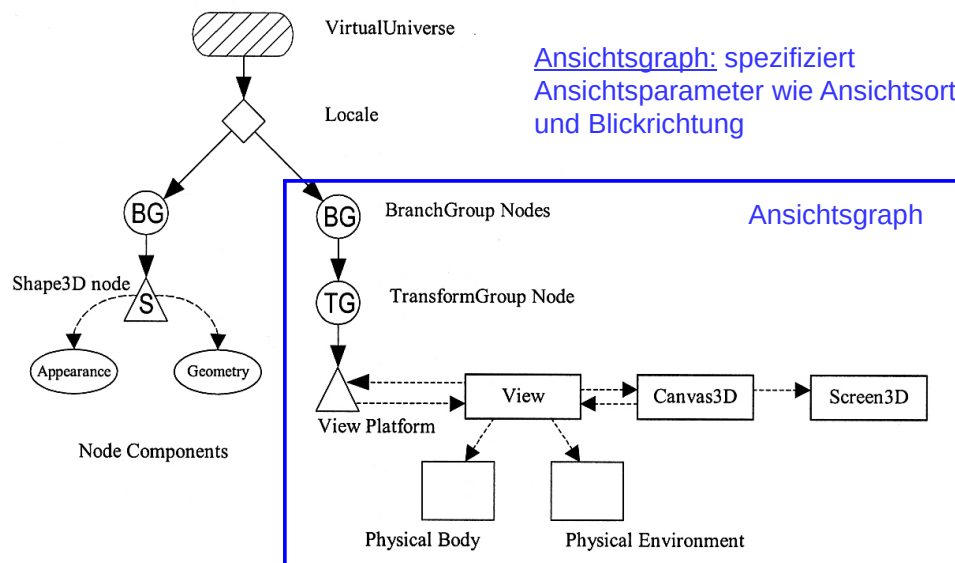
andere Objekte:

z.B. Objekte zur Ansichtsdefinition

Aufbau eines Java 3D-Szenengraphen



Aufbau eines Java 3D-Szenengraphen





Zwei Typen von Unterszenengraphen:

- **Ansichtsgraph (view branch graph):**

spezifiziert Ansichtsparameter wie Ansichtsort und Blickrichtung

- **Inhaltsgraph (content branch graph):**

spezifiziert Geometrie, Aussehen, Verhalten, Ort, Klang und Lichtquellen des Inhalts des virtuellen Universums über Knotenkomponenten.

Bemerkung:

Beide Unterszenengraphen spezifizieren die wesentlichen Parameter für die Bilderzeugung.



Struktur des Szenengraphen:

- Der Teilgraph ohne Knotenkomponenten und Referenzkanten ist ein Baum.
- Der Pfad von der Wurzel des Baums zu einem Blatt wird der Szenengraphenpfad des Blattes genannt.
Der Szenengraphenpfad definiert die Zustandsinformation des Blatts.

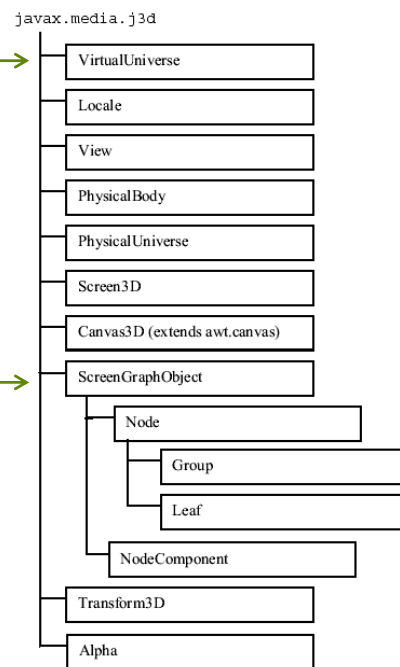
Abarbeitung eines Java 3D-Szenengraphen:

Das Java 3D-Bilderzeugungssystem stellt die Knoten unter Berücksichtigung ihres Zustands dar. Der Benutzer hat im Allgemeinen keinen Einfluss auf die Reihenfolge der Ausführung.



**Obere Stufen der Java
3D-Klassenhierarchie:**

Klasse SceneGraphObject:
hat zwei Unterklassen,
Node und *NodeComponent*



Klasse Node:

- Die Unterklassen von Node stellen die meisten Objekte des Szenengraphen zur Verfügung.
- Node ist eine abstrakte Oberklasse der Klassen Group und Leaf.

Klasse Group:

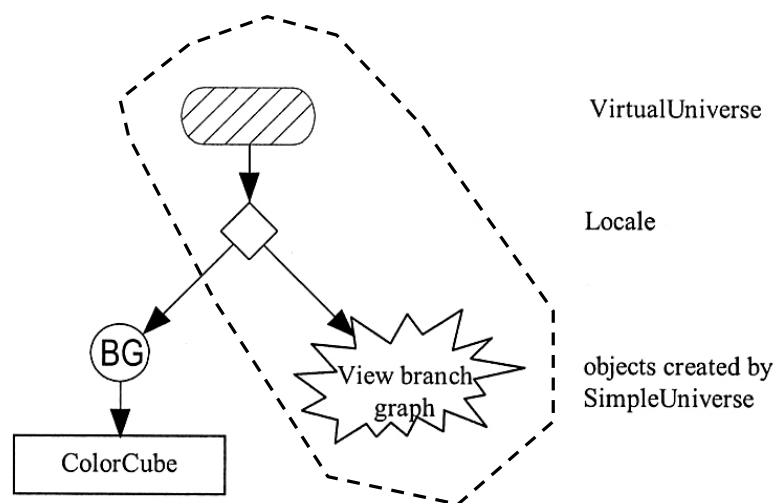
- spezifiziert den Ort und die Orientierung von visuellen Objekten des virtuellen Universums.
- wichtige Unterklassen: *BranchGroup* (BG) und *TransformGroup* (TG)

**Klasse Leaf:**

- ist die Oberklasse zur Spezifikation von Gestalt, Klang und Verhalten von visuellen Objekten des virtuellen Universums.
- hat keine Kinder, kann aber Referenzkanten zu NodeComponents haben.

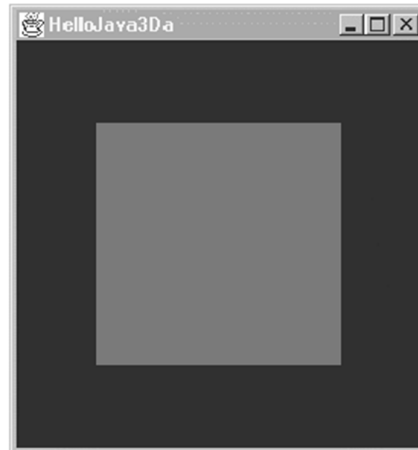
Klasse NodeComponent:

- ist die Oberklasse zur Spezifikation von Geometrie, Aussehen, Textur und optischen Materialeigenschaften eines Shape3D-Blattknotens.
- NodeComponents sind nicht Teil des Szenengraphen, können aber durch ihn referenziert werden.
- Eine NodeComponent kann von mehr als einem Shape3D-Objekt referenziert werden.

**Beispiel: Szenengraph HelloJava3Da**



durch HelloJava3Da erzeugtes Bild



Schritte beim Schreiben eines Java-Programms mit der SimpleUniverse-Klasse:

1. Erzeuge ein Canvas3D-Objekt.
2. Erzeuge ein SimpleUniverse3D-Objekt, das auf das Canvas3D-Objekt aus 1. verweist. Passe das SimpleUniverse3D-Objekt an.
3. Konstruiere Inhaltsuntergraph(en) (content branch graph(s)).
4. Compiliere die Inhaltsuntergraphen.
5. Füge die Inhaltsuntergraphen in das Locale-Objekt des SimpleUniverse ein.

**Code Fragment Class HelloJava3Da**

```
1.  public class HelloJava3Da extends Applet {
2.      public HelloJava3Da( ) {
3.          setLayout(new BorderLayout( ));
4.          GraphicsConfiguration config =
5.              SimpleUniverse.getPreferredConfiguration();
6.          Canvas3D canvas3D = new Canvas3D(config);
7.          add("Center", canvas3D);
8.
9.          BranchGroup scene = createSceneGraph();
10.         scene.compile( );
```

**Code Fragment Class HelloJava3Da**

```
11.
12.     // SimpleUniverse is a Convenience Utility class
13.     SimpleUniverse simpleU = new SimpleUniverse(canvas3D);
14.
15.     // This moves the ViewPlatform back a bit so the
16.     // objects in the scene can be viewed.
17.     simpleU.getViewingPlatform().setNominalViewingTransform();
18.
19.     simpleU.addBranchGraph(scene);
20. } // end of HelloJava3Da (constructor)
```

**Code Fragment Method createSceneGraph for Class HelloJava3Da**

```
1. public BranchGroup createSceneGraph( ) {
2.     // Create the root of the branch graph
3.     BranchGroup objRoot = new BranchGroup( );
4.
5.     // Create a simple shape leaf node, add it to the
        // scene graph.
6.     // ColorCube is a Convenience Utility class
7.     objRoot.addChild(new ColorCube(0.4));
8.
9.     return objRoot;
10. } // end of createSceneGraph method of HelloJava3Da
11. } // end of class HelloJava3Da
```

**Code Fragment Main() Method of HelloJava3Da Invokes MainFrame**

```
1. // The following allows this to be run as an application
2. // as well as an applet
3.
4. public static void main(String[ ] args) {
5.     Frame frame=new MainFrame(new HelloJava3Da( ),256,256);
6. } //      end of main (method of HelloJava3Da)
```




Code Fragment Import Statements für HelloJava3Da.java

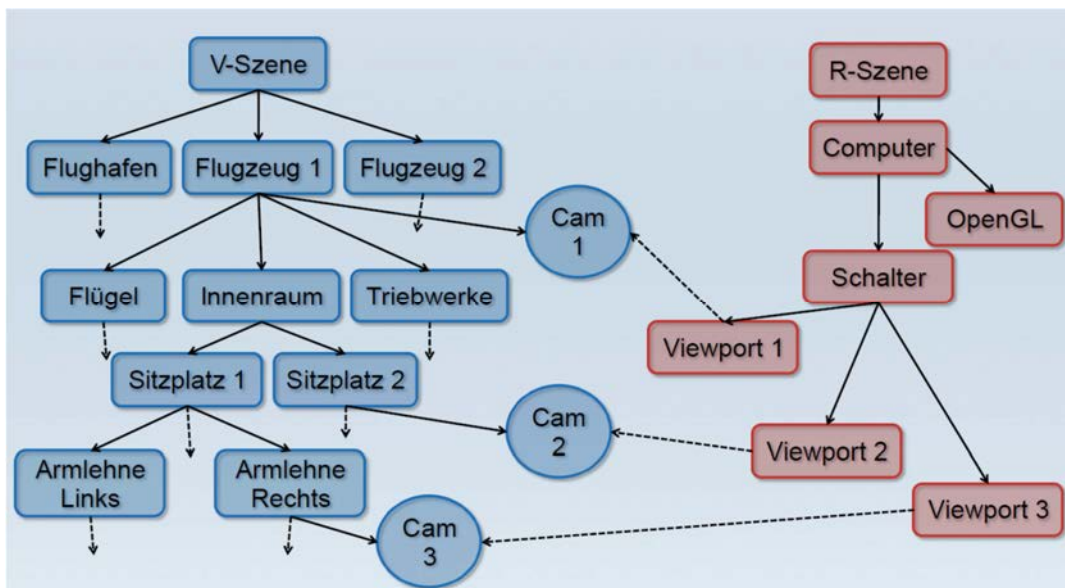
```

1. import java.applet.Applet;
2. import java.awt.BorderLayout;
3. import java.awt.Frame;
4. import java.awt.event.*;
5. import java.awt.GraphicsConfiguration;
6. import com.sun.j3d.utils.applet.MainFrame;
7. import com.sun.j3d.utils.universe.*;
8. import com.sun.j3d.utils.geometry.ColorCube;
9. import javax.media.j3d.*;
10. import javax.vecmath.*;

```



Beispiel (Virtuelle und reale Szene)



E.1. Graphik-Pipeline-basierte Bilderzeugung

- E.1.1. Einleitung
- E.1.2. Graphik-Pipeline
- E.1.3. Grundkonzepte von OpenGL
- E.1.4. Pufferspeicherobjekte
- E.1.5. Shader
- E.1.6. Vereinfachte OpenGL-Shader Pipeline
- E.1.7. Einbindung von Shader-Programmen

E.2. Szenengraphen

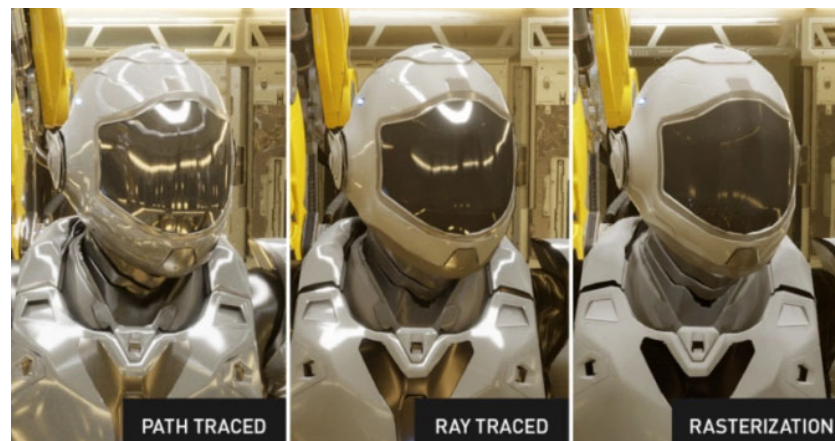
- E.2.1. Einleitung
- E.2.2. Aufbau von Szenengraphen
- E.2.3. Java 3D Szenengraphen
- E.2.4. Beispiel: HelloJava3Da
- E.2.5. Virtuelle und reale Szene

E.3. Strahlverfolgungsbasierte Bilderzeugung

- E.3.1. Einführung
- E.3.2. Strahlverfolgungsverfahren
- E.3.3. Anti-Alias-Behandlung
- E.3.4. Strahl-Dreieck-Schnitt
- E.3.5. Spezifikation von Eingabeszenen
- E.3.6. Effiziente Strahlverfolgung
- E.3.7. Fotorealistisches Rendering

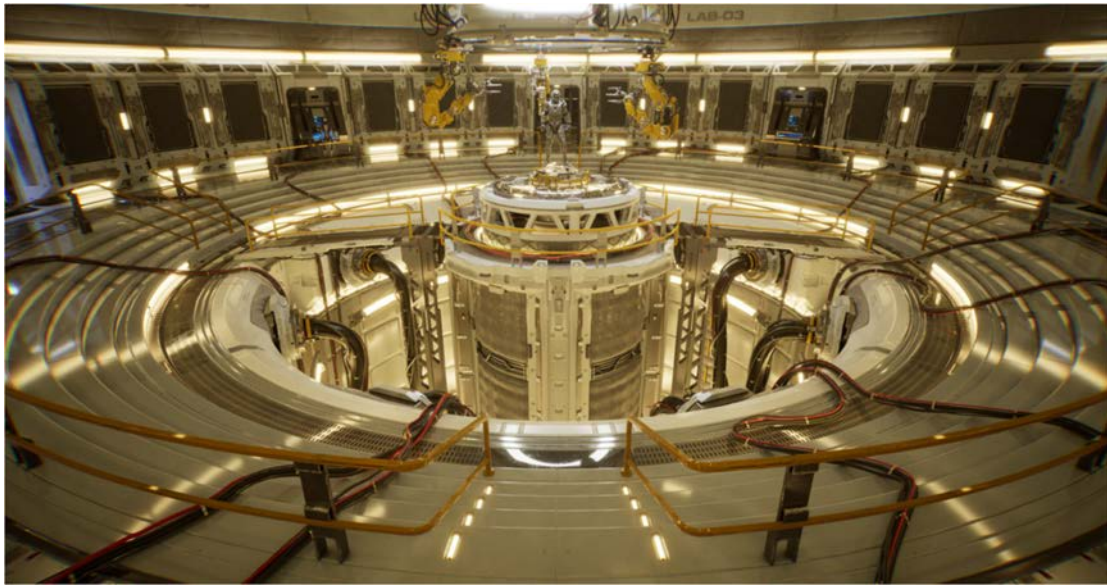
E.3. Strahlverfolgungsbasierte Bilderzeugung **E.3.1. Einführung****Strahlverfolgungsbasierte Bilderzeugung****Ziel:**

mehr Realismus durch Simulation von Schlagschatten, Spiegelung und Brechung

Beispiel:

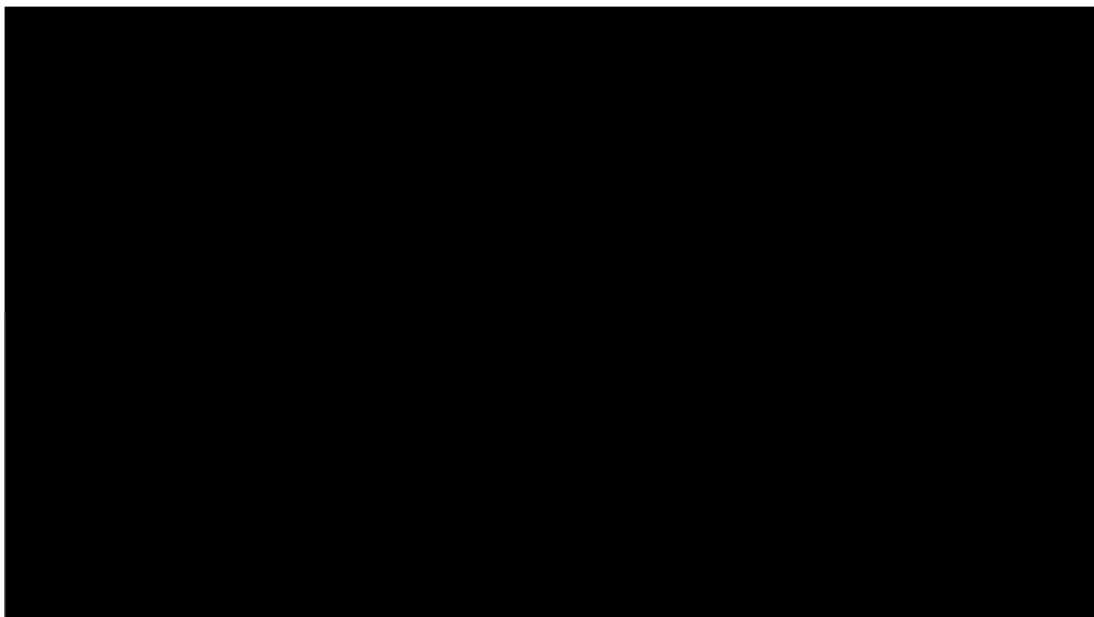
Quelle: nvidia

Beispiel:



Quelle: nvidia

Beispiel:

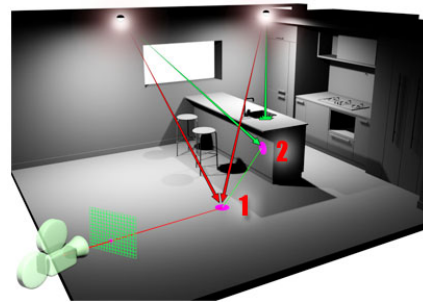


Globale Beleuchtungsmodelle

• Ray Tracing (Strahlverfolgung)

„From Eye to Light“

- Reflexionen
- Spiegelungen
- Schatten (implizit)

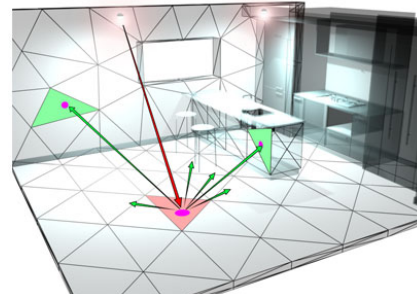


Prinzip der Strahlverfolgung

• Radiosity (Strahlungsverfahren)

„From Light to Surface“

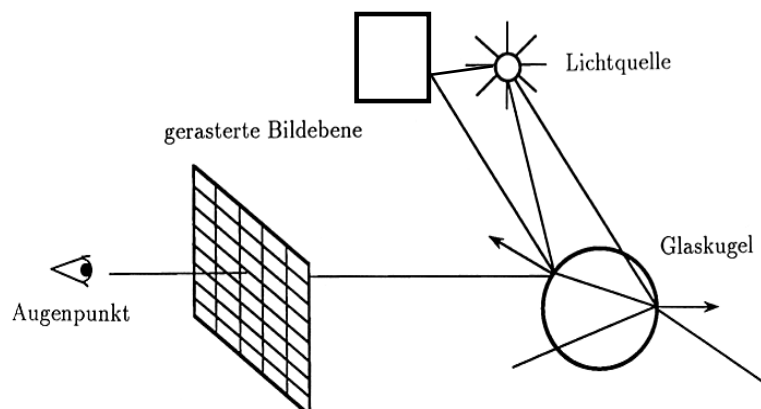
- Indirektes Licht
- Licht über mehrere diffuse Flächen
- Photon Mapping
- Kaustiken

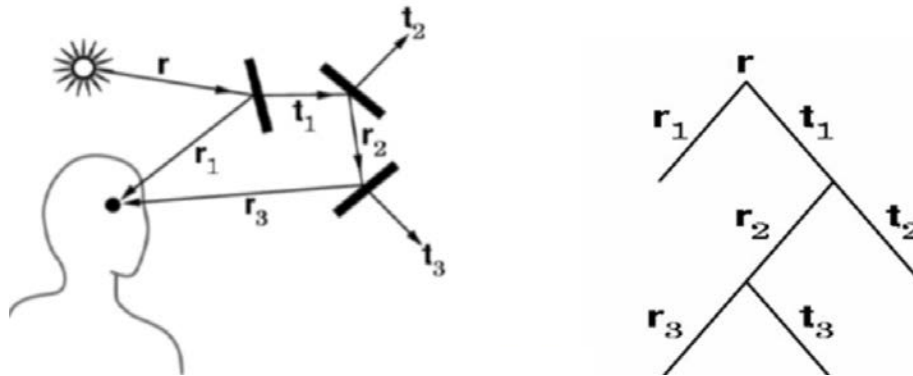


Prinzip des Strahlungsverfahrens

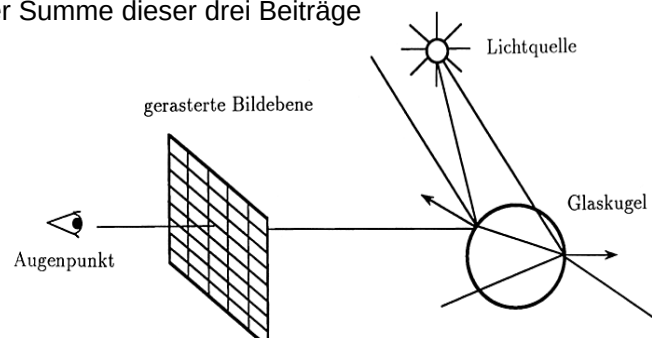
Quelle: Autodesk 3ds Max 2012

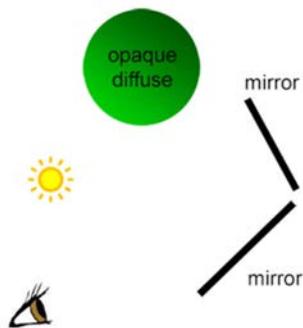
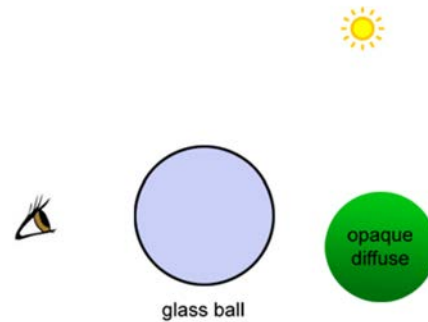
Strahlverfolgungsverfahren:



Strahlverfolgungsverfahren:**Basisalgorithmus**

- schicke einen Strahl vom Betrachterstandpunkt durch jedes Pixel der Bildebene
- bestimme den Schnittpunkt des Strahls mit einem Objekt der Szene und am Schnittpunkt den Beitrag ❶ des lokalen Beleuchtungsmodells (Phong)
- berechne Strahl in Reflexionsrichtung und evtl. in Brechungsrichtung
- Berechne den Beitrag von Reflexion ❷ und Brechung ❸ rekursiv durch Verfolgen dieser beiden Strahlen
- Beleuchtung am Schnittpunkt (und damit Farbe des Pixels) ergibt sich aus gewichteter Summe dieser drei Beiträge



Basialgorithmus: GrundkonzepteReflektion durch
einen SpiegelReflektion bei einem
Objekt vom Typ „Glas“Quelle: www.scratchapixel.com**Eingabe:**

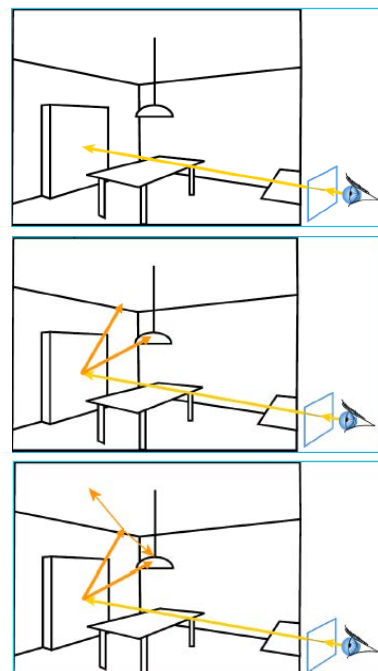
- 3D-Szene S aus Flächen,
- l Punktlichtquellen $L_i, i = 1, \dots, l$,
- Oberflächenattribute,
- Augenpunkt a ,
- Projektionsebene
- Bildauflösung $m \times n$.

Ausgabe: $m \times n$ -Rasterbild**Algorithmus:**

- ein Strahl pro Pixel
- Reflexionsrichtung verfolgen,
Strahlen zu den Lichtquellen
- Rekursion

Bedingungen der Rekursion:

- vorgegebene maximale Rekursionstiefe erreicht
- Dämpfung auf dem Sehweg vom Augenpunkt zum
aktuellen Punkt stärker als gegebene Schranke.



**ALGORITHMUS** (Strahlverfolgung)

Eingabe: Eine 3D-Szene S aus Flächen, l Punktlichtquellen L_i , $i = 1, \dots, l$, Oberflächenattribute, ein Augenpunkt $\mathbf{a}$, eine Projektionsebene und eine Bildauflösung $m \times n$.

Ausgabe: ein $m \times n$ -Rasterbild.

Ablauf:**Datenstrukturen:**

VAR Bild: **ARRAY** [1.. m ,1.. n] **OF** colortype.

UNTERALGORITHMUS **FindeSchnitt**($\mathbf{p}$, $\mathbf{v}$: point,
VAR $\mathbf{q}$: point): **BOOLEAN**;

(* liefert **TRUE**, falls der von $\mathbf{p}$ in Richtung $\mathbf{v}$ gehende Strahl eine Fläche schneidet; $\mathbf{q}$ ist dann der Schnittpunkt, der $\mathbf{p}$ am nächsten liegt. **FindeSchnitt** hängt vom Flächentyp ab und ist deshalb hier nicht weiter ausgeführt.*)

**UNTERALGORITHMUS** **Strahlverfolgung**(

$\mathbf{p}$, $\mathbf{v}$: Punkt, **VAR** $\mathbf{I}$: Intensität);

(* $\mathbf{p}$ ist der Ursprung, $\mathbf{v}$ die Richtung des verfolgten Strahls. $\mathbf{I}$ ist die Intensität, die für diesen Strahl berechnet wird . *)

BEGIN

gefunden := **FindeSchnitt**($\mathbf{p}$, $\mathbf{v}$, $\mathbf{q}$);

IF gefunden **THEN BEGIN**

FOR alle Lichtquellen L_i **DO BEGIN**

bestimme die Richtung $\mathbf{l}_i$ von $\mathbf{q}$ nach L_i ;

gefunden := **FindeSchnitt**($\mathbf{q}$, $\mathbf{l}_i$, $\mathbf{q}^*$);

IF NOT gefunden **THEN**

berechne die neue Intensität $\mathbf{I}$ aus dem alten Wert von $\mathbf{I}$ und der Intensität, die von L_i kommt durch Anwenden des Beleuchtungsmodells

END;

```

IF nicht Abbruch( ) THEN BEGIN
  IF die Fläche bei q ein Spiegel ist THEN BEGIN
    bestimme die Reflexionsrichtung r zu v nach dem Reflexionsgesetz;
    Strahlverfolgung(q, r, I*);
    berechne die neue Intensität I aus dem alten Wert von I und der
    Intensität I* (z.B. durch Addition) nach dem Beleuchtungsmodell
  END;
  IF die Oberfläche an q durchsichtig ist THEN BEGIN
    berechne die Brechungsrichtung b von v nach dem Brechungsgesetz;
    Strahlverfolgung(q, b, I*);
    berechne die neue Intensität I aus dem alten Wert von I und der
    Intensität I* (z.B. durch Addition) nach dem Beleuchtungsmodell END
  END
END

```

```

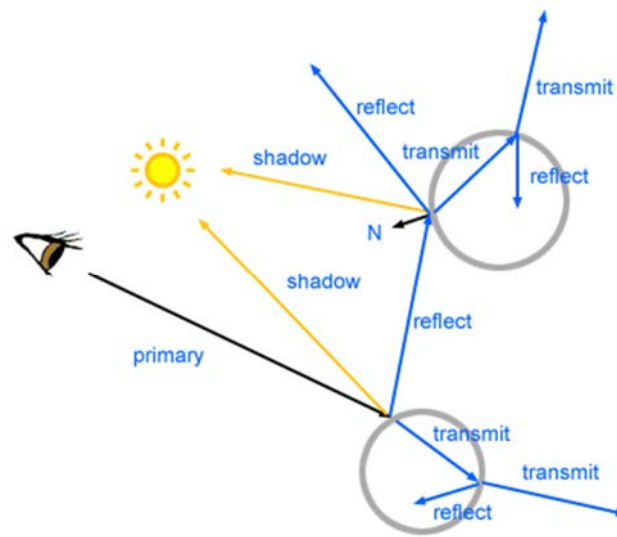
HAUPTALGORITHMUS:
BEGIN
  FOR i:=1 TO m DO
    FOR j:=1 TO n DO BEGIN
      bestimme die Richtung v des Sehstrahls vom Augenpunkt a durch das
      Zentrum von Pixel [i, j];
      Strahlverfolgung (a, v, IMAGE[i, j])
    END
  END.

```

Bedingungen für Abbruch():

- Es wurde eine vorgegebene maximale Rekursionstiefe erreicht, z.B. 5.
- Die Dämpfung auf dem Sehwegs vom Augenpunkt bis zum aktuellen Punkt **p** ist stärker als eine gegebene Schranke.

Zusammenfassung zu den Strahltypen

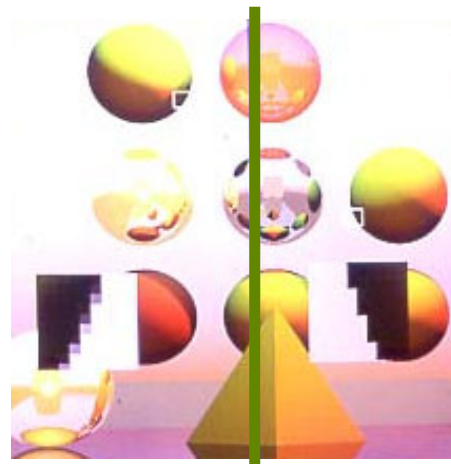
**Alias-Effekt:**

unerwünschte Muster,
hervorgerufen durch zu geringe
Abtastung durch das Pixelgitter
des Bildes Kap. C.10.3

Anti-Alias-Behandlung:

Elimination durch
Tiefpassfilterung der gegebenen
Szene, siehe glatte Linien in
Kap. C.10.4

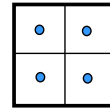
im Folgenden: Abtastung mit
einem verfeinerten Raster
(„Super-Sampling“) mit
anschließender Tiefpassfilterung
durch Mittelung



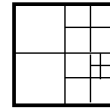
Anti-Alias-behandelt nicht Anti-Alias-behandelt

**Anti-Alias-Behandlung durch Super-Sampling:****nicht adaptiv:**

Das Pixel wird in eine feste Anzahl Unterpixel unterteilt, für die Intensitäten berechnet werden, die gewichtet aufsummiert werden.

**adaptiv:**

Verfolge mehr als einen Strahl pro Pixel und
mittle die gelieferten Farbwerte gewichtet nach Flächenanteil.

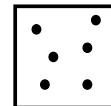


Bsp.:

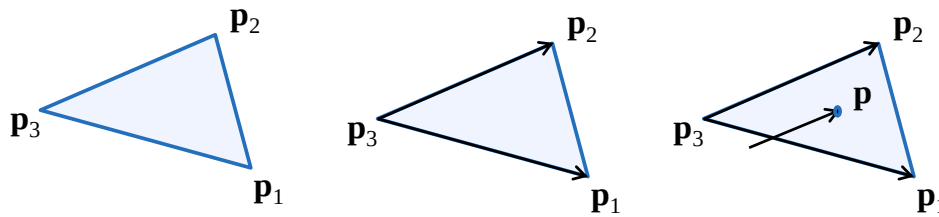
1. Berechne die Intensitäten an den Mittelpunkten der vier Unterrechtecke des als Rechteck aufgefassten Pixels.
2. Wenn die Intensitätsunterschiede stark sind, unterteile das Pixel in vier gleiche große Teilpixel und verfahre mit diesen auf dieselbe Weise.
3. Andernfalls berechne die Pixelintensität als Mittel der vier Unterpixelintensitäten.

**stochastisch:**

Die Strahlen werden gleichverteilt über das Pixel zufällig verteilt geschossen und die gefundenen Intensitätswerte gleichgewichtet aufsummiert.



Baryzentrische Koordinaten μ, ν



$$\mathbf{p} = \mathbf{p}_3 + \mu (\mathbf{p}_1 - \mathbf{p}_3) + \nu (\mathbf{p}_2 - \mathbf{p}_3) = \mu \mathbf{p}_1 + \nu \mathbf{p}_2 + (1 - \mu - \nu) \mathbf{p}_3$$

$$0 \leq \mu, \nu, \mu + \nu \leq 1$$

 www.geogebra.org

Strahl-Dreieck-Schnitt

Für ein Dreieck mit Eckpunkten $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ führt das Strahl-Flächen-Schnittproblem bei Verwendung von baryzentrischen Koordinaten μ, ν auf das lineare Gleichungssystem

$$\mathbf{a} + w \cdot \mathbf{w} = \mu \mathbf{p}_1 + \nu \mathbf{p}_2 + (1 - \mu - \nu) \mathbf{p}_3, \quad 0 \leq \mu, \nu, \mu + \nu \leq 1,$$

mit der Lösung

$$w = \frac{|\mathbf{p}_1 - \mathbf{p}_3 \quad \mathbf{p}_2 - \mathbf{p}_3 \quad \mathbf{a} - \mathbf{p}_3|}{|\mathbf{p}_1 - \mathbf{p}_3 \quad \mathbf{p}_2 - \mathbf{p}_3 \quad \mathbf{w}|}.$$

(Lösung erhalten durch Anwendung der Cramer'schen Regel)

Beschreibung der Eingabeszene: Shading-Sprachen**Shading-Sprachen** (engl. *shading language*):

Beschreibung der Schnittstelle zwischen Modellierprogrammen und Bilderzeugungsprogrammen

Bsp.: Renderman:

- technische Schnittstelle zwischen Modellierprogrammen und Bilderzeugungsprogrammen
- entwickelt von Pixar
- Information: <https://renderman.pixar.com>

Schwierigkeit der Strahlverfolgung: Zeitbedarf

mit 10^{-7} Sekunden/Operation: $5 \cdot 10^4$ Sekunden,
d.h. etwa 14 Std. $\rightarrow$ inpraktikabel

**Lösung:**

- A. Effiziente Ausführung des Tests auf Schnitt zwischen Strahl und Szenenelement

DETAILS:

Suffern, Kevin. Ray Tracing from the Ground up. CRC Press, 2016

Lengyel, Eric. Foundations of Game Engine Development, Volume 2: Rendering, Terathon Software LLC, 2019

- B. Reduzierung der Anzahl der auszuführenden Schnittpoints durch geeignete Suchdatenstrukturen.

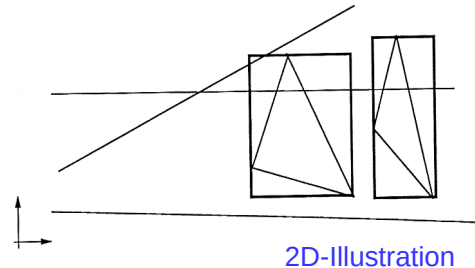
**Effiziente Strahlverfolgung** (Reduktion des Rechenaufwandes für Schnittpoints)

- a. Effiziente Ausführung des Tests auf Schnitt zwischen Strahl und Szenenelement
- Hüllquader
- b. Reduzierung der Anzahl der auszuführenden Schnittpoints durch geeignete Suchdatenstrukturen
- geschachtelte Hüllquader
 - reguläres, hierarchische oder geschachtelte Gitter
 - kd-Baum
- c. Strahlverfolgung in Echtzeit (Real Time Ray Tracing)
- Ausnutzen interner Parallelverarbeitung von PC-Prozessoren und Graphikprozessoren
 - Parallelverarbeitung auf einem schnellen Netzwerk von PCs

zu A. Effiziente Ausführung des Schnittests zwischen Strahl und Szenenobjekten

Hüllquader

Anstelle der Durchführung des Schnittests mit einem gegebenen Szenenelement wird zunächst ein dieses umgebender Quader mit dem Strahl auf Schnitt getestet. Erst wenn hier ein Schnitt festgestellt wird, wird der Schnittest mit dem eigentlichen Element ausgeführt.



Vorteil:

Falls häufig kein Schnitt gefunden wird und der Schnittest für das gegebene Element relativ aufwendig ist, kann durch den schnell ausführbaren Schnittest Strahl/Quader signifikant Rechenzeit gespart werden.

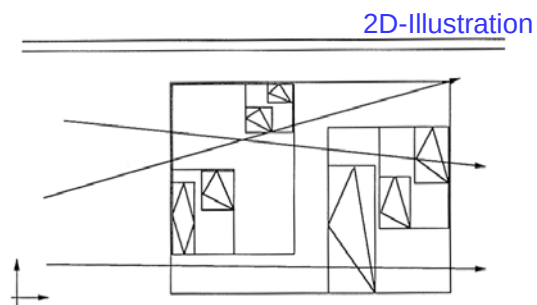
Nachteil:

Die Hüllquader werden vorberechnet und gespeichert. Dadurch wird zusätzlicher Speicherplatz für die Quader benötigt.

zu B. Reduzierung der Anzahl der auszuführenden Schnittests

Geschachtelte Hüllquader:

Die gegebene Szene wird in zwei Teilszenen zerlegt, die aus jeweils räumlich benachbarten Elementen bestehen. Die Teilszenen werden mit Hüllquadern versehen. Mit den beiden Teilszenen wird analog verfahren.



Ergebnis ist eine Baumstruktur mit Hüllquadern als inneren Knoten und den gegebenen Szenenelementen als Blätter. Die Wurzel ist der Hüllquader der Gesamtszene. Die Quader längs eines Pfades im Baum sind ineinander geschachtelt. Der Baum wird in einem Vorverarbeitungsschritt berechnet und gespeichert.

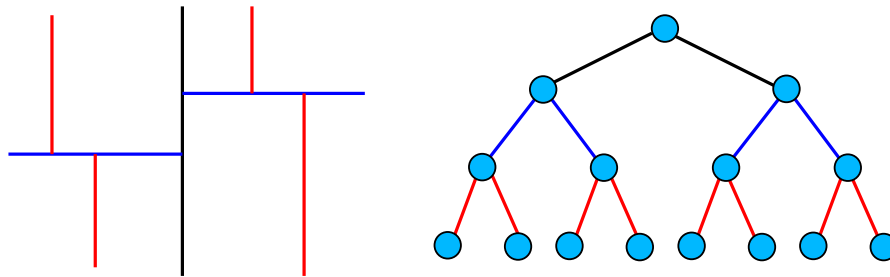
Bei der Strahlverfolgung werden ausgehend von der Wurzel die vom Strahl geschnittenen Hüllquader bestimmt. Die Rechenzeiterparnis ergibt sich dadurch, dass ein Teilbaum nicht weiter untersucht werden muss, wenn sein Wurzelquader nicht vom Strahl geschnitten wird.

**kd-Baum:**

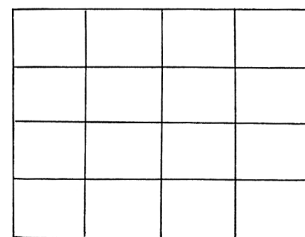
Binärbaum, der den Szenenraum iterativ durch achsenparallele Ebenen in zwei Teile aufteilt.

Dabei geschieht die Teilung alternierend in den Koordinatenrichtungen.

Beispiel: Illustration eines 2-D-Baums (in der Anwendung ist $k=3$)

**Reguläres Gitter:**

In einem Vorverarbeitungsschritt wird der Hüllquader der gegebenen Szene durch äquidistante Unterteilung seiner Seiten in ein Gitter aus kongruenten quaderförmigen Gitterzellen zerlegt. Für jede Zelle werden die sie schneidenden Szenenelemente bestimmt und gespeichert.

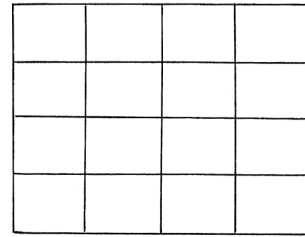


2D-Illustration

Als Datenstruktur kann ein dreidimensionales Array verwendet werden. Seine Elemente enthalten Listen aus Verweisen auf die Szenenelemente, die von der entsprechenden Zelle geschnitten werden.

Die beschreibenden Daten der Szenenelemente sind in einer Szenenliste abgelegt.

Zur Strahlverfolgung werden die vom Strahl durchlaufenen Zellen in Strahlrichtung bestimmt. Für dabei gefundene nichtleere Zellen wird der Schnittpunkt mit den dort gefundenen Szenenelementen ausgeführt. Wird ein Treffer gefunden, wird maximal soweit durch das Gitter weiter fortgeschritten, bis eine Zelle hinter dem bisher gefundenen dem Strahlanfang am nächsten liegenden Schnittpunkt erreicht wird.



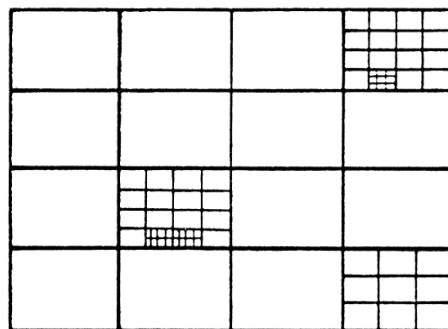
2D-Illustration

Die von einem Strahl getroffenen Zellen können durch inkrementelle Additionen effizient aufgefunden werden:

Schwierigkeit: Das reguläre Gitter bringt nur dann eine Beschleunigung der Berechnung, wenn sich die Szenenelemente nicht in Gitterzellen häufen.

Hierarchische Gitter (Gitterbaum):

Gitterzellen, in denen sich bei der Verteilung nach der Strategie des regulären Gitters Szenenelemente häufen, werden ihrerseits wieder durch ein reguläres Gitter zerlegt. Bei erneuter Häufung wird die Zerlegung in den betroffenen Zellen



2D-Illustration

Spezialfall: Ein $2 \times 2 \times 2$ -Gitter führt auf die sogenannte *Octree-Zerlegung*.

Verallgemeinerung der Octree-Zerlegung:

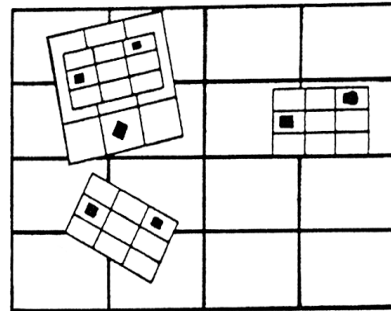
Binary Space Partitioning (BSP-Baum, kd-Baum)

Geschachtelte Gitter:

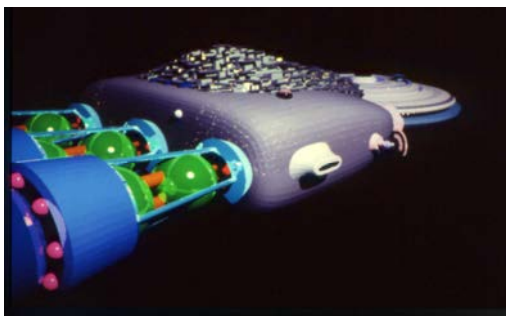
lassen sich in natürlicher Weise bei hierarchisch aus Teilszenen zusammengesetzten Szenen anwenden.

Jede Teilszene wird in ein einfaches reguläres Gitter vorverarbeitet. Eine Unterszene wird im Gitter der Szene, in der sie verwendet wird, durch ihren Hüllquader repräsentiert.

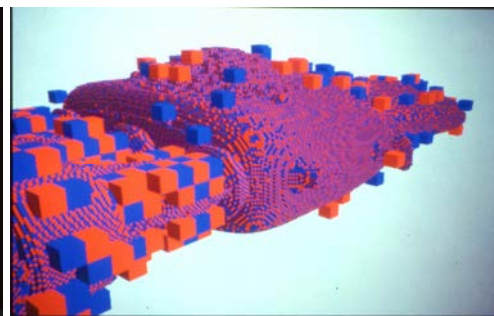
Wird bei der Strahlverfolgung in einer Gitterzelle ein Hüllquader einer Unterszene gefunden, wird zur Durchführung des Schnitttests mit der Unterszene der Strahl in deren Gitter verfolgt. Anschließend wird durch Verfolgung im ursprünglichen Gitter fortgesetzt.



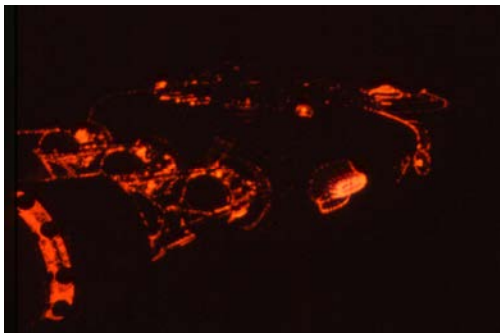
2D-Illustration

Beispiel: Gitterbaum

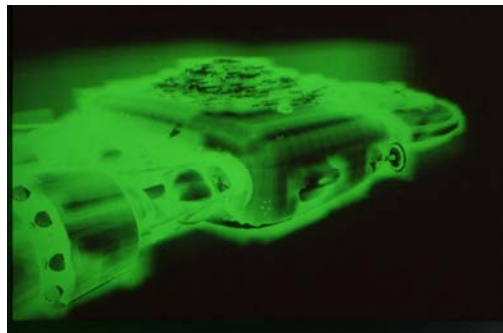
Szene: ein Raumschiff



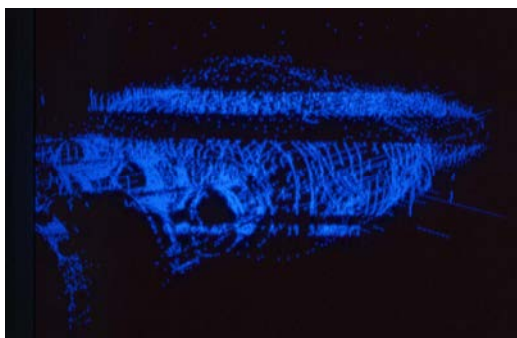
Einhüllender Gitterbaum für das Raumschiff



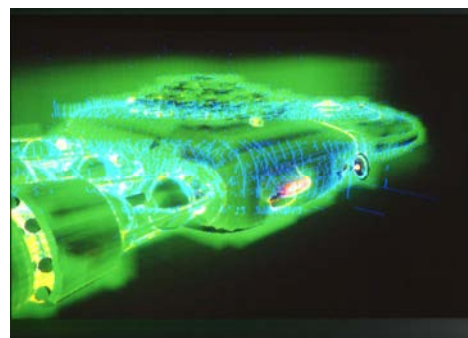
Visualisierung der Anzahl von Schnitttests (hell = aufwendig)



Visualisierung der durchlaufenen Anzahl von Zellen (hell = aufwendig)



Visualisierung von Sekundärspeicherzugriffen (1 Punkt pro Zugriff)



Visualisierung des Gesamtaufwands durch Überlagerung (hell = aufwendig)

**Worst-Case-effiziente Lösungen:****Strahlanfrageproblem:**

Gegeben: eine Menge von geometrischen Objekten im Raum, z.B. Dreiecke.

Gesucht: eine Datenstruktur für die Menge der geometrischen Objekte, die es erlaubt, für einen beliebigen Strahl den ersten Schnittpunkt mit einem Element der gegebenen Menge schnell zu finden.

Lösungen:

- H. Müller, Realistische Computergraphik, Springer-Verlag, 1988
- H. Müller, J. Winckler, Distributed Image Synthesis with Breadth-First Ray Tracing and the Ray-z-Buffer, Lecture Notes in Computer Science 594, Springer-Verlag, 1992
- M. Pellegrini, Stabbing and Ray Shooting in 3 Dimensional Space, in: Proc. 6th ACM Conference on Computational Geometry, 1990, 177-186
- M. de Berg, D. Halperin, M.H. Overmars, J. Snoeyink, M. van Krefeld, Efficient Ray Shooting and Hidden Surface Removal, Proc. 7th ACM Symp. on Computational Geometry, 1991, 21-30

**Strahlverfolgung in Echtzeit (Real Time Ray Tracing):**

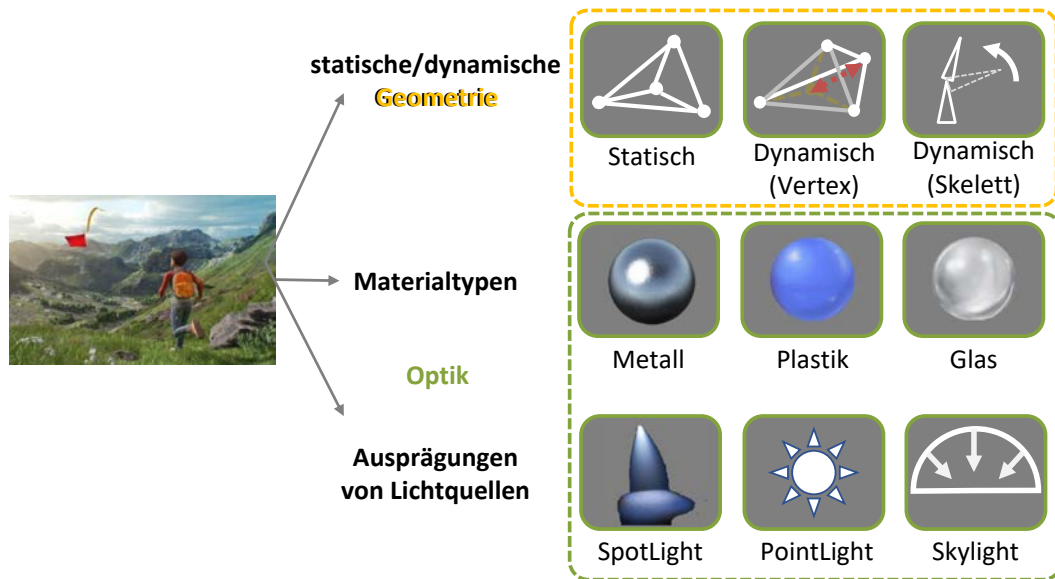
- Ausnutzen interner Parallelverarbeitung von PC-Prozessoren und Graphikprozessoren
- Parallelverarbeitung auf einem schnellen Netzwerk von PCs
- Engpass bei interaktiver Graphik: Aktualisierung oder Neuaufbau der Datenstruktur für die effiziente Strahlverfolgung bei jedem neuen Bild.

LITERATUR

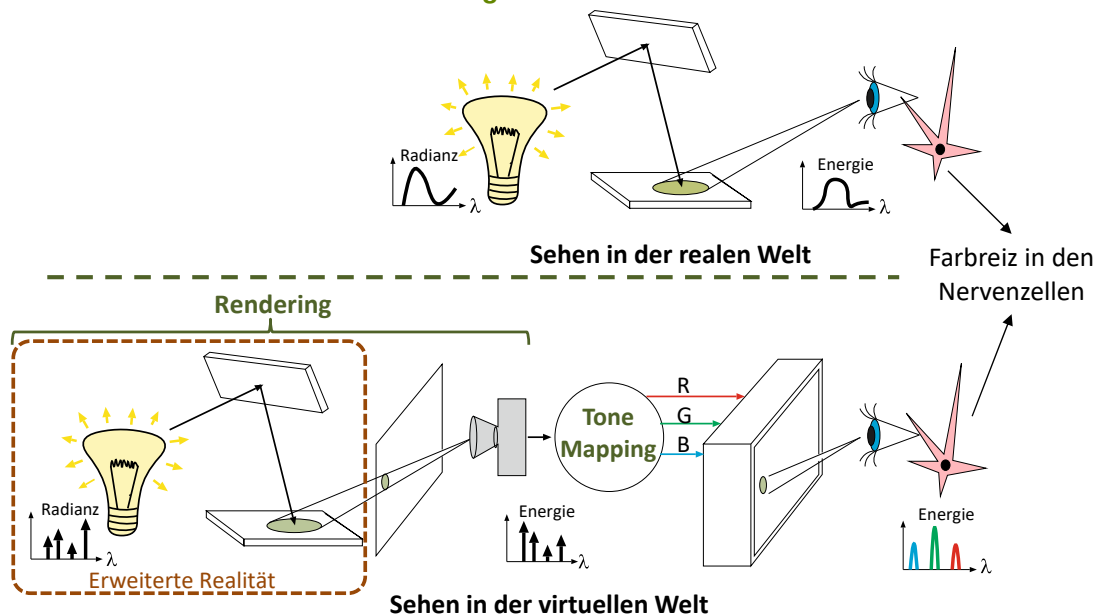
I. Wald, T.J. Purcell, J. Schmittler, C. Benthin, P. Slusallek, Realtime Ray Tracing and its use for Interactive Global Illumination, Eurographics State of the Art Report, 2003, <http://www.openrt.de/>

I. Wald, W. R. Mark, J. Günther, S. Boulos, T. Ize, W. Hunt, S. G Parker, P. Shirley, State of the Art in Ray Tracing Animated Scenes, Eurographics 2007 State of the Art Reports, 2007, <http://www.sci.utah.edu/~wald/Publications/>

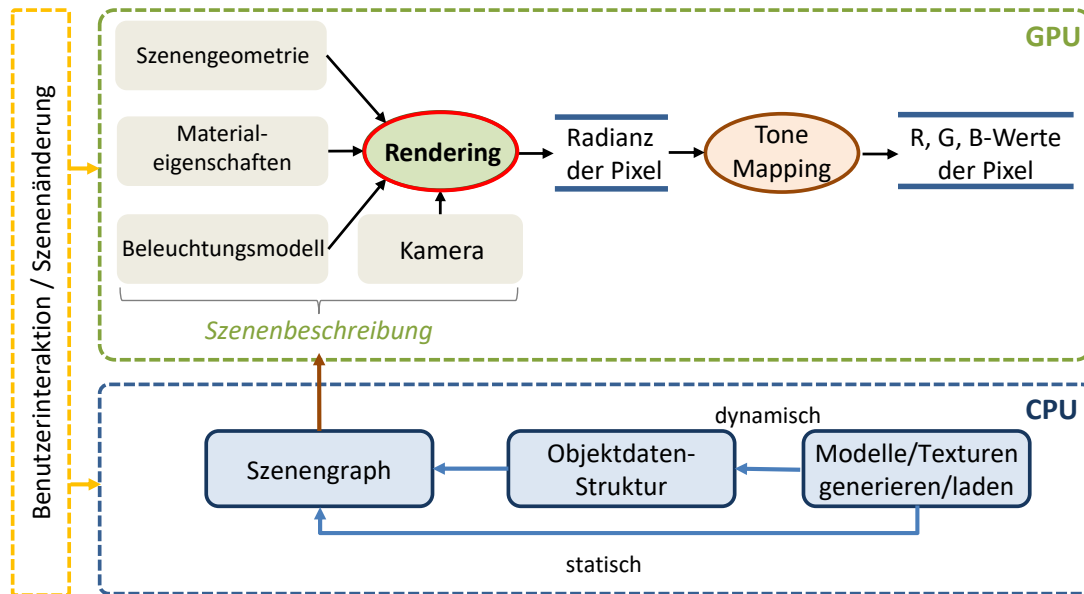
Dekomposition der Bild-basierten Szene



Reale vs. virtuelle Wahrnehmung

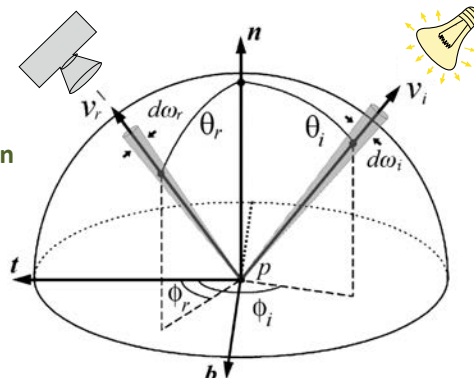


Daten-/Prozessmodell der Bilderzeugung



Allgemeine Bilderzeugungsgleichung nach Kajiya

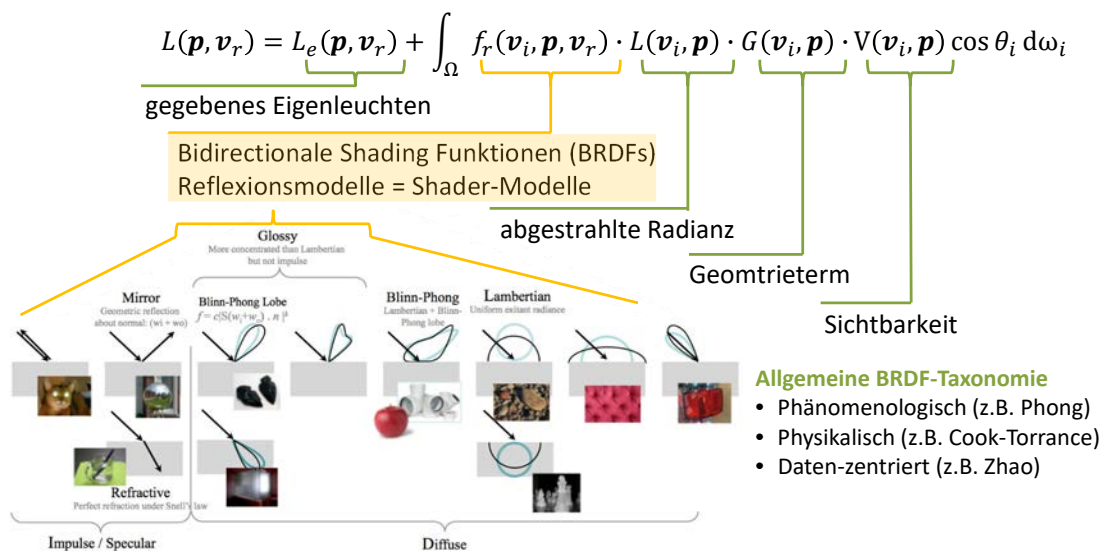
- Schematische Repräsentation gemäß der Bilderzeugung



- Radianz-Gleichung

$$L(\mathbf{p}, \mathbf{v}_r) = L_e(\mathbf{p}, \mathbf{v}_r) + \int_{\Omega} f_r(\mathbf{v}_i, \mathbf{p}, \mathbf{v}_r) \cdot L(\mathbf{v}_i, \mathbf{p}) \cdot G(\mathbf{v}_i, \mathbf{p}) \cdot V(\mathbf{v}_i, \mathbf{p}) \cos \theta_i d\omega_i$$

Radianz-Gleichung (Allgemeine Bilderzeugungsgleichung)



Vorgehensweise beim Strahlungsverfahren:

Simulation des Lichttransports zwischen den Oberflächen der Szene.

Grundlagen:

- (alternative) **Strahlungsgleichung** (ohne Fläche A_i des Flächenelements i)
- Energieerhaltungssatz

Finite Elemente Verfahren: Unterteilen der Szene in „Patches“

Prinzip:

Berechnung des Anteils an Licht, das ausgehend von einem Patch ein anderes trifft

- nur diffuse Reflexion betrachtet
- Anteil des Lichts nur abhängig von Geometrie – aufwendig zu berechnen

Strahlungsgleichung: $B_i = E_i + \rho_i \sum_{j=1}^n B_j F_{i,j}$

mit

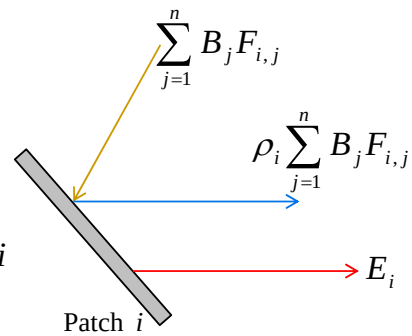
B_i : Strahlung des Patches i

E_i : Eigenemission des Patches i

ρ_i : diffuser Reflektionskoeffizient des Patches i

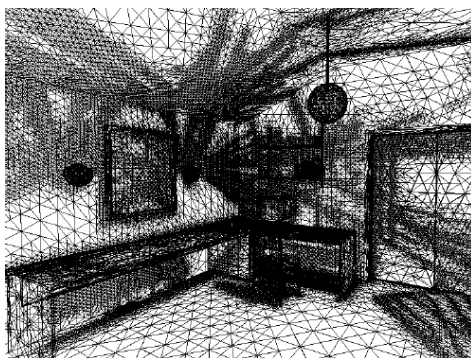
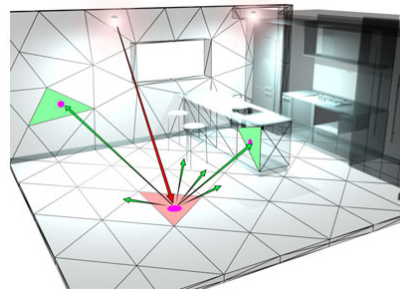
$F_{j,i}$: Formfaktor von Patch j nach Patch i

n : Zahl der Patches



Algorithmus

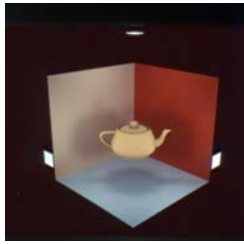
- starte mit den Patches, die Licht aussenden (Lichtquellen)
- sende von diesen Licht in die Szene
- berechne für alle anderen Patches ankommende Lichtmenge
- iteriere bis Energiegleichgewicht



Szene mit Patches



Beleuchtungssimulation

**Beispiel: Radiosity (Strahlungsverfahren)**

„Cornell-Box“



Pietrek, Informatik VII, Univ. Dortmund, 1992



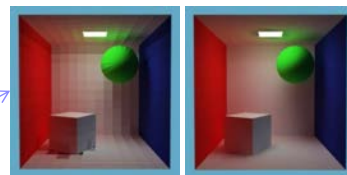
FhG IGD Darmstadt



Pietrek, Informatik VII, Univ. Dortmund, 1992

**Lösung:**

- Exakte Lösung im Allgemeinen nicht möglich
- Näherungsverfahren:
 - Finite-Elemente-Methode
 - Monte-Carlo-Methode

**LITERATUR:**

Matt Pharr, Wenzel Jakob, Greg Humphreys, Physically Based Rendering: From Theory to Implementation, Morgan Kaufmann, 2016

A.S. Glassner, Principles of Digital Image Synthesis I, II, Morgan Kaufmann Publ., 1995

Materialabhängige-Reflexionsmodelle

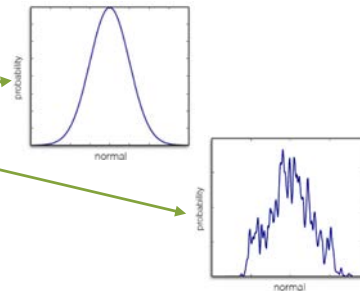
- **Isotrope Modelle:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope Modelle:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierten Lichtintensität



Verteilung der Normalen (NDF)

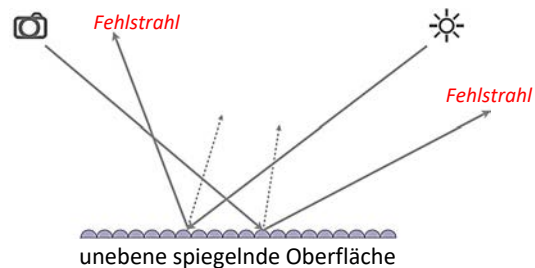


Materialabhängige-Reflexionsmodelle

- **Isotrope Modelle:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope Modelle:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierten Lichtintensität



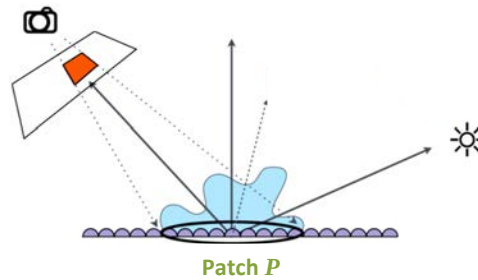
→ Samplingproblem bei der Pfadverfolgung

Materialabhängige-Reflexionsmodelle

- **Isotrope Modelle:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope Modelle:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierten Lichtintensität



Microfacetten-basierte BRDF mit P-NDF (patch normal distribution function)

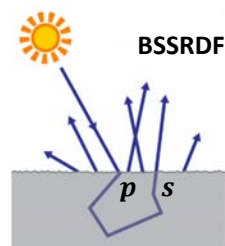
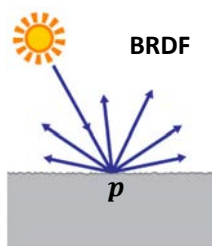
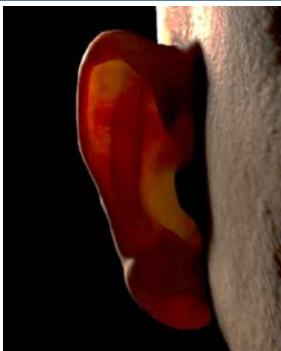
Materialabhängige-Reflexionsmodelle

- **Isotrope Modell:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope BRDFs:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierte Lichtintensität

Transparente Materialien



➡ Visuelle (ausfallende) Lichtcharakteristik resultiert durch (einfallende) Lichtcharakteristik an verschiedenen Positionen

Materialabhängige-Reflexionsmodelle

- **Isotrope Modell:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope BRDFs:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierte Lichtintensität

Transparente Materialien



Bidirectional Surface Scattering Distribution Function

- „Austritts“-Radianz an $\mathbf{p}$ in Abhängigkeit von „Eintritts“-Radianz an $\mathbf{s}$: $f_r(\mathbf{v}_i, \mathbf{p}, \mathbf{s}, \mathbf{v}_r)$
- BSSRDF integriert über alle Oberflächenpunkte und alle Richtungen

Materialabhängige-Reflexionsmodelle

- **Isotrope Modell:** Materialien mit rotationsinvarianter Reflexion
- **Anisotrope BRDFs:** Kohärenz zwischen Reflexionsvariation und Rotation

➡ Exemplarische Herausforderungen...

hochfrequente Schwankungen in der reflektierte Lichtintensität

Transparente Materialien



BRDF



BSSRDF

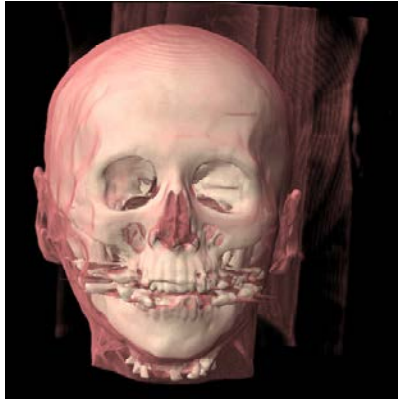
Bidirectional Surface Scattering Distribution Function

- „Austritts“-Radianz an $\mathbf{p}$ in Abhängigkeit von „Eintritts“-Radianz an $\mathbf{s}$: $f_r(\mathbf{v}_i, \mathbf{p}, \mathbf{s}, \mathbf{v}_r)$
- BSSRDF integriert über alle Oberflächenpunkte und alle Richtungen

Alternative Szenenrepräsentation: dreidimensionale Rastervolumen (Voxel-Modelle)

Bsp.: Tomographiedaten

Bilderzeugung durch Strahlverfolgung
(Ray Casting, Direct Volume Rendering)



Die Intensität
des Pixels
nimmt
proportional mit
der Entfernung
des Voxels ab.

LITERATUR: A. Kaufman, K. Mueller, Volume Visualization and Volume Graphics,
Center for Visual Computing, Computer Science Department, Stony Brook
University <http://www.volumegraphics.com/products/vgl>